



Administrator Training Manual

SCRIPTING USERHOOKS

Infosys

www.infosys.com/finacle

Document number:		VersionRev:	1.4
Authorized by:	R N NAGARAJ	Signature/Date:	

Document revision list

Ver.Rev	Date	Author	Description
1.00	14 July 2000	Balaji. S	<i>Original version</i>
1.1	20 July 2001	Sameer Shirgurkar	<i>Updated to 6.2</i>
1.2	17 June 2002	Balaji. S	<i>Added the Userbooks of 6.2</i>
1.3	17-Feb-2003	R.K.Agrawal	<i>Updated to Version 6.2.1.09</i>
1.4	25-04-2004	Anju Jocelin Mathew	<i>Updated to Fin 7.0</i>

© 2005 Infosys Technologies Limited, Bangalore, India

All rights reserved by
Infosys Technologies Limited,
Electronics City ,
Hosur Road,
Bangalore – 560 100
India

No part of this volume may be reproduced or transmitted in any form or by any means electronic or mechanical including photocopying and recording or by any information storage or retrieval system except as may be expressly permitted.

This Training manual has been written and produced by the *FINACLE -USER EDUCATION TEAM* of Infosys Technologies Limited.

Infosys believes that the information in this publication is accurate as of its publication date. This document could include typographical errors, omissions or technical inaccuracies. Infosys reserves the right to revise the document and to make changes without notice. Infosys acknowledges the proprietary rights in the trademarks and product names of other companies mentioned in this document.

TABLE OF CONTENTS

1	<i>SCRIPT ENGINE BASED CUSTOMIZATION</i>	1
1.1	WHAT IS ‘SCRIPT ENGINE BASED CUSTOMISATION’?	1
1.2	WHAT IS A FINACLE USER HOOK?	2
1.3	REPOSITORIES AND CLASSES	2
1.4	RETURN STATUS OF THE FUNCTION.....	2
2	<i>GENERAL SCRIPTING FUNCTIONS</i>	2
2.1	VALIDATING THE ACCOUNT.....	3
2.2	GET THE ACCOUNT DETAILS INTO BANCS REPOSITORY	4
2.3	GET THE SCHEME DETAILS INTO BANCS REPOSITORY	8
2.4	GETTING THE LOCATION OF A FILE IN THE PATH.....	9
2.5	BACID TO ACCT ID AND NAME	10
2.6	CONVERT AN AMOUNT FROM ONE CURRENCY TO ANOTHER	12
2.7	ROUND OFF AN AMOUNT	13
2.8	GET AMOUNT CALCULATED BASED ON SET-UP IN AMOUNT CODE TABLE	15
2.9	GET LIST OF ACCOUNTS BASED ON INSTRUMENT NUMBER	16
2.10	VALIDATE BANK CODE	17
2.11	VALIDATE BANK AND BRANCH CODE.....	18
2.12	VALIDATE HO TRANSACTION CATEGORY CODE	19
2.13	VALIDATE NUMERIC.....	20
2.14	VALIDATE DATE	21
2.15	VALIDATE REFERENCE CODE.....	22
2.16	GET 4 DIGIT YEAR.....	22
2.17	DATE ADD.....	23
2.18	SELECTION OF FIELDS FROM DATABASE.....	24
2.19	USING A SQL STATEMENT IN A SCRIPT	25
2.20	DISPLAY REPOSITORY FIELDS.....	26
2.21	ADVANCE DETAILS OF AN ACCOUNT IN BANCS REPOSITORY	26
2.22	CUSTOMER DETAILS OF A CUSTOMER IN BANCS REPOSITORY :	28
2.23	ASSET CLASSIFICATION DETAILS OF AN ACCOUNT IN BANCS REPOSITORY :	31
2.24	INTEREST CALCULATION DETAILS FOR AN ACCOUNT IN BANCS REPOSITORY :	33
2.25	ACCOUNT BALANCE AS ON A GIVEN DATE:.....	35

2.26	INTEREST RATE FOR AN ACCOUNT ON A GIVEN DATE IN BANCs REPOSITORY	36
2.27	SUM CREDIT AND DEBIT TRANSACTION AMOUNT OVER A PERIOD FOR AN ACCOUNT	37
2.28	SUM TRANSACTION AMOUNT FOR A SPECIFIED ACCOUNT ON A ACCOUNT LABEL.....	38
2.29	SECURITY DETAILS FOR A GIVEN ACCOUNT NUMBER	39
2.30	CHECK FOR PIPE CHAR IN THE INPUT STRING.	41
2.31	CHECK WHETHER THE ACCOUNT IS A CASH ACCOUNT.	41
2.32	CHECK WHETHER THE ACCOUNT IS A SYSTEM ONLY ACCOUNT.	42
2.33	CHECK WHETHER THE ACCOUNT IS FCNR ACCOUNT.	42
2.34	CHECK WHETHER THE ACCOUNT IS HO ACCOUNT.	43
2.35	VALIDATE THE AMOUNT WITH CURRENCY	43
2.36	GIVEN A DATE GET THE DATE DIFF FROM BOD.....	44
2.37	VALIDATE CRNCY CODE.....	44
2.38	VALIDATE TRAN REPORT CODE FOR THE GIVEN ACCOUNT.....	45
2.39	GET THE SCHM DETAILS FOR A GIVEN DD SCHM CODE AND CURRENCY.....	45
2.40	VALIDATE INSTRUMENT DATE.....	47
2.41	ADD A USER DEFINED MESSAGE INTO IBR FOR RECONCILIATION	48
2.42	GETDIFFBETWEENTWOFREQ	49
2.43	B2K_ISCHECKSUMRIGHT.....	49
2.44	URHK_ GETACCTTURNOVERDETAILS.....	49
2.45	GET THE PID OF THE PROCESS	49
2.46	SLEEP.....	50
2.47	VALIDATE INSTRUMENT FOR AN ACCOUNT	51
2.48	VALIDATION OF SOLID	51
2.49	DBC_CURSOROPEN.....	52
2.50	DBC_CURSORFETCH	52
2.51	DBC_CURSORCLOSE	53
2.52	GET DSA TURNOVER DETAILS FOR A SCHEME TYPE.....	54
2.53	GET THE VALUE OF ENVIRONMENT VARIABLE	56
2.54	GETTING INSURANCE DETAILS FOR A SECURITY	57
2.55	ISSUE CHEQUE LEAVES	58
2.56	CONVERTS THE LOWER STRING TO UPPER CASE.....	59
2.57	UPDATE LIEN	59
2.58	USER DEFINED TOD.....	60
2.59	NEXT NUMBER GENERATION.....	61

3	<i>ONLINE SCRIPTING FUNCTIONS.....</i>	<i>64</i>
3.1	SET THE DEFAULT ATTRIBUTE OF A FIELD IN THE FORM.....	64
3.2	SET THE DEFAULT VALUE FOR A FIELD IN THE FORM DURING FORM LOAD.....	66
3.3	CHANGE THE FIELD VALUE IN THE FORM.....	68
3.4	INQUIRE THE FIELD VALUE IN THE FORM.....	69
3.5	CHANGE THE ATTRIBUTE OF A FIELD IN THE FORM.....	72
3.6	GET THE ‘POST-COMMIT’ VALUE FROM A FIELD IN THE FORM.....	73
3.7	EXECUTING A KEY-BASED EVENT FROM THE SCRIPT.....	75
3.8	CALLING GENERAL PARAMETER ACCEPTANCE FORM.....	79
3.9	GET THE NAME OF THE CURRENT FIELD IN THE CURRENT FORM.....	85
3.10	DISPLAY A MESSAGE AT THE BOTTOM LINE IN A FORM.....	85
3.11	SPECIFY ADDITIONAL ACTIONS FOR A FUNCTION KEY AT ANY FIELD	86
4	<i>WORKFLOW SCRIPTING FUNCTIONS.....</i>	<i>88</i>
4.1	EXECUTING A MENU OPTION	88
4.2	QUEING UP A NEW WORKFLOW ITEM FROM WITHIN A WORKFLOW SCRIPT	90
4.3	FORWARD THE WORKFLOW (TRANSFER TO SOME OTHER USER).	94
4.4	CALLING INPUT ACCEPTANCE FORM	95
4.5	SET CHECK POINT TO RESTART THE SCRIPT IN CASE OF FATAL / RUN TIME ERRORS.....	99
4.6	SET WORK GROUP OF A USER FOR USING WORK FLOW.....	101
4.7	WFSRE_EXECUTE	101
4.8	WFSRE_LOGON.....	102
4.9	WFSRE_LOGOUT.....	104
5	<i>CLEARING SCRIPTING FUNCTIONS.....</i>	<i>105</i>
5.1	GET DETAILS FROM CTC TABLE FOR A TRANSACTION CODE	105
5.2	GET INWARD CLEARING ZONE HEADER DETAILS	106
5.3	GET OUTWARD CLEARING ZONE HEADER DETAILS.....	107
5.4	GET OUTWARD CLEARING INSTRUMENT DETAILS	109
5.5	GET OUTWARD CLEARING PART TRAN DETAILS	112
6	<i>MTTS SCRIPTING FUNCTIONS</i>	<i>114</i>
6.1	GET THE INPUT DETAILS OF THE MTT EVENT	114
6.2	DEFINING A FINANCIAL TRANSACTION	114
6.3	DEFINING A FINANCIAL PART-TRANSACTION.....	115
6.4	END THE DEFINITION OF A TRANSACTION/PART-TRANSACTION.....	117
6.5	REPORTING AN ERROR CONDITION	118

6.6	CHECK IF CUSTOMER INSTRUCTION EXISTS	118
6.7	DISTRIBUTE AMOUNT AS PER CUSTOMER INSTRUCTION.....	119
6.8	MTTS_DEFINECHARGEPARTTRAN	120
6.9	MTTS_INITPTT	122
6.10	MTTS_GETPTTREC	123
7	<i>USER HOOKS TO RAISE EXCEPTIONS FOR FINANCIAL TRANSACTIONS</i>	125
7.1	RAISE EXCEPTIONS FOR TRANSACTION CREATED THROUGH TM.....	125
7.2	RAISE EXCEPTIONS FOR CLEARING TRANSACTIONS	127
8	<i>USER HOOKS FOR QUEUING UP WORKFLOW ITEMS FROM WITHIN A NON-WORKFLOW SCRIPT</i>	128
8.1	INITIALIZING THE WORK FLOW REPOSITORIES IN A SPECIFIC EVENT HOOK	128
8.2	CREATING WORK FLOW ITEM FROM SPECIFIC EVENTS.....	130
9	<i>USER HOOKS FOR BANCSCONNECT</i>	132
9.1	DETERMINING IF THE BANCSCONNECT ONLINE FEES IS TO BE TREATED AS CHARGE INCOME OR NOT.....	132
9.2	GENERATE HASHED NUMBER FOR BANCSCONNECT MESSAGES	133
9.3	CREATE DCTI RECORD	133
10	<i>USER HOOKS TO GENERATE PAST DUE DETAILS INQUIRY/REPORT</i>	135
10.1	GET PAST DUE RECORD	135
10.2	PUT PAST DUE DISPLAY/PRINT STRING	136
10.3	GENERATE PAST DUE REPORT	139
11	<i>TRADE FINANCE RELATED HOOKS</i>	141
11.1	GET USER ADDITIONAL DETAILS	141
11.2	SAVE USER ADDITIONAL DETAILS	142
11.3	RAISING EXCEPTIONS	142
11.4	GETTING ADDITIONAL INPUT DETAILS IN MTT.....	143
11.5	FXBILL_GETTRAN.....	144
11.6	FXBILL_INITTRAN	145
12	<i>DSA RELATED USER HOOKS</i>	145
12.1	URHK_GETDSATURNOVERDETAILS	145
12.2	URHK_GETDSAMASTERDETAILS	146
12.3	URHK_GETDSAPRODUCTDETAILS	146
12.4	URHK_GETDSAPARAMDETAILS	146
12.5	URHK_GETPASTDUECYCLES.....	146
12.6	UPDATE DSA TURNOVER DETAILS	146

12.7	GET TOTAL SANCTION LIMIT OF ALL ACCOUNTS LINKED TO A DSA	147
12.8	GET DSA TURNOVER DETAILS FOR A SCHEME TYPE.....	148
13	DATE FUNCTIONS.....	151
13.1	ADDING DAYS AND MONTHS IN A GIVEN DATE.....	151
13.2	DIFFERENCE BETWEEN TWO DATES.....	151
13.3	GETNEXTWORKINGDATE.....	152
13.4	GETTING THE HOLIDAY STATUS FOR A GIVEN DATE.....	153
14	INDEX OF USER HOOKS	155

1 SCRIPT ENGINE BASED CUSTOMIZATION

1.1 WHAT IS 'SCRIPT ENGINE BASED CUSTOMISATION'?

Finacle is a Core Banking package which supports a number of deployment topologies (Fully Distributed to Fully Centralized) and Implementation scenarios (Single currency, Multiple currency, Various Retail and Trade finance banking products and Interest methods etc.). In order to support these multitudes of features, it is essential that Finacle is open to 'Customization' by the Bank so that it can implement the features that it requires and in the way that it desires. Also, the Bank will need to interface some of their 'peripheral applications' (like Treasury, Consumer Finance etc.) which are not supported directly by Finacle, to Finacle in various ways. Another area of 'Customizability' is in the interface to various special purpose equipment like ATM switches, VRUs, MICR encoders etc. which are supplied by many vendors with some differences in actual implementation.

So far, Finacle had addressed this issue by defining 'parameters' which was set up by the Bank depending upon its requirements. However, this has the limitation of requiring that the developers of Finacle know all the possible business logic before hand and then implement them using parameters. However, a far more powerful approach is to allow the Bank to 'take control ' at certain 'events', apply their own logic on the data associated with that event and be able to either influence the outcome of that event or communicate the occurrence of the event to other 'Custom' applications. The Finacle 'Script Engine based customization' allows the bank to do just this.

The Script Engine Based Customization (**SEBC**) consists of 3 parts:

- 1) The Script Engine – This is an 'interpretive language' processor which allows a wide range of key programming constructs, while allowing the Bank to 'custom develop' additional functions that can be called from the 'scripts'.
- 2) The Finacle functions – Certain standard functions and certain 'module specific' functions have been developed by the Finacle developers which can be called from the 'scripts'. This document contains more details about this.
- 3) The Finacle 'script events' – Right across Finacle, there are several events that have been 'opened up' by defining script events. What it means is that for a 'script-enabled event', the Finacle application, checks for the existence of a defined script and if present, transfers control to the Script Engine to execute that script after populating all the data required by that event in 'input fields'. It is the script's responsibility to apply whatever logic it wants and provide the necessary 'output fields' at the end of the script for Finacle to continue its processing. Please refer to Scripting Events document for more details.

1.2 WHAT IS A FINACLE USER HOOK?

The document on scripting Syntax describes the Finacle Script Engine capabilities and the Finacle scripting language syntax. This document describes all such functions that are provided in Finacle, which can be used while writing a Finacle script. Some of these functions are general and can be used in the context of any of the Script Events described in scripting Events document or in any Workflow script. However, certain functions are specific either to the Workflow context or to a specific event. In such a case it will be documented as such along with its description below.

The general format for describing each function is as follows:

Brief description of the function and context in which it can be used.

Function syntax – (Also gives change log between Finacle versions)

Detailed functionality –

Description of the Input fields and Output fields –

Example of the usage of the function –

1.3 REPOSITORIES AND CLASSES

Please refer to the document on scripting syntax for a general discussions on Repositories and classes.

All Finacle user hooks (except for Workflow related functions and MTT related functions) interface with the scripts using a standard repository called "BANCS". There are 2 classes that have been predefined in this repository, INPARAM and OUTPARAM each of which hold 'String' type of fields. INPARAM is the class that is used to populate specified fields (depending upon the function) in the script so that the function can access those as parameters. All the fields (depending upon the function) that are output by the function are populated in the OUTPARAM class.

In addition to INPARAM class, the input to the functions can also be provided as arguments to the function. Please note that the INPARAM class is cleared out once the user hook is executed and needs to be repopulated by the script when calling another user hook or the same user hook in a loop.

1.4 RETURN STATUS OF THE FUNCTION.

All functions return a value of either SUCCESS (0) or FAILURE (1).

2 GENERAL SCRIPTING FUNCTIONS

These scripting user hooks can be used in the context of any script.

2.1 VALIDATING THE ACCOUNT

This function validates a given account number in the FINACLE database and returns a value depending on whether the account number is valid or not. It can be used in the context of any Script Event or Workflow.

SYNTAX

The Syntax for calling the function is

`sv_a = urhk_valAcctNumber (Variable)`

The variable can be a scratch pad variable (sv_b) or a string ("SBGEN1").

FUNCTIONALITY

This function validates whether a given account number (Foracid) exists in the datacenter database or not. It simply checks for the presence of the account and does not distinguish between any status of the account (for e.g. closed or not, office a/c or customer a/c etc.). If more information about an account is required please refer to the function `urhk_getAcctDetailsInRepository` in this document.

INPUT

The input to this function is an account number (FORACID).

OUTPUT

The return value. This value will be 0 in case a valid account number with the input string is found. Otherwise the return value is 1.

EXAMPLE

As a result of moving from distributed to a Flex-centralised topology, some banks may desire to change the account numbering format. However, external references to old accounts will still exist. Hence there may be situations where a Finacle user may enter an old account number on the screen. Now, if the new account number is derivable from the old account number (either by keeping a mapping file or by applying a simple transformation) we can code the `formatAcct.scr` as follows to ensure that the user does not get an 'invalid account' message when he enters the old account number.

```

<--start
sv_s = BANCS.INPUT.uAccount
sv_i = urhk_valAcctNumber(sv_s)
if (sv_i = 0) then                                /* Account exists */
    BANCS.OUTPUT.fAccount = sv_s
    EXITSCRIPT
else                                              /* Account does not exist */
    /* apply new account number derivation logic here */
    BANCS.OUTPUT.fAccount = Result of above logic
endif
exitscript
end-->

```

2.2 GET THE ACCOUNT DETAILS INTO BANCS REPOSITORY

This function returns information about a given account number in the FINACLE database. It can be used in the context of any Script Event or Workflow.

SYNTAX

```
sv_a = urhk_getAcctDetailsInRepository(Variable)
```

The variable can be a scratch pad variable (sv_b) or a string ("SBGEN1").

FUNCTIONALITY

This function checks whether the given account number exists in the datacenter database. If the account exists (irrespective of the state of the account), then some attributes of the account, as detailed below, are returned in repository fields.

INPUT

The input to this function is an account number (FORACID).

OUTPUT

Return value is 0, if account found, else 1.

This function will return the following Account details into OUTPARAM class of BANCS repository if the account exists in the datacenter database.

Field Name in GAM table	Field name in BANCS repository
sol_id	BANCS.OUTPARAM.acctSolId
Bacid	BANCS.OUTPARAM.acctPlaceholder
acct_name	BANCS.OUTPARAM.acctName
cust_id	BANCS.OUTPARAM.custId
emp_id	BANCS.OUTPARAM.empId
gl_sub_head_code	BANCS.OUTPARAM.glSubHeadCode
acct_ownership	BANCS.OUTPARAM.acctOwnerShip (C'ustomer, E'mployee, O'ffice account)
schm_code	BANCS.OUTPARAM.schmCode
schm_type	BANCS.OUTPARAM.schmType
acct_opn_date	BANCS.OUTPARAM.acctOpenDate
acct_cls_date	BANCS.OUTPARAM.acctCloseDate
acct_cls_flg	BANCS.OUTPARAM.acctCloseflg (Y/N)

Field Name in GAM table	Field name in BANCS repository
mode_of_oper_code	BANCS.OUTPUTPARAM.modeOprnCode
acct_locn_code	BANCS.OUTPUTPARAM.acctLocnCode
acct_crncy_code	BANCS.OUTPUTPARAM.acctCrncyCode
system_only_acct_flg	BANCS. OUTPUTPARAM.systemOnlyAcctFlg (Y/N)
Foracid	BANCS.OUTPUTPARAM.AcctId
dr_bal_lim	BANCS.OUTPUTPARAM.debitBalanceLimit
frez_code	BANCS.OUTPUTPARAM. freezeCode
frez_reason_code	BANCS.OUTPUTPARAM. freezeReasonCode
clr_bal_amt	BANCS.OUTPUTPARAM. clearBalance
un_clr_bal_amt	BANCS.OUTPUTPARAM. unclearBalance
ledg_num	BANCS.OUTPUTPARAM. ledgerNumber
drwng_power	BANCS.OUTPUTPARAM. drawingPower
sanct_lim	BANCS.OUTPUTPARAM. sanctionLimit
adhoc_lim	BANCS.OUTPUTPARAM. adhocLimit
emer_advn	BANCS.OUTPUTPARAM. emergencyAdvance
dacc_lim	BANCS.OUTPUTPARAM. DACCLimit
system_reserved_amt	BANCS.OUTPUTPARAM. systemReservedAmt
single_tran_lim	BANCS.OUTPUTPARAM. singleTranLimit
clean_adhoc_lim	BANCS.OUTPUTPARAM. cleanAdhocLimit
clean_emer_advn	BANCS.OUTPUTPARAM. cleanEmergencyAdvance
clean_single_tran_lim	BANCS.OUTPUTPARAM. cleanSingleTranLimit
system_gen_lim	BANCS.OUTPUTPARAM. systemGeneratedLimit
chq_alwd_flg	BANCS.OUTPUTPARAM. chequeAllowed
cash_excp_amt_lim	BANCS.OUTPUTPARAM. cashExceptionAmtLimit
clg_excp_amt_lim	BANCS.OUTPUTPARAM. clearingExceptionAmtLimit
xfer_excp_amt_lim	BANCS.OUTPUTPARAM. transferExceptionAmtLimit
cash_cr_excp_amt_lim	BANCS.OUTPUTPARAM. cashCrExceptionAmtLimit
clg_cr_excp_amt_lim	BANCS.OUTPUTPARAM. clearingCrExceptionAmtLimit
xfer_cr_excp_amt_lim	BANCS.OUTPUTPARAM. transferCrExceptionAmtLimit
cash_abnrml_amt_lim	BANCS.OUTPUTPARAM. cashAbnormalAmtLimit
clg_abnrml_amt_lim	BANCS.OUTPUTPARAM. clearingAbnormalAmtLimit
xfer_abnrml_amt_lim	BANCS.OUTPUTPARAM. TransferAbnormalAmtLimit
acrd_cr_amt	BANCS.OUTPUTPARAM. accruedCreditAmt
pb_ps_code	BANCS.OUTPUTPARAM. passbookOrPasssheet
serv_chrg_coll_flg	BANCS.OUTPUTPARAM. serviceChrgCollectedFlg
int_paid_flg	BANCS.OUTPUTPARAM. interestPaidFlag
int_coll_flg	BANCS.OUTPUTPARAM. interestCollectedFlag
limit_prefix	BANCS.OUTPUTPARAM. limitPrefix
limit_suffix	BANCS.OUTPUTPARAM. limitSuffix
drwng_power_ind	BANCS.OUTPUTPARAM. drawingPowerIndicator
drwng_power_pcmt	BANCS.OUTPUTPARAM. drawingPowerPercentage
notional_rate	BANCS.OUTPUTPARAM. notionalRate
notional_rate_code	BANCS.OUTPUTPARAM. notionalRateCode
fx_clr_bal_amt	BANCS.OUTPUTPARAM. FXClearBalance
crncy_code	BANCS.OUTPUTPARAM. FCNRCrncyCode
wtax_flg	BANCS.OUTPUTPARAM. TaxFlg
wtax_amount_scope_flg	BANCS.OUTPUTPARAM. TaxAmtScopeFlg
lien_amt	BANCS.OUTPUTPARAM. lienAmt
acct_mgr_user_id	BANCS.OUTPUTPARAM. accountManagerUserId
schm_type	BANCS.OUTPUTPARAM. schemeType
Partitioned_flg	BANCS.OUTPUTPARAM. partitionedFlg
Partitioned_type	BANCS.OUTPUTPARAM. partitionedType
Not in GAM table (Sum of	BANCS.OUTPUTPARAM AvailableAmt

Field Name in GAM table	Field name in BANCS repository
different limits)	
Not in GAM table (Sum of different limits)	BANCS.OUTPUTPARAM FxAvailableAmt
Not in GAM table (Sum of different limits)	BANCS.OUTPUTPARAM FFDAvailableAmt
Not in GAM table (Sum of different limits)	BANCS.OUTPUTPARAM FxFFDAvailableAmt
Not in GAM table (Sum of different limits)	BANCS.OUTPUTPARAM EffAvailableAmt
Not in GAM table (Sum of different limits)	BANCS.OUTPUTPARAM FxEffAvailableAmt
Not in GAM table (Sum of different limits)	BANCS.OUTPUTPARAM FullAvailableAmt
Not in GAM table (Sum of different limits)	BANCS.OUTPUTPARAM FxFullAvailableAmt
Not in GAM table (Sum of different limits)	BANCS.OUTPUTPARAM FullEffAvailableAmt
Not in GAM table (Sum of different limits)	BANCS.OUTPUTPARAM FxFullEffAvailableAmt

EXAMPLE:

Let us say that the bank does not want the account balance of 'Office accounts' to be displayed in the lower block of the 'Transaction Maintenance' screen when doing a transaction (eg: *this information should not be available to the Teller*). Then the CheckAndDispAcctBal.scr script can be coded as follows:


```

<--start

sv_a = BANCS.INPUT.AcctId

# -----
# - sv_a (account id) is the input to user hook getAcctDetailsInRepository
# - Call user hook getAcctDetailsInRepository to put the Account details into BANCS
# repository.
# -----

sv_b = urhk_getAcctDetailsInRepository(sv_a)

# if getAcctDetailsInRepository function returns failure exit out of script.
# This condition should not occur since BANCS.INPUT.AcctId would
# already have been validated to exists when this script is invoked.

if( sv_b == 1 ) then
    exitscript
endif

sv_c="0"
# acctOwnership flag is "0" for office accounts

# -----
# If the acctOwnership flag of ACCOUNT class is equal to "0"
# then don't display dispblk2.acct_bal field.
# -----

if (BANCS.OUTPUT.acctOwnerShip == sv_c) then
    sv_z = "dispblk2.acct_bal|H"
    sv_z = urhk_TBAF_ChangeFieldAttrib(sv_z)
else
    sv_z = "dispblk2.acct_bal|U"
    sv_z = urhk_TBAF_ChangeFieldAttrib(sv_z)
endif
exitscript
end-->

```

Note: Refer to urhk_TBAF_ChangeFieldAttrib given separately for its usage

```

# -----
# If the tod amt changes at the time of posting (InstantTodAmt
# and TodLvlIntFlg are inputs to the script),
# raise exception if the difference is more than 100
# -----
sv_e = cdouble(BANCS.INPUT.InstantTodAmt)
if (sv_e != 0.00 ) then
    sv_a = BANCS.INPUT.AcctNum
    sv_b = urhk_getAcctDetailsInRepository(sv_a)
    if (sv_b == 1) then
        BANCS.OUTPUT.OutString = "Account number is invalid"
        BANCS.OUTPUT.successOrFailure="F"
        exitscript
    endif
    sv_c = cdouble(BANCS.OUTPUT.AvailableAmt)
    sv_d = cdouble(BANCS.OUTPUT.FullEffAvailableAmt)
    if ( sv_d < 0 ) then
        sv_d = 0
    endif
    sv_f = cdouble(BANCS.OUTPUT.FullAvailableAmt)
    sv_g = BANCS.INPUT.TodLvlIntFlg
    sv_h = cdouble(BANCS.INPUT.TranAmt)
    sv_i = ( sv_h - sv_d )
    sv_j = ( sv_e - 100 )
    sv_k = ( sv_e + 100 )
    if ( ( sv_i < sv_j ) OR ( sv_i > sv_k ) ) then
        BANCS.OUTPUT.successOrFailure = "F"
        BANCS.OUTPUT.ErrorDesc = "Error: TOD Amt. is different from that of
specified value"
    endif
endif
endif

```

2.3 GET THE SCHEME DETAILS INTO BANCS REPOSITORY

This function returns scheme details information in the FINACLE database. It can be used in the context of any Script Event or Workflow.

SYNTAX

sv_a = urhk_getSchemeDetailsInRepository (Variable)

The variable can be a scratch pad variable (sv_b) or a string("SBGEN").

FUNCTIONALITY

This function will return some attributes of the given scheme in the repository fields as detailed below if the scheme is found. If the scheme does not exist, then a fatal error occurs. Hence the scheme code passed must be a valid scheme code.

INPUT

The input to this function is a valid (existing) SCHEME CODE.

OUTPUT

Return value is always 0. If scheme not found then Fatal Error will result.

This function will return the following Scheme details into OUTPARAM class of BANCS repository if the scheme exists in the datacenter database.

Field Name in GSP table	Field name in BANCS repository
schm_desc	BANCS.OUTPARAM. SchemeDescription
int_paid_bacid	BANCS.OUTPARAM. InterestPaidPlaceHolder
int_coll_bacid	BANCS.OUTPARAM. InterestCollPlaceHolder
serv_chrg_bacid	BANCS.OUTPARAM. ServiceChargePlaceHolder
chq_alwd_flg	BANCS.OUTPARAM. SchmChequeAllowed
schm_supr_user_id	BANCS.OUTPARAM. SchemeSupervisorUserId
new_acct_duration	BANCS.OUTPARAM. NewAccountDuration
dorm_acct_criteria	BANCS.OUTPARAM. DormantAccountCriteria
min_posting_workclass	BANCS.OUTPARAM. MinPostingWorkClass
schm_type	BANCS.OUTPARAM. SchemeType
int_paid_flg	BANCS.OUTPARAM. InterestPaidFlg
int_coll_flg	BANCS.OUTPARAM. InterestCollectedFlg
staff_schm_flg	BANCS.OUTPARAM. StaffSchemeFlg
nre_schm_flg	BANCS.OUTPARAM. NRE_SchemeFlg
fd_cr_bacid	BANCS.OUTPARAM. FD_CreditPlaceHolder
fd_dr_bacid	BANCS.OUTPARAM. FD_DebitPlaceHolder
stp_cr_dr_ind	BANCS.OUTPARAM. StopCrDrInd
fcnr_flg	BANCS.OUTPARAM. IsFCNR
int_pandl_bacid	BANCS.OUTPARAM. Int_P_and_L_PlaceHolder
Parking_bacid	BANCS.OUTPARAM. ParkingPlaceHolder
Advance_int_bacid	BANCS.OUTPARAM. AdvanceIntPlaceHolder

tds_parking_bacid	BANCS.OUTPARAM. TDS_ParkingPlaceholder
penal_pandl_bacid	BANCS.OUTPARAM. Penal_P_and_L_PlaceHolder
dflt_clg_tran_code	BANCS.OUTPARAM. DefaultClearingTranCode
dflt_inst_type	BANCS.OUTPARAM. DefaultInstrumentType
int_pandl_bacid_cr	BANCS.OUTPARAM. Int_P_and_L_CrPlaceholder
int_pandl_bacid_dr	BANCS.OUTPARAM. Int_P_and_L_DrPlaceholder

EXAMPLE:

Let us say that banker wants to change a certain scheme which is fcnr to change later to non fcnr so the fcnr flg field should be unprotected and if it is non fcnr the field should be protected.

```
<--start

sv_a = BANCS.INPUT. SchmCode

# -----
# sv_a (SchmCode) is the input to user hook getSchemeDetailsInRepository
# Call user hook getSchemeDetailsInRepository to put the Scheme details
# into BANCS repository.
# -----

sv_b = urhk_getSchemeDetailsInRepository (sv_a)

# FCNR flag is "Y" for fcnr type of schemes.

# -----
# If the FCNR flag of scheme code class is equal to "Y"
# then protect dispblk1.fcnr_flg field.
# -----

if (BANCS.OUTPARAM. IsFCNR == "Y") then
    sv_z = "dispblk1.fcnr_flg|U"
    sv_z = urhk_TBAF_ChangeFieldAttrib(sv_z)
else
    sv_z = "dispblk1.fcnr_flg|P"
    sv_z = urhk_TBAF_ChangeFieldAttrib(sv_z)
endif
exitscript

end-->
```

 **Note:** Refer to *urhk_TBAF_ChangeFieldAttrib* given separately for its usage.

2.4 GETTING THE LOCATION OF A FILE IN THE PATH

This function is used to get the path of a file by specifying the file name.

SYNTAX

```
sv_a = urhk_getFileLocation(sv_b)
```

The variable can be a scratch pad variable (sv_b) or a string ("<file type>|" + <filename>")

FUNCTIONALITY

This function will return the full path of the file name specified as input. It can typically be used in scripts to get the location of a script file, which needs to be called from within another script. The search for the file will be carried out according to the site-customization directory search rules.

INPUT

Input through INPARAM repository :

NONE

Input String as function argument :

contains two parts separated by a '|' character.

1. File Type – The type of the file whose path is required. The possible values for this are: "FORM", "EXE", "SCRIPT", "MRT", "SQL", "COM" and "MENU".

2. File Name – The name of the file whose path is required.

Both the input parameters are mandatory.

Example – "SCRIPT|EVTWFS.scr"

OUTPUT

Output through OUTPARAM repository:

The user hook returns the path of the file in the variable "fileLocation".

This can be accessed as follows:

```
sv_d = BANCS.OUTPARAM.fileLocation
```

Function return value:

Returns 1 if path is successfully got else returns 0.

EXAMPLE:

```
<--start
sv_a = STDWFS.WFSINPUT.CallScriptName
sv_b = "SCRIPT" + "|" + sv_a
sv_c = urhk_getFileLocation(sv_b)
sv_d = BANCS.OUTPARAM.fileLocation
START(sv_d, sv_a)
exitscript
end-->
```

2.5 BACID TO ACCT ID AND NAME

This function returns the Account Id and name of the account that is identified via a BACID, Currency code and Sol Id, if such an account exists. This is a subset of

getAcctDetailsInRepository and may be removed in future versions. It is usually used as a part of a script where the Bacid, Currency and Sol Id are known (or are obtained) before this user hook is called.

SYNTAX

The Syntax for calling the function is

```
sv_r = urhk_B2k_BacidToAcctId (Variable)
```

The variable can be a scratch pad variable (sv_b) or a string ("PURCH|INR|123456").

FUNCTIONALITY

Get the account id for passed Bacid, Currency and Sol Id.

INPUT

Input String contains three parts separated by a '|'

1. Bacid
2. Currency code
3. Sol id

Example – "PURCH|INR|123456"

OUTPUT

Return value will be 1 if no account is found, else it will be 0 and fields AcctName and AcctId of OUTPARAM class of BANCS repository will be populated for the deduced account.

Field Name in GAM table	Field name in BANCS repository
sol_id	BANCS.OUTPARAM.acctSolId
Bacid	BANCS.OUTPARAM.Bacid
CrncyCode	BANCS.OUTPARAM.CrncyCode
AcctName	BANCS.OUTPARAM.AcctName
AcctId	BANCS.OUTPARAM.AcctId

EXAMPLE:

This user hook is a subset of getAcctDetailsInRepository user hook. It is used to manipulate only acctName and acctId (Foracid). In the example below, the bank has named all system cash accounts as "SYSTEM CASH" and wants these accounts to be skipped in the script.

```

<--start

...
# The above statements set the context of Bacid, Currency Code and Foracid
# in variables sv_b, sv_c and sv_f respectively for this example.

sv_a = sv_b + "|" + sv_c + "|" sv_f

# -----
# - sv_a (see Input section above) is the input to user hook
#   B2k_BacidToAcctId
# - Call user hook B2k_BacidToAcctId to put the account name and foracid
#   in the repository
# -----

sv_r = urhk_B2k_BacidToAcctId (sv_a)

# if B2k_BacidToAcctId function returns failure exit out of script.

if( sv_r == 1 ) then
    exitscript
endif

sv_s ="SYSTEM CASH"
# All cash accounts in various currencies have this name
# -----
# acctName is compared
# If system cash, the script skips the account
# -----

if (BANCS.OUTPARAM.acctName == sv_s) then
    GOTO lblSkipSystemCash
else
    ...
    # do the rest of the processing as desired
endif
end-->

```

2.6 CONVERT AN AMOUNT FROM ONE CURRENCY TO ANOTHER

Given an input amount, 'from currency', 'to currency' and either a rate code or conversion rate, this function gives the equivalent amount in the 'to currency'.

SYNTAX

The Syntax for calling the function is

```
sv_r = urhk_B2k_ConvertAmount("")
```

FUNCTIONALITY

This function applies the same logic as Finacle in determining equivalent amounts in different currencies.

INPUT

All the input variables should be populated in the INPARAM class of the BANCS repository. The variables are (all STRINGS):

InputAmount	The amount that needs to be converted
FromCurrency	The currency that the input amount is in
ToCurrency	The currency in which the equivalent amount is needed
RateCode	The rate code to be used during conversion. Will be used only if Rate is "0"
Rate	The conversion rate to be used for the equivalence Calculation, if not "0".

OUTPUT

Function returns 0 or 1. If 1, then there was failure in conversion. If 0, all output variables will be populated in the OUTPARAM class of the BANCS repository. The variables are (all STRINGS):

OutputAmount - The equivalent amount in 'to currency'

Rate - The conversion rate actually used (in case INPARAM.RateCode is used, INPARAM.Rate is zero).

EXAMPLE:

```
<--start
sv_a = "USD"
#From Currency
sv_b = "GBP"
#To Currency
sv_c = "TCB"
# Want to get the rate for Travellers Cheque buying
...
...
# sv_p contains the amount to be converted

BANCS.INPARAM.InputAmount = sv_p
BANCS.INPARAM.FromCurrency = sv_a
BANCS.INPARAM.ToCurrency = sv_b
sv_r = urhk_B2k_ConvertAmount("")
if (sv_r ==1) then
    # Error processing
    ...
endif
sv_q = BANCS.OUTPARAM.OutputAmount
# sv_q contains the converted amount
sv_r = BANCS.OUTPARAM.Rate
# sv_r contains the TC buying rate
end-->
```

2.7 ROUNDOFF AN AMOUNT

Given an input amount, 'round off to' amount, and roundoff flag, this function gives the roundedoff amount.

SYNTAX

The Syntax for calling the function is

```
sv_r = urhk_B2k_RoundOff ("" )
```

FUNCTIONALITY

This function applies the same logic as Finacle in roundingoff the amount.

INPUT

All the input variables should be populated in the INPARAM class of the BANCS repository. The variables are(all STRINGS) :

InputAmount - The amount that needs to be converted

RoundOffAmt - The amount to be rounded off to

Example : If the amount is in Rupees ...

100 - 100 Rupees

.01 - 1 Paise

RoundOffFlag - Valid values are:

H - roundoff to the next higher amount

L - roundoff to the previous lower amount

N - roundoff to the nearest amount

OUTPUT

Function returns 0 or 1. If 1, then there was failure in roundingoff. If 0,

All output variables will be populated in the OUTPARAM class of the BANCS repository.

The variables are(all STRINGS) :

OutputAmount - The roundedoff amount

EXAMPLE:

```
<--start
trace on
sv_a = 10986792.2358
#Input Amount
sv_b = 100
# Round off to 100 Rupees
sv_c = "L"
# Lower amount

BANCS.INPARAM.InputAmount = sv_a
BANCS.INPARAM.RoundOffAmt = sv_b
BANCS.INPARAM.RoundOffFlag = sv_c
sv_r = urhk_B2k_RoundOff (")
if (sv_r == 1) then
# Error processing
endif
sv_q = BANCS.OUTPARAM.OutputAmount
# sv_q contains the rounded amount. This should be 10986700.0000
end-->
```


2.8 GET AMOUNT CALCULATED BASED ON SET-UP IN AMOUNT CODE TABLE

Given an input amount, currency code and Amount Table code, this function returns the resultant % amount.

SYNTAX

The Syntax for calling the function is

```
sv_r = urhk_B2k_GetASTMAmount ("")
```

FUNCTIONALITY

This function applies the same logic as Finacle in determining the commission % based on the amount slab setup.

INPUT

All the input variables should be populated in the INPARAM class of the BANCS repository. The variables are (all STRINGS):

InputAmount - The amount that determines the slab

CurrencyCode - The currency code of the amount

AmountTableCode - The code to be used to access the Amount Slab table

OUTPUT

Function returns 0 or 1. If 1, then there was failure in determining the %. If 0, all output variables will be populated in the OUTPARAM class of the BANCS repository.

The variables are(all STRINGS) :

OutputAmount - The % amount

EXAMPLE:

```
<--Start
...
sv_p = BANCS.INPUT.DDAmount
...

BANCS.INPARAM.InputAmount = sv_p
BANCS.INPARAM.CurrencyCode = "INR"
BANCS.INPARAM.AmountTableCode = "DDCOM"
sv_r = urhk_B2k_GetASTMAmount ("")
If (sv_r ==1)
    # Error processing
    ...
endif
sv_q = BANCS.OUTPARAM.OutputAmount
# sv_q contains the charge amount
...

End-->
```

2.9 GET LIST OF ACCOUNTS BASED ON INSTRUMENT NUMBER

Given an instrument, this user hook will give all the accounts for which the instrument number has been issued and are usable (i.e. status is not one of following (PASSED, DESTROYED, PAID CHEQUE RETURNED)).

This user hook can be used for postscript for Read Media Input Event in Inward Clearing Also.

SYNTAX

The Syntax for calling the function is

```
sv_r = urhk B2k_GetAccountsForInstrument("")
```

FUNCTIONALITY

Given a cheque number, this function lists all the accounts for which a cheque having that number has been issued.

INPUT

All the input variables should be populated in the INPARAM class of the BANCS repository. The variables are (all STRINGS):

InstrumentNumber	The numeric portion of the cheque number
InstrumentAlpha	The alpha portion of the cheque number (if passed as "" then for all instrument alpha's)
MinIssueDate	Match cheque books which have been issued beyond this date (format of date in DD-MM-YYYY HH24:MM:SS) (if passed as "" then for records are selected for all dates)
SchemeType	Match accounts which belong to this scheme type (if passed as "" then records for all schemes are considered)
SolId	Match accounts which belong to this Sol. (if passed as "" then records for all sols are selected)

OUTPUT

Always returns a 0.

All output variables will be populated in the OUTPARAM class of the BANCS repository. The variables are (all STRINGS) :

AcctIdList - concatenated string of 16 character account numbers

left justified, filled by trailing spaces

(e.g. "ACCOUNT1 ACCOUNT2 ") in case of two accounts

AcctIdCount - The number of accounts in the list

EXAMPLE:

```
# Get the list of Saving Bank accounts for all sol's to which 56005678 instrument has
been issued after 12th January and it is usable.

BANCS.INPARAM.Solid = ""
BANCS.INPARAM.SchemeType = "SBA"
BANCS.INPARAM.MinIssueDate = "12-01-1999 00:00:00"
BANCS.INPARAM.InstrumentAlpha = ""
BANCS.INPARAM.InstrumentNumber = "56005678"

sv_r = urhk_B2k_GetAccountsForInstrument(sv_s)
if (sv_r == 0) then
sv_l = CINT(BANCS.OUTPUTPARAM.AcctIdCount)
if (sv_l == 1) then
#single Account
sv_a = rtrim(BANCS.OUTPUTPARAM.AcctIdList)
print(sv_a)
else
# Multipl Accounts
sv_i = 0
while (sv_i < sv_l)
sv_d = sv_i * 8
sv_a = mid$(BANCS.OUTPUTPARAM.AcctIdList, sv_d, 8)
rtrim(sv_a)
print(sv_a)
sv_i = sv_i + 1
do
endif
endif
```

2.10 VALIDATE BANK CODE

This function returns SUCCESS (0) if the input bank is a valid, else returns FAILURE (1).

SYNTAX

The Syntax for calling the function is

```
sv_r = urhk_B2k_valBankCode (bankCode)
```

The variable can be a scratch pad variable (sv_b) or
a string("CAB").

FUNCTIONALITY

This user hook is provided to validate the given bank code.

INPUT

1. Bank Code

OUTPUT

Return value will be 0 if the bank code is valid, else it will be 1.

EXAMPLE:

```
<--start  
  
sv_a = Bank Code  
sv_r = urhk_B2k_valBankCode (sv_a)  
  
# if B2k_valBankCode function returns failure exit out of script.  
  
if( sv_r != 0 ) then  
    exitscript  
endif  
  
end-->
```

2.11 VALIDATE BANK AND BRANCH CODE

The user hook urhk_B2k_valBankBranchCode(variable) works as follows :

This function returns SUCCESS(0) if the input bank code, branch code combination is valid, else returns FAILURE(1). If the bank and branch code combination is valid, then some information regarding the branch is given as output.

SYNTAX:

The Syntax for calling the function is

```
sv_r = urhk_B2k_valBankBranchCode (variable)
```

The variable can be a scratch pad variable (sv_a) or a string("CAB|1098").

FUNCTIONALITY:

This user hook is provided to validate the given bank code, branch code combination. This also gives the details of the branch in OUTPARAM variables as given in the output section. Note that the output variables will be available only if the branch and bank code combination is valid and hence the script returns SUCCESS(0).

INPUT:

- Bank Code
- Branch Code

OUTPUT:

Return value will be 0 if the bank code, branch code combination is valid, else it will be 1. If the bank and branch code is valid the following details are available as the BANCS.OUTPARAM variables.

BANCS.OUTPUTPARAM.	Description
BranchName	Name of the branch
ShortName	Short name of the branch
Addr1	Address field 1
Addr2	Address field 2
CityCode	Branch city code
StateCode	Branch state code
CountryCode	Branch Country Code
TelexNumber	Branch Telex Number
PhoneNumber	Branch Phone Number
FaxNumber	Branch Fax Number
SwiftCode	Swift Code of the branch
SwiftMsgFlg	Whether swift allowed ('Y' or 'N')
BcbCode	BCB code of the branch
FbrFlg	Foreign branch flag ('Y' or 'N')

EXAMPLE:

```

<--start

sv_a=Bank Code
sv_b=Branch Code
sv_c=sv_a + "|" + sv_b
sv_r = urhk_B2k_valBankBranchCode (sv_c)

# If B2k_valBankBranchCode function returns failure exit out of script. Else use the
outparam variables for further processing.
if ( sv_r == 1 ) then
  exitscript
else
  if (BANCS.OUTPUTPARAM.SwiftMsgFlg == "Y") then
    .....
  endif
endif
endif

end-->

```

2.12 VALIDATE HO TRANSACTION CATEGORY CODE

This function returns SUCCESS(0) if the input Scheme code, Transaction Category Code combination is valid, else returns FAILURE(1).

SYNTAX

The Syntax for calling the function is

```
sv_r = urhk_B2k_valHoTranCode (variable)
```

The variable can be a scratch pad variable (sv_a) or

a string("ABSOT| 23").

FUNCTIONALITY

This user hook is provided to validate the given Scheme Code and Transaction Category code combination.

INPUT

1. Scheme Code
2. Transaction Category Code

OUTPUT

Return value will be 0 if the Scheme Code and Transaction Category Code combination is valid, else it will be 1.

EXAMPLE:

```
<--start

sv_a = Scheme Code
sv_b= Transaction Category Code
sv_c=sv_a + "|" + sv_b
sv_r = urhk_B2k_valHoTranCode (sv_c)

# if B2k_valHoTranCode function returns failure exit out of script.

if( sv_r == 1 ) then
    exitscript
endif

end-->
```

2.13 VALIDATE NUMERIC

This function returns SUCCESS(0) if the input string contains valid numeric digits, else returns FAILURE(1).

SYNTAX

The Syntax for calling the function is

```
sv_r = urhk_B2k_isStringNumeric (Amount String)
```

The variable can be a scratch pad variable (sv_b) or
a string("1234").

FUNCTIONALITY

This user hook is provided to validate the given string for numeric digits.

INPUT

1. Amount String

OUTPUT

Return value will be 0 if the input string contains valid digits, else it will be 1.

EXAMPLE:

```
<--start  
sv_r = urhk_B2k_isStringNumeric (Amount string)  
# if B2k_isStringNumeric function returns failure exit out of script.  
if( sv_r != 0 ) then  
    exitscript  
endif  
end-->
```

2.14 VALIDATE DATE

This user hook will validate if the string passed to it is a valid date. Only the date component is validated not the time component. However when passing the string either a string with only date or string with date and time may be passed. The user hook truncates the time component and validates the date.

SYNTAX:

```
sv_a = urhk_ B2k_valDate(Variable)
```

sv_a: scratch pad variable to store the return value

Variable: Scratch pad variable containing string to be validate.

FUNCTIONALITY:

This user hook is used to validate if a given string is a valid date. The first 10 characters of the string are taken and validated to check if it is a valid date. The time component is not validated.

INPUT:

String to be validated whether it is a valid date or not

OUTPUT

Return value will be success ('0') if string is a valid date else failure.

EXAMPLE:

```
sv_r=urhk_B2k_valDate(PASTDUE.PDC.dateofreckoning)  
if (sv_r != 0) then  
    sv_a="Invalid Date given for field Date Of Reckon "  
else  
    sv_a=""  
endif
```

2.15 VALIDATE REFERENCE CODE

This user hook will validate if the string passed to it is a valid reference code. The input to the user hook is reference code type and reference code. The user hook validates that the reference code is valid and of type reference code type as passed to it.

SYNTAX:

```
sv_a = urhk_ B2k_valRefCode(Variable)
```

sv_a: scratch pad variable to store the return value

Variable: Scratch pad variable containing string consisting of reference code type and reference code delimited by `|`.

FUNCTIONALITY:

This user hook will validate if the string passed to it is a valid reference code. The input to the user hook is reference code type and reference code. The user hook validates that the reference code is valid and of type reference code type as passed to it.

INPUT:

String consisting of reference code type and reference code to be validated delimited by `|`.

OUTPUT

Return value will be success ('0') if string is a valid reference code of the given reference code type else failure.

EXAMPLE:

```
sv_z="12|"+PASTDUE.PDL.authpermwaiver
sv_r=urhk_B2k_valRefCode(sv_z)
if (sv_r != 0) then
    sv_a="Auth. Perm. Waiver"
else
    sv_a=""
endif
```

2.16 GET 4 DIGIT YEAR

This function returns a four digit year , taking a two digit year as input , this function applies the **B2K_slidingWindow** on the two digit year and returns a four digit year.

SYNTAX

The Syntax for calling the function is


```
sv_r = urhk_B2k_getYear (2digitYear)
```

The variable can be a scratch pad variable (sv_b) or
a string("00").

FUNCTIONALITY

This user hook is provided to get a 4 digit year given a 2 digit year, This has been introduced to take care of external systems which still have 2 digit years, Finacle always expects a 4 digit year.

INPUT

1. 2 digit year string

OUTPUT

The output is available in the repository variable "year" , which can be accessed as
BANCS.OUTPARAM.year

EXAMPLE:

```
<--start
sv_r = urhk_B2k_getYear (2digitYear)
# if B2k_getYear function returns failure exit out of script.
if( sv_r != 0 ) then
    exitscript
endif
sv_y = BANCS.OUTPARAM.year
end-->
```

2.17 DATE ADD

This function adds(subtracts) the given number of days to the given year and returns
a date.

SYNTAX

The Syntax for calling the function is

```
sv_r = urhk_B2k_date_add (YearAndDays)
```

The variable can be a scratch pad variable (sv_b) or
a string("1999|235").

FUNCTIONALITY

This user hook is provided to get a date given a year and no; of days.

INPUT

1. 4 digit year string

2. number of days ("-\" indicates that it has to be subtracted)

OUTPUT

The output is available in the repository variable "dateadd" , which can be accessed as BANCS.OUTPARAM.dateadd

EXAMPLE:

```
<--start
sv_a = "1999|123"
sv_r = urhk_B2k_date_add (sv_a)

# if B2k_date_add function returns failure exit out of script.

if( sv_r != 0 ) then
    exitscript
endif
sv_y = BANCS.OUTPARAM.dateadd
end-->
```

2.18 SELECTION OF FIELDS FROM DATABASE

SYNTAX

```
sv_r = urhk_dbSelect(variable)
```

The variable can be a scratch pad variable (sv_a) or a string ("user_id | select user_id from upr where session_id = 'xxxx'").

Here note that everything before '|' (pipe symbol) is taken as the title value where as post pipe symbol is taken as query.

FUNCTIONALITY

This user hook allows script writer to select data from database tables. The user specifies the title of each selected field separated by comma before pipe symbol and specifies the complete query after that. It stores the output error code of the query in BANCS.OUTPARAM.DB_ERRCODE and message in BANCS.OUTPARAM.DB_ERRMSG.

INPUT

Comma separated Field titles followed by the Query, separated by pipe symbol. The field symbols should not have spaces.

OUTPUT

The output fields are stored in BANCS.OUTPARAM.<title field>. An error free condition is represented by 0 value of BANCS.OUTPARAM.DB_ERRCODE.

LIMITATION

- In case the query returns more than single row, it gives a FATAL error hence one need to be very meticulous during query writing.
- There is a size limitation of the query string size. In case where query size is very large make use of scratch pad variables.

EXAMPLE

```
sv_a = getenv("B2K_SESSION_ID")
sv_b = "user_id | select user_id from lgi where session_id = '" + sv_a + "'"
sv_r = usrhk_dbSelect(sv_b)
```

The above will fetch the value of **user_id** in **BANC.OUTPARAM.user_id** field.

2.19 USING A SQL STATEMENT IN A SCRIPT

SYNTAX

```
sv_r = urhk_dbSQL(variable)
```

The variable can be a scratch pad variable (sv_a) or a string ("update Upr Set User_logged_on_flg = 'N' where session_id = 'xxxx'").

FUNCTIONALITY

This user hook allows script writer to perform DML Statements on the database tables. Any updation in the database with this userhook will make an entry in SDA table.

INPUT

The SQL Statement

OUTPUT

The output fields are stored in BANC.OUTPARAM.<title field>. An error free condition is represented by 0 value of BANC.OUTPARAM.DB_ERRCODE.



This user hook must be used with utmost caution since none of the Finacle Application logic will apply here.

2.20 DISPLAY REPOSITORY FIELDS

SYNTAX

```
sv_r = urhk_SE_Dispositories(variable)
```

The variable can be a scratch pad variable (sv_a) or a string specifying repository variable.

FUNCTIONALITY

This user hook allows script writer to display repository values on the screen. Note that this can be used only if **BANCS.STDIN.TestFlg** variable or **B2K_SCRIPT_TEST_MODE** environment variable is set to **Yes**.

INPUT

The name of the repository.

OUTPUT

The output is shown on the screen via form.

EXAMPLE

```
sv_r = urhk_SE_Dispositories("BANCS")
```

2.21 ADVANCE DETAILS OF AN ACCOUNT IN BANCS REPOSITORY

This user hook will return advance details specified through ACM menu option for a given account number. This user hook can be called from any script event.

SYNTAX

```
sv_a = urhk_getAdvanceDetailsForAccount(Variable)
```

sv_a -> scratch pad variable to store the return value.

Variable -> This variable can be a scratch pad variable (sv_b) or a string ("SBGEN1").

FUNCTIONALITY

- Checks for validity of account number. Unverified and closed accounts will be considered as valid.

- If account number passed is found to be valid, the advance details of the are populated into repository fields.

INPUT

Valid Account number.

OUTPUT

If the account number specified is valid and advance details exist for account then return value will be '0'. Else return value will be '1'.

The following advance details will be populated into class OUTPARAM of BANCS repository. Individual repository field names are mentioned against the corresponding field name in Table.

Field Name in GAC (Advance) table	BANCS repository field name
sector_code	BANCS.OUTPARAM.SectCode
sub_sector_code	BANCS.OUTPARAM.SubSectCode
gaur_cover_code	BANCS.OUTPARAM.GuarCoverCode
nature_of_advn	BANCS.OUTPARAM.NatureOfAdvn
type_of_advn	BANCS.OUTPARAM.TypeOfAdvn
mode_of_advn	BANCS.OUTPARAM.ModeOfAdvn
purpose_of_advn	BANCS.OUTPARAM.PurposeOfAdvn
free_code_1	BANCS.OUTPARAM.FreeCode1
free_code_2	BANCS.OUTPARAM.FreeCode2
free_code_3	BANCS.OUTPARAM.FreeCode3
free_code_4	BANCS.OUTPARAM.FreeCode4
free_code_5	BANCS.OUTPARAM.FreeCode5
free_code_6	BANCS.OUTPARAM.FreeCode6
free_code_7	BANCS.OUTPARAM.FreeCode7
free_code_8	BANCS.OUTPARAM.FreeCode8
free_code_9	BANCS.OUTPARAM.FreeCode9
free_code_10	BANCS.OUTPARAM.FreeCode10
industry_type	BANCS.OUTPARAM.IndustryType
acct_occp_code	BANCS.OUTPARAM.AcctOccpCode
borrower_category_code	BANCS.OUTPARAM.BorwrCtgCode
pd_flg	BANCS.OUTPARAM.PastDueFlag
pd_xfer_date	BANCS.OUTPARAM.PastDueXferDate
pd_re_xfer_date	BANCS.OUTPARAM.PastDueReXferDate
prv_to_pd_gl_sub_head_code	BANCS.OUTPARAM.PrvToPastDueGLSubHeadCode
int_cr_acct_flg	BANCS.OUTPARAM.IntCrAcctFlag
int_dr_acct_flg	BANCS.OUTPARAM.IntDrAcctFlag

EXAMPLE

Advance details of an account may be required during generation of reports on Past Due accounts. Asset type, category and sector codes specified at account level may be obtained as below.

```
<--start

sv_a = BANCS.INPUT.AcctId

# sv_a (account id) is the input to user hook

sv_b = urhk_getAdvanceDetailsForAccount(sv_a)

# If urhk_getAdvanceDetailsForAccount function returns '1'
# if account number is invalid.
# If the return value is '1' then populate error message and
# exit from script.

if( sv_b == 1 ) then
    exitscript
endif

sv_i = BANCS.OUTPUTPARAM.IndustryType
sv_j = BANCS.OUTPUTPARAM.SectCode
sv_k = BANCS.OUTPUTPARAM.SubSectCode

end→
```

2.22 CUSTOMER DETAILS OF A CUSTOMER IN BANCS REPOSITORY :

This user hook will return Customer details specified through CUMM menu option for a given customer. This user hook can be called from any script event.

SYNTAX

sv_a = urhk_getCustomerDetails(Variable)

sv_a: scratch pad variable to store the return value.

Variable: Can be a scratch pad variable (sv_b) or a string ("CUST12").

FUNCTIONALITY:

- Checks for validity of Customer ID. Deleted and unverified status of customer ID will not be looked into.
- For a valid Customer ID, the customer details from CMG table are populated into repository fields.

INPUT

Valid Customer ID

OUTPUT

If the specified Customer is found to be valid return value will be '0'. Else return value will be '1'.

The following details will be populated into class OUTPARAM of BANCS repository. Individual repository field names are specified against the field name.

Field Name in CMG table	BANCS repository field name
cust_short_name	BANCS.OUTPARAM.CustShortName
cust_title_code	BANCS.OUTPARAM.CustTitleCode
cust_name	BANCS.OUTPARAM.CustName
cust_perm_addr1	BANCS.OUTPARAM.CustPermAddr1
cust_perm_addr2	BANCS.OUTPARAM.CustPermAddr2
cust_perm_city_code	BANCS.OUTPARAM.CustPermCityCode
cust_perm_state_code	BANCS.OUTPARAM.CustPermStateCode
cust_perm_pin_code	BANCS.OUTPARAM.CustPermPinCode
cust_perm_cntry_code	BANCS.OUTPARAM.CustPermCntryCode
cust_perm_phone_num	BANCS.OUTPARAM.CustPermPhoneNum
cust_perm_telex_num	BANCS.OUTPARAM.CustPermTelexNum
cust_sex	BANCS.OUTPARAM.CustSex
cust_grp	BANCS.OUTPARAM.CustGroup
cust_comu_addr1	BANCS.OUTPARAM.CustComuAddr1
cust_comu_addr2	BANCS.OUTPARAM.CustComuAddr2
cust_comu_city_code	BANCS.OUTPARAM.CustComuCityCode
cust_comu_state_code	BANCS.OUTPARAM.CustComuStateCode
cust_comu_pin_code	BANCS.OUTPARAM.CustComuPinCode
cust_comu_cntry_code	BANCS.OUTPARAM.CustComuCntryCode
cust_comu_phone_num_1	BANCS.OUTPARAM.CustComuPhoneNum1
cust_comu_phone_num_2	BANCS.OUTPARAM.CustComuPhoneNum2
cust_comu_telex_num	BANCS.OUTPARAM.CustComuTelexNum
cust_occp_code	BANCS.OUTPARAM.CustOccpCode
cust_commu_code	BANCS.OUTPARAM.CustComuCode
cust_sector_code	BANCS.OUTPARAM.CustSectCode
cust_sub_sector_code	BANCS.OUTPARAM.CustSubSectCode
cust_first_acct_date	BANCS.OUTPARAM.CustFirstAcctDate
cust_rating_code	BANCS.OUTPARAM.CustRatingCode
cust_rating_date	BANCS.OUTPARAM.CustRatingDate
cust_mgr_opin	BANCS.OUTPARAM.CustMgrOpin
cust_pw	BANCS.OUTPARAM.CustPassWord
tot_tod_alwd_times	BANCS.OUTPARAM.TotTodAlwdTimes
cust_introd_cust_id	BANCS.OUTPARAM.CustIntrodCustId
introd_title_code	BANCS.OUTPARAM.IntrodTitleCode
cust_introd_name	BANCS.OUTPARAM.CustIntrodName
cust_introd_stat_code	BANCS.OUTPARAM.CustIntrodStatCode
cust_type_code	BANCS.OUTPARAM.CustTypeCode
cust_emp_id	BANCS.OUTPARAM.CustEmpId
cust_stat_code	BANCS.OUTPARAM.CustStatCode
cust_othr_bank_code	BANCS.OUTPARAM.CustOthrBankCode
cust_stat_chg_date	BANCS.OUTPARAM.CustStatChgDate
cust_business_assets	BANCS.OUTPARAM.CustBusinessAssets
cust_prop_assets	BANCS.OUTPARAM.CustPropAssets

cust_investmnts	BANCS.OUTPARAM.CustInvestments
cust_net_worth	BANCS.OUTPARAM.CustNetWorth
cust_dep_in_othr_bank	BANCS.OUTPARAM.CustDepInOthrBank
cust_assets_as_on_date	BANCS.OUTPARAM.CustAssetsAsOnDate
cust_tot_fund_base	BANCS.OUTPARAM.CustTotFundBase
cust_tot_non_fund_base	BANCS.OUTPARAM.CustTotNonFundBase
cust_advn_as_on_date	BANCS.OUTPARAM.CustAdvnAsOnDate
cust_othr_lim	BANCS.OUTPARAM.CustOthrLim
cust_const	BANCS.OUTPARAM.CustConst
cust_fin_year_end_mnth	BANCS.OUTPARAM.CustFinYearEndMnth
cust_minor_flg	BANCS.OUTPARAM.CustMinorFlg
cust_nre_flg	BANCS.OUTPARAM.CustNreFlg
cust_hlth_code	BANCS.OUTPARAM.CustHlthCode
cust_card_hold_flg	BANCS.OUTPARAM.CustCardHoldFlg
cust_free_text	BANCS.OUTPARAM.CustFreeText
cust_chore_mbr_flg	BANCS.OUTPARAM.CustChoreMbrFlg
tot_mod_times	BANCS.OUTPARAM.TotModTimes
cust_asset_class	BANCS.OUTPARAM.CustAssetClass
cust_asset_class_date	BANCS.OUTPARAM.CustAssetClassDate
cust_caste_code	BANCS.OUTPARAM.CustCasteCode
tds_tbl_code	BANCS.OUTPARAM.TdsTblCode
tds_exemp_flg	BANCS.OUTPARAM.TdsExempFlg
reset_exemp_flg	BANCS.OUTPARAM.ResetExempFlg
tds_cust_id	BANCS.OUTPARAM.TdsCustId
num_of_accounts	BANCS.OUTPARAM.NumOfAccounts
share_holder_flg	BANCS.OUTPARAM.ShareHolderFlg
member_type	BANCS.OUTPARAM.MemberType
cust_membership_date	BANCS.OUTPARAM.CustMembershipDate
value_share_held	BANCS.OUTPARAM.ValueShareLead
total_shares_held	BANCS.OUTPARAM.TotalSharesHeld
nat_id_card_num	BANCS.OUTPARAM.NatIdCardNum
ps_freq_type	BANCS.OUTPARAM.PsFreqType
ps_freq_week_num	BANCS.OUTPARAM.PsFreqWeekNum
ps_freq_week_day	BANCS.OUTPARAM.PsFreqWeekDay
ps_freq_start_dd	BANCS.OUTPARAM.PsFreqStartDd
ps_freq_hldy_stat	BANCS.OUTPARAM.PsFreqHldyStat
date_of_birth	BANCS.OUTPARAM.DateOfBirth
pan_gir_num	BANCS.OUTPARAM.PanGirNum
psprt_num	BANCS.OUTPARAM.PsprtNum
psprt_issu_date	BANCS.OUTPARAM.PsprtIssuDate
psprt_det	BANCS.OUTPARAM.PsprtDet
psprt_exp_date	BANCS.OUTPARAM.PsprtExpDate
cust_marital_status	BANCS.OUTPARAM.CustMaritalStatus
cust_pager_no	BANCS.OUTPARAM.CustPagerNo
cust_fax_no	BANCS.OUTPARAM.CustFaxNo
free_text_1	BANCS.OUTPARAM.FreeText1
free_text_2	BANCS.OUTPARAM.FreeText2
free_text_3	BANCS.OUTPARAM.FreeText3
free_text_4	BANCS.OUTPARAM.FreeText4
free_text_5	BANCS.OUTPARAM.FreeText5
free_text_6	BANCS.OUTPARAM.FreeText6
free_text_7	BANCS.OUTPARAM.FreeText7

free_text_8	BANCS.OUTPUTPARAM.FreeText8
free_text_9	BANCS.OUTPUTPARAM.FreeText9
free_text_10	BANCS.OUTPUTPARAM.FreeText10
free_text_11	BANCS.OUTPUTPARAM.FreeText11
free_text_12	BANCS.OUTPUTPARAM.FreeText12
free_text_13	BANCS.OUTPUTPARAM.FreeText13
free_text_14	BANCS.OUTPUTPARAM.FreeText14
free_text_15	BANCS.OUTPUTPARAM.FreeText15
purge_allowed_flg	BANCS.OUTPUTPARAM.PurgeAllowedFlg
purge_text	BANCS.OUTPUTPARAM.PurgeText
cust_pref_till_date	BANCS.OUTPUTPARAM.CustPrefTillDate
acct_mgr_user_id	BANCS.OUTPUTPARAM.AcctMgrUserId
cust_swift_code	BANCS.OUTPUTPARAM.CustSwiftCode
is_swift_code_of_bank	BANCS.OUTPUTPARAM.IsSwiftCodeOfBank
chrg_level_code	BANCS.OUTPUTPARAM.ChrgLevelCode
chrg_dr_foracid	BANCS.OUTPUTPARAM.ChrgDrForacid
chrg_dr_sol_id	BANCS.OUTPUTPARAM.ChrgDrSolId
cust_chrg_history_flg	BANCS.OUTPUTPARAM.CustChrgHistoryFlg

EXAMPLE

Customer Net worth may be required during generation of past due reports. The same may be obtained as below.

```
<--start

sv_a = BANCS.INPUT.CustId

# sv_a (Customer Id) is the input to user hook

sv_b = urhk_getCustomerDetails(sv_a)

# urhk_getCustomerDetails will return failure if customer Id
# passed is invalid.
# If return value is '1' then populate error message and exit
# from script.

if( sv_b == 1 ) then
  exitscript
endif

sv_i = BANCS.OUTPUTPARAM.CustName
sv_j = cint(BANCS.OUTPUTPARAM.NumOfAccounts)
sv_k = cdouble(BANCS.OUTPUTPARAM.CustNetWorth)

end→
```

2.23 ASSET CLASSIFICATION DETAILS OF AN ACCOUNT IN BANCS REPOSITORY :

This user hook will return the latest Asset Classification details specified through menu options SASCL and MEAC menu options for a given account number. This user hook can be called from any script event.

SYNTAX

```
sv_a = urhk_getAssetClassfnDetailsForAccount(Variable)
```

sv_a: scratch pad variable to store the return value.

Variable: Can be a scratch pad variable (sv_b) or a string ("EMILN1").

FUNCTIONALITY

- Checks for validity of account number. Then checks for existence of account classification details.
- If input is found valid, populates system and user asset classification details and date of classification into repository.

INPUT

Valid Account Number

OUTPUT

If account number specified is valid and asset classification exists for the same, return value will be '0'. Else return value will be '1'.

The following details will be populated into class OUTPARAM of BANCS repository. Individual repository field names are specified against the table field names.

Field Name in ACD table	BANCS repository field name
main_classification_system	BANCS.OUTPARAM.MainClsfnSystem
sub_classification_system	BANCS.OUTPARAM.SubClsfnSystem
system_classification_date	BANCS.OUTPARAM.SystemClsfnDate
main_classification_user	BANCS.OUTPARAM.MainClsfnUser
sub_classification_user	BANCS.OUTPARAM.SubClsfnUser
user_classification_date	BANCS.OUTPARAM.UserClsfnDate

EXAMPLE

Main and sub user classification will be required for classifying accounts based on their asset classification during generation of reports on Past Due accounts.

```
<--start

sv_a = BANCS.INPUT.AcctId

# - sv_a (Account Number) is the input to user hook

sv_b = urhk_getAssetClassfnDetailsForAccount(sv_a)

# User Hook will return failure if Account number passed is
# invalid or asset classification details does not exist for
# account. If the return value is '1' then populate error
# message and exit from script.

if( sv_b == 1 ) then
  exitscript
endif

sv_t = BANCS.OUTPARAM.MainClsfnUser
sv_u = BANCS.OUTPARAM.SubClsfnUser
sv_k = BANCS.OUTPARAM. UserClsfnDate
end➔
```

2.24 INTEREST CALCULATION DETAILS FOR AN ACCOUNT IN BANCS REPOSITORY :

This user hook will return interest calculation details for a given account number from FINACLE. This user hook can be called from any script event.

SYNTAX

sv_a = urhk_getInterestDetailsForAccount (Variable)

sv_a: scratch pad variable to store the return value.

Variable: Can be a scratch pad variable (sv_b) or a string ("EMILN1").

FUNCTIONALITY

- Checks for validity of Account number specified. Checks for existence of account level interest entries. Note that BIA and FBA type of accounts will not have interest details at account level.
- If input is valid, populates interest details of account into FINACLE repositories.

INPUT

Valid Account Number

OUTPUT

If specified Account Number is valid and interest details exist for the same at account level then return value will be '0'. Else return value will be '1'.

The following details will be populated into class OUTPARAM of BANCS repository. Individual repository field names are specified against the table field name.

Field Name in EIT table	BANCS repository field name
accrued_upto_date_cr	BANCS.OUTPARAM.AccrdUptoDateCr
accrued_upto_date_dr	BANCS.OUTPARAM.AccrdUptoDateDr
last_accrual_run_date_cr	BANCS.OUTPARAM.LastAccrIRunDateCr
last_accrual_run_date_dr	BANCS.OUTPARAM.LastAccrIRunDateDr
booked_upto_date_cr	BANCS.OUTPARAM.BookedUptoDateCr
booked_upto_date_dr	BANCS.OUTPARAM.BookedUptoDateDr
last_book_run_date_cr	BANCS.OUTPARAM.LastBookRunDateCr
last_book_run_date_dr	BANCS.OUTPARAM.LastBookRunDateDr
book_for_rev_date_cr	BANCS.OUTPARAM.BookForRevDateCr
book_for_rev_date_dr	BANCS.OUTPARAM.BookForRevDateDr
last_book_cr_tran_id	BANCS.OUTPARAM.LastBookCrTranId
last_book_dr_tran_id	BANCS.OUTPARAM.LastBookDrTranId
last_book_cr_tan_date	BANCS.OUTPARAM.LastBookCrTranDate
last_book_dr_tan_date	BANCS.OUTPARAM.LastBookDrTranDate
interest_calc_upto_date_cr	BANCS.OUTPARAM.IntCalcUptoDateCr

interest_calc_upto_date_dr	BANCS.OUTPUTPARAM.IntCalcUptoDateDr
last_interest_run_date_cr	BANCS.OUTPUTPARAM.LastIntRunDateCr
last_interest_run_date_dr	BANCS.OUTPUTPARAM.LastIntRunDateDr
last_int_cr_tran_id	BANCS.OUTPUTPARAM.LastIntCrTranId
last_int_dr_tran_id	BANCS.OUTPUTPARAM.LastIntDrTranId
last_int_cr_tran_date	BANCS.OUTPUTPARAM.LastIntCrTranDate
last_int_dr_tran_date	BANCS.OUTPUTPARAM.LastIntDrTranDate
cumm_cr_int_adj_amt	BANCS.OUTPUTPARAM.CummCrIntAdjAmt
cumm_dr_int_adj_amt	BANCS.OUTPUTPARAM.CummDrIntAdjAmt
xfer_int_amt_cr	BANCS.OUTPUTPARAM.XferIntAmtCr
xfer_int_amt_dr	BANCS.OUTPUTPARAM.XferIntAmtDr
xfer_min_bal	BANCS.OUTPUTPARAM.XferMinBal
xfer_min_bal_date	BANCS.OUTPUTPARAM.XferMinBalDate
int_freq_type_cr	BANCS.OUTPUTPARAM.IntFreqTypeCr
int_freq_week_num_cr	BANCS.OUTPUTPARAM.IntFreqWeekNumCr
int_freq_week_day_cr	BANCS.OUTPUTPARAM.IntFreqWeekDayCr
int_freq_start_dd_cr	BANCS.OUTPUTPARAM.IntFreqStartDdCr
int_freq_hldy_stat_cr	BANCS.OUTPUTPARAM.IntFreqHldyStatCr
next_int_run_date_cr	BANCS.OUTPUTPARAM.NextIntRunDateCr
int_freq_type_dr	BANCS.OUTPUTPARAM.IntFreqTypeDr
int_freq_week_num_dr	BANCS.OUTPUTPARAM.IntFreqWeekNumDr
int_freq_week_day_dr	BANCS.OUTPUTPARAM.IntFreqWeekDayDr
int_freq_start_dd_dr	BANCS.OUTPUTPARAM.IntFreqStartDdDr
int_freq_hldy_stat_dr	BANCS.OUTPUTPARAM.IntFreqHldyStatDr
next_int_run_date_dr	BANCS.OUTPUTPARAM.NextIntRunDateDr

EXAMPLE

Last debit interest calculation date, last debit interest accrued up to date of a past due account may be used for reporting purposes. These may be obtained as below.

```

<--start

sv_a = BANCS.INPUT.AcctId

# -----
# - sv_a (Account Number) is the input to user hook
#   getInterestDetailsForAccount
# -----

sv_b = urhk_getInterestDetailsForAccount (sv_a)

# User Hook will return failure if Account number passed is
# invalid or interest details are not maintained at account
# level for the account (BIA and FBA type).
# If the return value is '1' then populate error message and
# exit from script.

if( sv_b == 1 ) then
  exitscript
endif

sv_i = BANCS.OUTPUTPARAM.AccrdUptoDateDr
sv_j = BANCS.OUTPUTPARAM.IntCalcUptoDateDr
sv_k = BANCS.OUTPUTPARAM.BookedUptoDateDr
sv_l = BANCS.OUTPUTPARAM.LastIntDrTranId

end→

```

2.25 ACCOUNT BALANCE AS ON A GIVEN DATE:

This user hook will return end of day account balance as on a given date from FINACLE. This user hook can be called from any script event.

SYNTAX:

```
sv_a = urhk_getAcctBalanceAsOnDate("")
```

sv_a: scratch pad variable to store the return value.

FUNCTIONALITY

- Validates account number and as on date. Will return failure if account number is invalid or if date format is not correct.
- If inputs are valid, populates end of day account balance into FINACLE repository.

INPUT

- Valid account number
- As on Date

The above inputs are to be populated into the following repositories.

Valid Account Number	BANCS.INPARAM.acctId
As on Date	BANCS.INPARAM.asOnDate (in DD-MM-YYYY format)

OUTPUT

If the input values are invalid return value will be '1'.

End of day account balance based on transaction date will be populated into the repository specified below.

Account balance	BANCS.OUTPARAM.acctBal
-----------------	------------------------

EXAMPLE

To obtain account balance as on BOD date for an account, the following may be implemented.

```
<--start
# Populate BOD date and into as on date and account number

BANCS.INPARAM.acctId = BANCS.INPUT.acctId
BANCS.INPARAM.asOnDate = BANCS.STDIN.BODDate

sv_a = urhk_getAcctBalanceAsOnDate("")

if ( sv_a == 1 ) then
    exitscript
endif

sv_i = cdouble(BANCS.INPARAM.balance)
end-->
```

2.26 INTEREST RATE FOR AN ACCOUNT ON A GIVEN DATE IN BANCS REPOSITORY

This user hook will return absolute value of interest rate for a given account number and a given date and specified amount. This user hook can be called from any script event.

SYNTAX:

```
sv_a = urhk_getInterestRateForAccount("")
```

sv_a: scratch pad variable to store the return value.

FUNCTIONALITY:

- Validates account number, as on date and amount.
- If inputs are valid, populates interest rate into FINACLE repositories.

INPUT:

- Valid account number
- As on Date
- Balance as on date

The above inputs are to be populated into the following repositories.

Valid Account Number	BANCS.INPARAM.acctNum
As on Date	BANCS.INPARAM.asOnDate (in DD-MM-YYYY format)
Balance	BANCS.INPARAM.balance

OUTPUT

If the input values are invalid return value will be '1'. Also if the interest rate could not be arrived at due to improper interest version set up or non-maintenance of interest version as on the specified date, return value will be 1.

This user hook will obtain the normal absolute value of interest rate as of BOD date. If the balance amount specified is positive, then the interest rates mentioned against credit balances in IVSM/TVSM will be looked into. Similarly the debit balance interest rate entries will be looked into for negative balances.

Interest rate will be populated into the repository specified below.

Interest Rate	BANCS.OUTPARAM.interestRate
---------------	-----------------------------

EXAMPLE

To obtain interest rate as on BOD date for an account, the following may be implemented.

```

<--start

# Get account balance as on BOD date.

    BANCS.INPARAM.acctId = BANCS.INPUT.acctId
    BANCS.INPARAM.asOnDate = BANCS.STDIN.BODDate

    sv_a = urhk_getAcctBalanceAsOnDate("")

# Populate the input repositories for getting interest rate
    BANCS.INPARAM.balance = BANCS.OUTPUTPARAM.acctBal
    BANCS.INPARAM.acctNum = BANCS.INPUT.acctId
    BANCS.INPARAM.asOnDate = BANCS.STDIN.BODDate

    sv_a = urhk_getInterestRateForAccount("")
    if ( sv_a == 1 ) then
        exitscript
    endif

    sv_i = cdouble(BANCS.OUTPUTPARAM.interestRate)
end-->

```

2.27 SUM CREDIT AND DEBIT TRANSACTION AMOUNT OVER A PERIOD FOR AN ACCOUNT

This user hook will return sum of credit and debit transaction amount over a period for a given account. This user hook can be called from any script event.

SYNTAX

```
sv_a = urhk_getSumCrDrTranAmtsForPeriod("")
```

sv_a: scratch pad variable to store the return value.

FUNCTIONALITY

- Validates account number, from date and to date specified as inputs. Also checks for data range (i.e., begin date must be lesser than or equal to end date). Will return failure if any of the inputs is found invalid.
- If inputs are valid, populates sum credit and debit balances into FINACLE repositories. Only posted transactions will be considered. Transaction amounts will be based on transaction date (and not value date of transaction).

INPUT

- Valid account number
- From Date
- To Date

The above inputs are to be populated into the following repositories.

Valid Account Number	BANCS.INPARAM.acctId
From Date	BANCS.INPARAM.fromDate (in DD-MM-YYYY format)
To Date	BANCS.INPARAM.toDate (in DD-MM-YYYY format)

OUTPUT

If the input values are invalid return value will be '1'.

Sum credit and debit transaction amount for period will be populated into the following repositories.

Sum Credit transaction amount	BANCS.OUTPUTPARAM.sumCreditAmt
Sum Debit transaction amount	BANCS.OUTPUTPARAM.sumDebitAmt

EXAMPLE

To obtain sum credits made towards a past due account from the date of transfer to past due status, the following code may be incorporated.

```
<--start
# Get past due transfer date
sv_b = BANCS.INPUT.acctId

sv_a = urhk_getAdvanceDetailsForAccount(sv_b)

#Populate from date as past due transfer date. To date as BOD
#date and call user hook.
BANCS.INPUTPARAM.acctId = sv_b
BANCS.INPUTPARAM.fromDate = BANCS.OUTPUTPARAM.PastDueXferDate
BANCS.INPUTPARAM.toDate = BANCS.STDIN.BODDate

sv_a = getSumCrDrTranAmtsForPeriod (")
if ( sv_a == 1 ) then
    exitscript
endif

sv_i = cdouble(BANCS.OUTPUTPARAM.sumCreditAmt)

end-->
```

2.28 SUM TRANSACTION AMOUNT FOR A SPECIFIED ACCOUNT ON A ACCOUNT LABEL

This user hook will return sum transaction amount on a account label for a given account. This user hook can be called from any script event.

SYNTAX

```
sv_a = urhk_getSumTranAmtForAcctLabel (")
```

sv_a: scratch pad variable to store the return value.

FUNCTIONALITY

- Validates account number and account label. Will return failure ('1') if the inputs are invalid.

- If inputs are valid, populates sum transaction amount on account label for account into FINACLE repository. Only posted transactions will be considered.

INPUT

- Valid account number
- Valid account label

The above inputs are to be populated into the following repositories.

Valid Account Number	BANCS.INPARAM.acctId
Account label	BANCS.INPARAM.acctLabel

OUTPUT

If the input values are invalid return value will be '1'.

Sum transaction amount be populated into the following repository.

Sum transaction amount	BANCS.OUTPARAM.sumTranAmt
------------------------	---------------------------

EXAMPLE

Provision amount made for a customer account can be obtained as below.

```
<--start
BANCS.INPARAM.acctId = BANCS.INPUT.acctId
BANCS.INPARAM.acctLabel = "PROVISION"

sv_a = getSumCrDrTranAmtsForPeriod("")
if ( sv_a == 1 ) then
    exitscript
endif

sv_i = cdouble(BANCS.OUTPARAM. sumTranAmt)

end-->
```

2.29 SECURITY DETAILS FOR A GIVEN ACCOUNT NUMBER

This user hook will return Security details maintained through SRM menu option for an account. This user hook can be called from any script event.

SYNTAX

```
sv_a = urhk_getSecurityDetailsForAccount(Variable)
```

sv_a: scratch pad variable to store the return value.

FUNCTIONALITY:

- Validates account number. Will return failure ('1') if the input is invalid.
- If valid account is passed, on first call, returns the first security record detail. On successive calls for the same account the next record will be fetched. On

fetching all available records, when no more records exist for account, will return failure.

INPUT:

- Valid account number

OUTPUT

If the input value is invalid or all security records for account have been passed back and exhausted return value will be '1'.

The various field values are populated into the following repositories. These field values will be overwritten with the new values on sub subsequent calls.

Field Name in SDR table	BANCS repository field name
secu_srl_num	BANCS.OUTPUTPARAM.secuSrlNum
secu_code	BANCS.OUTPUTPARAM.secuCode
secu_type_ind	BANCS.OUTPUTPARAM.secuTypeInd
secu_group_code	BANCS.OUTPUTPARAM.secuGroupCode
secu_class_code	BANCS.OUTPUTPARAM.secuClassCode
use_for_int_flg	BANCS.OUTPUTPARAM.useForIntFlg
use_for_dp_ind	BANCS.OUTPUTPARAM.useForDpInd
lien_acid	BANCS.OUTPUTPARAM.lienAcct
unit_value	BANCS.OUTPUTPARAM.unitValue
num_of_units	BANCS.OUTPUTPARAM.noOfUnits
secu_value	BANCS.OUTPUTPARAM.secuValue
retain_margin_pcmt	BANCS.OUTPUTPARAM.retainMarginPcmt
max_ceiling_lim	BANCS.OUTPUTPARAM.maxCeilingLim
item_freq_type	BANCS.OUTPUTPARAM.itemFreqType
item_freq_week_num	BANCS.OUTPUTPARAM.itemFreqWeekNum
item_freq_week_day	BANCS.OUTPUTPARAM.itemFreqWeekDay
item_freq_start_dd	BANCS.OUTPUTPARAM.itemFreqStartDd
item_freq_hldy_stat	BANCS.OUTPUTPARAM.itemFreqHldyStat
item_due_date	BANCS.OUTPUTPARAM.itemDueDate
item_renew_date	BANCS.OUTPUTPARAM.itemRenewDate
secu_from_srl_num	BANCS.OUTPUTPARAM.secuFromSrlNum
secu_to_srl_num	BANCS.OUTPUTPARAM.secuToSrlNum
bank_code	BANCS.OUTPUTPARAM.bankCode
branch_code	BANCS.OUTPUTPARAM.branchCode
chrg_code	BANCS.OUTPUTPARAM.chrgCode
secu_id	BANCS.OUTPUTPARAM.secuId
secu_desc	BANCS.OUTPUTPARAM.secuDesc
def_value_override_flg	BANCS.OUTPUTPARAM.defValueOverrideFlg
free_text	BANCS.OUTPUTPARAM.freeText

EXAMPLE

Sum security value for an account may be arrived at as below.

```
<--start
sv_b = BANCS.INPUT.acctId
sv_j = 0
```

```
# call user hook to get first SRM entry
sv_a = urhk_getSecurityDetailsForAccount(sv_b)

# Until all records from SRM have been retrieved process in
# while loop.
while( sv_a < 1)
# Sum up security values of each record
sv_j = cdouble(BANCS.OUTPARAM.secuValue) + sv_j
# Fetch next record
sv_a = urhk_getSecurityDetailsForAccount(sv_b)
do

end-->
```

2.30 CHECK FOR PIPE CHAR IN THE INPUT STRING.

SYNTAX

```
sv_a = urhk_B2k_checkForPipeChar ("inputstring")
```

FUNCTIONALITY:

This user hook will search for pipe char ("|") in the inputstring. If the inputstring contains the pipe character the user hook will return failure.

INPUT:

User hook will accept "inputstring" as the input. The maximum size of the string must be 1024 characters only.

OUTPUT:

This function returns 0 if there are no pipe characters in the input string, otherwise it will return non zero value.

2.31 CHECK WHETHER THE ACCOUNT IS A CASH ACCOUNT.

SYNTAX:

```
sv_a=urhk_B2k_valCashAccount("Account")
```

FUNCTIONALITY:

This user hook will take account number as the input , and return failure if the input account is cash account.

INPUT:

User hook will accept "Account" as the input. As shown in the syntax.

OUTPUT:

This function return 0 if "Account" is not cash account. Otherwise it will return non zero value.

2.32 CHECK WHETHER THE ACCOUNT IS A SYSTEM ONLY ACCOUNT.

SYNTAX:

```
sv_a=urhk_B2k_valSystemAccount("Account")
```

FUNCTIONALITY:

This user hook will take account number as the input, and return failure if the input account is system only account.

INPUT:

User hook will accept "Account" as the input. As shown in the syntax.

OUTPUT:

This function returns 0 if Account is not system only account . Otherwise it will return non zero value.

2.33 CHECK WHETHER THE ACCOUNT IS FCNR ACCOUNT.

SYNTAX:

```
sv_a=urhk_B2k_valFCNRAccount("Account")
```

FUNCTIONALITY:

This user hook will take account number as the input and return failure if the input account is FCNR account.

INPUT:

User hook will accept "Account" as the input. As shown in the syntax.

OUTPUT:

This function returns 0 if Account is not FCNR account . Otherwise it will return non zero value.

2.34 CHECK WHETHER THE ACCOUNT IS HO ACCOUNT.**SYNTAX:**

```
sv_a=urhk_B2k_valHOAccount("Account")
```

FUNCTIONALITY:

This user hook will take account number as the input and return failure if the input account is not HO account.

INPUT:

User hook will accept "Account" as the input. As shown in the syntax.

OUTPUT:

This function returns 0 if Account is HO account . Otherwise it will return non zero value.

2.35 VALIDATE THE AMOUNT WITH CURRENCY .**SYNTAX:**

```
sv_a=urhk_B2k_validateAmtWithCrncy(amount| crncy)
```

FUNCTIONALITY:

This user hook will take amount and currency as the input . It will validate the amount with currency. And return back the amount in the precision of the currency.

INPUT :

Amount and crncy as shown in the syntax. The amount and currency could be a variable or a value itself. If it is a value it must be enclosed within quotes.

OUTPUT :

BANCS.OUTPARAM. OutputAmount

This will contain the amount in the precision of the currency.

2.36 GIVEN A DATE GET THE DATE DIFF FROM BOD.

SYNTAX:

```
sv_a=urhk_B2k_getDateDiffFromBODDate(date) [Date in format:dd-mm-yyyy]
```

FUNCTIONALITY:

This user hook will take the date in dd-mm-yyyy format and return back the date difference from BOD.

INPUT :

Date for which the date diff from BOD is needed. If the date is not in proper format, the user hook will return a failure.

OUTPUT:

BANCS.OUTPARAM.DateDiff

This will contain the date difference from BOD for a given date.

2.37 VALIDATE CRNCY CODE.

SYNTAX:

```
sv_a=urhk_B2k_valCrncy(crncyCode)
```

FUNCTIONALITY:

This user hook will take the crncyCode as the input and validates it.

INPUT :

crncy code as shown in the syntax.

OUTPUT:

This user hook return 0 if the input is valid crncy code , otherwise it returns a non zero value.

2.38 VALIDATE TRAN REPORT CODE FOR THE GIVEN ACCOUNT.

SYNTAX:

```
sv_a=urhk_B2k_valTranRptCode(Account|TranReportCode)
```

FUNCTIONALITY:

This user hook will take the account and tran report code. Check whether the given tran report code is valid for the given account.

INPUT :

Account and Tran report Code as shown in the syntax.

OUTPUT:

This user hook will return 0 if the given tran report code is valid for the account. Otherwise it returns a non zero value.

2.39 GET THE SCHM DETAILS FOR A GIVEN DD SCHM CODE AND CURRENCY.

SYNTAX:

```
sv_a=urhk_getDDSchemeCrncyDetails(schm_code|currency)
```

FUNCTIONALITY:

This user hook will take scheme code of DDA scheme type and currency. And give the details of that scheme. The details returned by user hook are as per the output.

INPUT :

Scheme code belonging to DDA scheme type and currency for which the details are needed. The input must be a valid DDA Scheme type only else the output will be a fatal error.

OUTPUT:

Following values related to the scheme code , currency combination will be available as output.

BANCS.OUTPARAM.schmCode

BANCS.OUTPARAM.crncyCode

BANCS.OUTPARAM.ddnUsedFlg
BANCS.OUTPARAM.ddaUsedFlg
BANCS.OUTPARAM.invtSerUsedFlg
BANCS.OUTPARAM.invtMicrUsedFlg
BANCS.OUTPARAM.rptLostUsedFlg
BANCS.OUTPARAM.ddXferBacid
BANCS.OUTPARAM.dfltInvtLocnClass
BANCS.OUTPARAM.dfltInvtLocnCode
BANCS.OUTPARAM.invtClass
BANCS.OUTPARAM.dfltInvtType
BANCS.OUTPARAM.dfltMicrInvtType
BANCS.OUTPARAM.otInvtType
BANCS.OUTPARAM.ttInvtType
BANCS.OUTPARAM.olInvtType
BANCS.OUTPARAM.tlInvtType
BANCS.OUTPARAM.otMicrInvtType
BANCS.OUTPARAM.ttMicrInvtType
BANCS.OUTPARAM.olMicrInvtType
BANCS.OUTPARAM.tlMicrInvtType
BANCS.OUTPARAM.prntSrlExistFlg
BANCS.OUTPARAM.ddnWarnLevelExcpCode
BANCS.OUTPARAM.ddCommChangedExcpCode
BANCS.OUTPARAM.ddPaidExcpCode
BANCS.OUTPARAM.ddRptLostExcpCode
BANCS.OUTPARAM.ddNonPymtAdvExcpCode
BANCS.OUTPARAM.ddDupIssuedExcpCode
BANCS.OUTPARAM.ddCancScChngdExcpCode
BANCS.OUTPARAM.ddOrgTranNaExcpCode
BANCS.OUTPARAM.ddAcctMismatchExcpCode
BANCS.OUTPARAM.ddModAfterEodExcpCode
BANCS.OUTPARAM.ddPrntAfterEodExcpCode
BANCS.OUTPARAM.ddReprntAftEodExcpCode
BANCS.OUTPARAM.improperAdvExcpCode
BANCS.OUTPARAM.improperStatusExcpCode

BANCS.OUTPUTPARAM.ddTemplateName
BANCS.OUTPUTPARAM.defaultBankCode
BANCS.OUTPUTPARAM.defaultBrCode
BANCS.OUTPUTPARAM.bankersChequeFlg
BANCS.OUTPUTPARAM.sendIbrFlg
BANCS.OUTPUTPARAM.commCrncyHcFc
BANCS.OUTPUTPARAM.ddSellRateCode
BANCS.OUTPUTPARAM.ddBuyRateCode
BANCS.OUTPUTPARAM.consolidatePartTranFlg
BANCS.OUTPUTPARAM.consolidatePartTranBacid
BANCS.OUTPUTPARAM.hotItemExcpCode
BANCS.OUTPUTPARAM.bankersChequeExcpCode
BANCS.OUTPUTPARAM.ddNotThroIbrExcpCode
BANCS.OUTPUTPARAM.pymtOfDupDdExcpCode
BANCS.OUTPUTPARAM.pymtOfLostDdExcpCode
BANCS.OUTPUTPARAM.pymtOfStopDdExcpCode
BANCS.OUTPUTPARAM.localEventId
BANCS.OUTPUTPARAM.otherEventId

2.40 VALIDATE INSTRUMENT DATE.

SYNTAX

```
sv_a = urhk_B2k_validateInstrmntDate(instrmnt_type|instrmnt_date)
```

FUNCTIONALITY :

This user hook will do basic date related validations for a given instrument. The validations done are whether the instrument is post dated or instrument is stale.

INPUT:

Instrument Type and Instrument Date. The date must be in dd-mm-yyyy format only.

OUTPUT:

The user hook will return 0 if the validation goes through , Otherwise it will return non zero value.

In case of user hook returning non zero value it will also return the status related to date in repository variable

BANCS.OUTPARAM.ReturnCode

Different status depending on the BANCS.OUTPARAM.ReturnCode value are as follows.

➤ Instrument is post dated, the return code is 2

➤ Instrument is stale then the return value is 3.

Neither of above value - The instrument type passed is wrong.

2.41 ADD A USER DEFINED MESSAGE INTO IBR FOR RECONCILIATION

SYNTAX

```
sv_a = urhk_PutUDTXN_into_IBR ("")
```

FUNCTIONALITY

This function will add a record into Finacle database which will be picked up by the IBR program for transmission to the IBR center. A corresponding 'originating entry' will have to be sent by a remote application to the IBR center so that the transactions get 'matched'. Typically, this function will be used in the process of responding to an online request by a remote application.

☞ *that this function does not 'commit' to the database. 'Commit' needs to be done by the Script using this function by calling the DBSql user hook.*

INPUT

The following input should be populated in the INPARAM class of the BANCS repository.

BANCS.INPARAM.Solid – The Solid in which the transaction takes place (max 6 chars)

BANCS.INPARAM.Tran_date – The system date of the remote application which originates the transaction request in dd-mm-yyyy hh:mi:ss format

BANCS.INPARAM.SerialNum – The unique number assigned to this transaction by the remote application (Max 12 chars)

BANCS.INPARAM.Application_id – The unique id assigned to the remote application (Max 3 chars)

BANCS.INPARAM.Tran_amt – The transaction amount associated with the transaction in decimal format (1234.45 for eg.)(Max 16 chars)

BANCS.INPARAM.Tran_Crncy – The currency of the transaction amount associated with the transaction (Max 3 chars)

BANCS.INPARAM.CustId - The Customer Id associated with the transaction.(Max 28 chars)

BANCS.INPARAM.IBRCenter – The IBR center at which reconciliation has to take place. (Max 2 chars)

BANCS.INPARAM.Particulars_1 - Anything put here will be available at IBR center reports to take corrective action in case of a mismatch. (Max 40 chars).

BANCS.INPARAM.Particulars_2 - Anything put here will be available at IBR center reports to take corrective action in case of a mismatch. (Max 40 chars).

OUTPUT

This function returns 0 if the creation is successful else returns a non-zero value.

2.42 GETDIFFBETWEENTWOFREQ

This user hook gets the difference between the given two frequencies and populates the ratio in terms of days in BANCS.OUTPARAM.FREQUENCY_RATIO.

Input Parameters : Two frequencies. (e.g) W| | |N|D| | |N|

Output Parameters : Ratio in terms of days between the two frequencies

2.43 B2K_ISCHECKSUMRIGHT

This user hook is used to calculate the checksum of the input string passed and compare it with the given checksum

Input Parameter : Input String

Out put : Success or Failure

2.44 urhk_getAcctTurnoverDetails

This user hook gives the account turnover details for the given foracid and given as on date.

The input to this user hook is the foracid and as on date separated by a delimiter '|'.

2.45 GET THE PID OF THE PROCESS

SYNTAX

```
sv_a = urhk_B2k_PID("")
```

FUNCTIONALITY

This user hook will return the pid of the process.

INPUT

none

OUTPUT

The user hook will return 0 on the success. Output is stored in BANCS.OUTPUT.PID

EXAMPLE

```
<--start
#trace on
sv_e = urhk_B2k_PID("")
print(BANCS.OUTPUT.PID)
#trace off
end-->
```

2.46 SLEEP

SYNTAX

```
sv_a = urhk_B2k_Sleep("No_of_seconds")
```

FUNCTIONALITY

This user hook will make the sleeps for specified no of seconds.

INPUT

No of seconds to sleep.

OUTPUT

None

EXAMPLE

```
<--start
#trace on
sv_e = urhk_B2k_Sleep("5")
#trace off
end-->
```

This will make the process sleep for 5 seconds.

2.47 VALIDATE INSTRUMENT FOR AN ACCOUNT

SYNTAX

```
sv_a = urhk_B2k_valInstrumentNumForAcct("")
```

FUNCTIONALITY

This USER HOOK Checks whether a given instrument is issued to passed account, if it is a passed account then returns back the status.

INPUT

acid,instrument number, instrument alpha

OUTPUT

BANCS.OUTPUTPARAM. ReturnCode
BANCS.OUTPUTPARAM. ChqStatus

EXAMPLE

```
<--start  
trace on  
BANCS.INPARAM.AccountId = "SB4"  
BANCS.INPARAM.InstrumentNumber = "101"  
BANCS.INPARAM.InstrumentAlpha = "SB/"  
sv_e = urhk_B2k_valInstrumentNumForAcct("")  
print (BANCS.OUTPUTPARAM.ReturnCode)  
print (BANCS.OUTPUTPARAM.ChqStatus)  
trace off  
end-->
```

2.48 VALIDATION OF SOLID

SYNTAX

```
sv_a = urhk_B2k_valSolId("Sol_id")
```

FUNCTIONALITY

This user hook will validate the given sol_id.

INPUT

Input will be a string that is the sol_id to be validated.

OUTPUT

Output value of the userhook will 0 if the given sol_id is a valid sol_id else it will return the value as 1 (not a valid sol_id).

EXAMPLE

```
<--start
#trace on
sv_e = urhk_B2k_valSolId("SCGL")
print(sv_e)
#trace off
end-->
```

2.49 DBCursorOpen

SYNTAX

```
Sv_m = "acid,foracid | select acid,foracid from gam where cust_id = 'XYZ'"
sv_a = urhk_dbCursorOpen(sv_m)
```

FUNCTIONALITY

This user hook will open the cursor for the given query and put the db_cursor_number in the repository.

INPUT

Input will be a string that is the query corresponding which cursor is opened.

OUTPUT

The userhook will return 0 on successful open.

And opening cursor number will be stored in BANCS.OUTPUTPARAM.DB_CURSOR_NUMBER

EXAMPLE

```
<--start
trace on
sv_p = ""
sv_q = ""
sv_b = "acid,foracid | select acid,foracid from gam where cust_id = '      ACME' "
sv_e = urhk_dbCursorOpen(sv_b)
if ( sv_e == 0 ) then
    sv_m = BANCS.OUTPUTPARAM.DB_CURSOR_NUMBER
    sv_r = urhk_dbCursorFetch(sv_m)
    while(sv_r == 0)
        sv_p = sv_p + BANCS.OUTPUTPARAM.acid
        sv_q = sv_q + BANCS.OUTPUTPARAM.foracid
        sv_r = urhk_dbCursorFetch(sv_m)
    do
        sv_x = urhk_dbCursorClose(sv_m)
    endif
    print(sv_p)
    print(sv_q)
    trace off
end-->
```

2.50 DBCursorFetch

SYNTAX

```
Sv_m = BANCS.OUTPUTPARAM.DB_CURSOR_NUMBER
```

```
sv_a = urhk_dbCursorFetch(sv_m)
```

FUNCTIONALITY

This user hook will fetch the records from open cursor for the given cursor_number. For fetching all the records from the cursor this userhook shall be used in the loop .

INPUT

Input will be a DB_CURSOR_NUMBER returned by dbCursorOpen .

OUTPUT

The userhook will return 0 on successful fetching the record

And the fetched record will be stored in the OUTPARAM repository as in the given example: BANCS.OUTPARAM.acid

BANCS.OUTPARAM.foracid

EXAMPLE

Refer above.

2.51 DBCURSORCLOSE

SYNTAX

```
Sv_m = BANCS.OUTPARAM.DB_CURSOR_NUMBER
```

```
sv_a = urhk_dbCursorClose(sv_m)
```

FUNCTIONALITY

This user hook will close the cursor for the given cursor_number. This userhook is called after the records have been fetched from the cursor through dbCursorFetch userhook.

INPUT

Input will be a DB_CURSOR_NUMBER returned by dbCursorOpen .

OUTPUT

This userhook will return 0 on successful closing the cursor

EXAMPLE

```

<--start
trace on
sv_p = ""
sv_q = ""
sv_b = "acid,foracid | select acid,foracid from gam where cust_id = '      ACME' "
sv_e = urhk_dbCursorOpen(sv_b)
if ( sv_e == 0 ) then
    sv_m = BANCS.OUTPUTPARAM.DB_CURSOR_NUMBER
    sv_r = urhk_dbCursorFetch(sv_m)
    while(sv_r == 0)
        sv_p = sv_p + BANCS.OUTPUTPARAM.acid
        sv_q = sv_q + BANCS.OUTPUTPARAM.foracid
        sv_r = urhk_dbCursorFetch(sv_m)
    do
        sv_x = urhk_dbCursorClose(sv_m)
    endif
    print(sv_p)
    print(sv_q)
    trace off
end-->

```

2.52 GET DSA TURNOVER DETAILS FOR A SCHEME TYPE**SYNTAX**

```
sv_a = urhk_ getDSATODetailsForSchemeType ()
```

FUNCTIONALITY

This user hook gives the DSA turnover details for the given dsa id, scheme type, crncy code and as on date. The inputs are to be separated by a delimiter `|`.

INPUT

DSA Id, Scheme type, Currency code and As on Date

OUTPUT

DSA Turnover details: Self and Others.

The Self-Turnover details will be available only if "DSATOInd_S" is set to YES. Otherwise other fields should not be accessed.

BANCS.OUTPUTPARAM. DSATOInd_S

BANCS.OUTPUTPARAM. dsaId_S

BANCS.OUTPUTPARAM. schemeType_S

BANCS.OUTPUTPARAM. crncyCode_S

BANCS.OUTPUTPARAM. turnoverInd_S

BANCS.OUTPUTPARAM. endDate_S

BANCS.OUTPUTPARAM. numOfDrTrans_S

BANCS.OUTPUTPARAM. numOfCrTrans_S

BANCS.OUTPUTPARAM. totDrBalAmt_S

BANCS.OUTPARAM. totCrBalAmt_S
BANCS.OUTPARAM. highDrBalAmt_S
BANCS.OUTPARAM. highCrBalAmt_S
BANCS.OUTPARAM. lowDrBalAmt_S
BANCS.OUTPARAM. lowCrBalAmt_S
BANCS.OUTPARAM. cumDrTotAmt_S
BANCS.OUTPARAM. cumCrTotAmt_S
BANCS.OUTPARAM. lastRunCumBal_S
BANCS.OUTPARAM. lastRunAvgBal_S
BANCS.OUTPARAM. cumCurBal_S
BANCS.OUTPARAM. cumOpenBal_S
BANCS.OUTPARAM. cumCloseBal_S
BANCS.OUTPARAM. totSanctionLim_S
BANCS.OUTPARAM. cumTotSanctionLim_S
BANCS.OUTPARAM. totDrawingPower_S
BANCS.OUTPARAM. cumTotDrawingPower_S
BANCS.OUTPARAM. numOfAcctsOpened_S
BANCS.OUTPARAM. totNumOfAcctsOpened_S
BANCS.OUTPARAM. numOfAcctsClosed_S
BANCS.OUTPARAM. totNumOfAcctsClosed_S
BANCS.OUTPARAM. clsAcctsElgForPenal_S
BANCS.OUTPARAM. totClsAcctsElgForPenal_S

The Others-Turnover details will be available only if "DSATOInd_O" is set to YES.
Otherwise other fields should not be accessed.

BANCS.OUTPARAM. DSATOInd_O
BANCS.OUTPARAM. dsaId_O
BANCS.OUTPARAM. schemeType_O
BANCS.OUTPARAM. crncyCode_O
BANCS.OUTPARAM. turnoverInd_O
BANCS.OUTPARAM. endDate_O
BANCS.OUTPARAM. numOfDrTrans_O
BANCS.OUTPARAM. numOfCrTrans_O
BANCS.OUTPARAM. totDrBalAmt_O
BANCS.OUTPARAM. totCrBalAmt_O

BANCS.OUTPUTPARAM. highDrBalAmt_O
BANCS.OUTPUTPARAM. highCrBalAmt_O
BANCS.OUTPUTPARAM. lowDrBalAmt_O
BANCS.OUTPUTPARAM. lowCrBalAmt_O
BANCS.OUTPUTPARAM. cumDrTotAmt_O
BANCS.OUTPUTPARAM. cumCrTotAmt_O
BANCS.OUTPUTPARAM. lastRunCumBal_O
BANCS.OUTPUTPARAM. lastRunAvgBal_O
BANCS.OUTPUTPARAM. cumCurBal_O
BANCS.OUTPUTPARAM. cumOpenBal_O
BANCS.OUTPUTPARAM. cumCloseBal_O
BANCS.OUTPUTPARAM. totSanctionLim_O
BANCS.OUTPUTPARAM. cumTotSanctionLim_O
BANCS.OUTPUTPARAM. totDrawingPower_O
BANCS.OUTPUTPARAM. cumTotDrawingPower_O
BANCS.OUTPUTPARAM. numOfAcctsOpened_O
BANCS.OUTPUTPARAM. totNumOfAcctsOpened_O
BANCS.OUTPUTPARAM. numOfAcctsClosed_O
BANCS.OUTPUTPARAM. totNumOfAcctsClosed_O
BANCS.OUTPUTPARAM. clsAcctsElgForPenal_O
BANCS.OUTPUTPARAM. totClsAcctsElgForPenal_O

EXAMPLE

```
# Get the inputs from repository

sv_a = BANCS.INPUT.dsaId
sv_b = BANCS.INPUT.schemeType
sv_c = BANCS.INPUT.crncyCode
sv_d = BANCS.INPUT.asOnDate

sv_e = sv_a + "|" + sv_b + "|" + sv_c + "|" + sv_d

#call the user hook for getting DSA turnover details
sv_n = urhk_ getDSATODetailsForSchemeType (sv_e)

IF (sv_n != 0) THEN
    BANCS.OUTPUT.successOrFailure="F"
    BANCS.OUTPUT.error= "DSA Turnover details not found"
ENDIF
```

2.53 GET THE VALUE OF ENVIRONMENT VARIABLE

SYNTAX

```
sv_a = urhk_ getEnv()
```

FUNCTIONALITY

This user hook will return the value of Environment variable. The output is populated in `BANCS.OUTPUTPARAM.envValue`. The input to this userhook is the Environment variable name. The value of the Enviroment variable is obtained and is populated in **`BANCS.OUTPUTPARAM.envValue`**.

INPUT

Input is the Environment variable name.

OUTPUT

The output is available in `BANCS.OUTPUTPARAM. envValue`.

EXAMPLE

```
sv_a = BANCS.INPARAM. EnvName  
sv_b = urhk_getEnv(sv_a)  
  
Output will be in this field:  
BANCS.OUTPUTPARAM. envValue.
```

2.54 GETTING INSURANCE DETAILS FOR A SECURITY

SYNTAX

```
sv_a = urhk_getInsuranceDetailsForSecurity("")
```

FUNCTIONALITY

Using this user hook, the user can get the insurance details for the **insuCount** passed. The system gets all the insurance details for a given security and populates the `BANCS.OUTPUTPARAM` repository fields with the insurance details of the `insuCount` passed as input to the user hook.

The output is available in `BANCS.OUTPUTPARAM.successOrFailure` variable. The valid values for the field will be "S" or "F". The other output details are
`BANCS.OUTPUTPARAM.invertInsuRiskCoverEndDate`: This field indicates the expiry date of the insurance in (YYYY MM DD format)
`BANCS.OUTPUTPARAM.invertBodDate` : This field indicates the BOD date in (YYYY MM DD format)

INPUT

Input is `insuCount` – count of the insurance information to be fetched

OUTPUT

The output is available in `BANCS.OUTPUTPARAM.invertInsuRiskCoverEndDate` and `BANCS.OUTPUTPARAM.invertBodDate`. Both the dates are in YYYY MM DD format. Hence, can be used for comparison in the scripts.

EXAMPLE

This is for raising an exception code called "013" if the security has an insurance that is expired.

```
BANCS.OUTPUT.successOrFailure = "S"
BANCS.INPARAM.secuLinkageType = BANCS.INPUT.secuLinkageType
BANCS.INPARAM.acctNum = BANCS.INPUT.acctNum
BANCS.INPARAM.limitPrefix = BANCS.INPUT.limitPrefix
BANCS.INPARAM.limitSuffix = BANCS.INPUT.limitSuffix
BANCS.INPARAM.secuSrlNum = BANCS.INPUT.secuSrlNum
BANCS.INPARAM.usrhkInsuLL = BANCS.INPUT.usrhkInsuLL
BANCS.INPARAM.firstRecFlg = BANCS.INPUT.firstRecFlg

# user hook to get the all Insurances details for a security
#Initilase counter
sv_x = 1
while(sv_x > 0)
    BANCS.INPARAM.insuCount = sv_x
    sv_f = urhk_getInsuranceDetailsForSecurity("")
    sv_k = BANCS.OUTPUT.successOrFailure
    print(sv_k)
    if (sv_k == "F") then
        # either there are no insurances or all existing insurances are read
        exitscript
    endif

    # Check insurance end date and raise exception if it is expired
    print(BANCS.OUTPUT.invertBodDate)
    print(BANCS.OUTPUT.invertInsuRiskCoverEndDate)
    if (BANCS.OUTPUT.invertInsuRiskCoverEndDate != "") then
if (BANCS.OUTPUT.invertInsuRiskCoverEndDate < BANCS.OUTPUT.invertBodDate)
then
                sv_c = urhk_B2k_PutUserException("013")
                BANCS.OUTPUT.successOrFailure = "F"
                exitscript
            endif
        endif
        sv_x = sv_x + 1
    do
```

2.55 ISSUE CHEQUE LEAVES

SYNTAX

```
sv_a = urhk_IssueChequeLeaves ("")
```

FUNCTIONALITY

This user hook will return 0 on success and 1 on failure. Userhook will take the AcctNum , BeginChqNum, BeginChqAlpha, NoOfChqLvs as input in repository. It will validate the acctNum is exeisting one or not. It will accept the NoOfChqLvs between 1 and 100 including both. And the corresponding cheque leaves(with that numbers) should not be issued . After validation it will make the entry in CBT table.

INPUT

Input to userhook will be a through input repository.

```
BANCS.INPARAM.AcctNum
```

```
BANCS.INPARAM.BeginChqNum
```

```
BANCS.INPARAM.BeginChqAlpha
```

BANCS.INPARAM.NoOfChqLvs

OUTPUT

This userhook will return 0 on success and 1 on failure.

EXAMPLE

Check the sample scripts named IssueCheckLeafForPayment.sscr

2.56 CONVERTS THE LOWER STRING TO UPPER CASE

SYNTAX

```
sv_a = urhk_ UpCase()
```

FUNCTIONALITY

This user hook will convert the passed value to upper case. The output is populated in BANCS.OUTPARAM.UpString. The input to this userhook is any string. The passed string is converted into upper case and is populated in **BANCS.OUTPARAM.UpString**.

INPUT

Input is any char string.

OUTPUT

The output is available in BANCS.OUTPARAM.UpString.

EXAMPLE

```
sv_a = BANCS.INPARAM.lowString
```

```
sv_b = urhk_UpCase (sv_a)
```

Output will be in this field:

BANCS.OUTPARAM.UpString.

2.57 UPDATELIEN

SYNTAX

```
sv_a = urhk_ updateLien("")
```

FUNCTIONALITY

This USER HOOK Updates a given line amount to the given account. The other required data is also passed from the script.

INPUT

```
BANCS.INPARAM.acctNum  
BANCS.INPARAM.blockMode  
BANCS.INPARAM.lienAmt  
BANCS.INPARAM.remarks  
BANCS.INPARAM.expiryDate  
BANCS.INPARAM.reasonCode  
BANCS.INPARAM.lienValueDate  
BANCS.INPARAM.b2kId
```

OUTPUT

This userhook will return 0 on successful completion and it's success and failure result is stored in BANCS.OUTPUTPARAM.successOrFailure. Which will store 'S' for success and 'F' for failure.

EXAMPLE

```
<--start  
trace on  
BANCS.INPARAM.acctNum="BPMKS-INR"  
BANCS.INPARAM.blockMode="B"  
BANCS.INPARAM.lienAmt="3067"  
BANCS.INPARAM.remarks="NO REMARKS"  
BANCS.INPARAM.expiryDate="21-12-1998"  
BANCS.INPARAM.reasonCode="RSN"  
BANCS.INPARAM.lienValueDate="22-12-1998"  
BANCS.INPARAM.b2kId="AA3"  
sv_e = urhk_updateLien("")  
print(sv_e)  
print(BANCS.OUTPUTPARAM.successOrFailure)  
trace off  
end-->
```

2.58 USER DEFINED TOD

SYNTAX

```
sv_a = urhk_createUserDefndTOD(sv_m)
```

Where sv_m = Format\$(sv_x,"19.6f")

Where sv_x = BANCS.INPUT.TranAmt - BANCS.OUTPUTPARAM.FullEffAvailableAmt

If sv_x > = 0, then only TOD will be granted.

FUNCTIONALITY

With this userhook we can give userdefined tod, if transaction amount is greater than the effective available amount. This userhook is used depending upon the condition defined in the script. This can be called through the sample script CDPPreProc.sscr.

EXAMPLE

```

sv_f = CDOUBLE(BANCS.INPUT.TranAmt)
sv_y = sv_f/100
sv_g = BANCS.INPUT.TranCrncy
sv_c = BANCS.INPUT.Command

sv_h = urhk_getAcctDetailsInRepository(sv_d)
if (sv_h != 0) then
    MTT.Error.Description="Unable To Get GAM record"
    sv_r = urhk_MTTs_ReportErrorCondition("")
endif

sv_a = CDOUBLE(BANCS.OUTPUT.FullEffAvailableAmt)
sv_x = sv_y - sv_a
sv_m = FORMAT$(sv_x, "%19.6f")
if (sv_e == "EBA") then
    if (sv_c == "CWDR" ) then
        if (sv_x >= 0) then
            sv_z = urhk_createUserDefndTOD(sv_m)
            if(sv_z != 0) then
                BANCS.OUTPUT.successOrFailure="S"
                exitscript
            endif
        endif
    endif
endif
else
    BANCS.OUTPUT.successOrFailure="S"
    exitscript
endif

BANCS.OUTPUT.successOrFailure="S"

end-->

```

2.59 NEXT NUMBER GENERATION**SYNTAX**

```
sv_a = urhk_GetNextNumber("")
```

FUNCTIONALITY

This userhook is used to generate the next number using check digit.

INPUT

```

BANCS.INPARAM.numType
BANCS.INPARAM.nextNum
BANCS.INPARAM.checkDigitFlag
BANCS.INPARAM.prefixString
BANCS.INPARAM.suffixString
BANCS.INPARAM.applyString
BANCS.INPARAM.CheckDigitFlag
BANCS.INPARAM.acctType
BANCS.INPARAM.crncyCode
BANCS.INPARAM.crncyAlias

```

BANCS.INPARAM.custId
BANCS.INPARAM.SchmCode
BANCS.INPARAM.glSubHeadCode
BANCS.INPARAM.acctOwnership
BANCS.INPARAM.nntSolId
BANCS.INPARAM.billRegSuffix
BANCS.INPARAM.billRegPrefix
BANCS.INPARAM.financialYear
BANCS.INPARAM.nntSolAlias
BANCS.INPARAM.dcAlias
BANCS.INPARAM.cntRctType

OUTPUT

BANCS.OUTPUT.errorString
BANCS.OUTPUT.successOrFailure
BANCS.OUTPARAM.applyString
BANCS.OUTPARAM.nextNum

EXAMPLE

```

<--start

BANCS.OUTPUT.errorString = ""
BANCS.OUTPUT.checkDigit = ""
BANCS.OUTPUT.nextNum = BANCS.INPUT.nextNum
BANCS.OUTPUT.applyString = BANCS.INPUT.applyString
BANCS.OUTPUT.successOrFailure = "S"

# check for num type other than account-"A"
IF (BANCS.INPUT.numType != "A") THEN
    BANCS.OUTPUT.errorString = "No logic specified in the script for num type
other than account
"
    BANCS.OUTPUT.successOrFailure = "F"
    exitscript
ENDIF

sv_a = CLASSEXISTS("BANCS", "CHKDGT")
if (sv_a == 0) then
    CREATECLASS("BANCS", "CHKDGT", 5)
endif

BANCS.CHKDGT.applyString = BANCS.INPUT.applyString
BANCS.CHKDGT.nextNum = BANCS.INPUT.nextNum

# MODULUS 11 LOGIC STARTS
# sv_r is the remainder counter set to 1 for outer while loop.
sv_r = 1

# sv_d is the counter in outer while loop.if sv_r equals 1, calls the user hook.
sv_d = 0

# sv_l stores the length of apply string.
sv_l = 0

# Find the remainder of the apply string which is not equal to 1.
while (sv_r == 1)
    sv_b = 2
    sv_s = 0
    sv_l = STRLEN(BANCS.CHKDGT.applyString)

    # sv_c is the counter for the inner while loop.
    sv_c = 1

    IF (sv_d > 0) THEN
        BANCS.INPARAM.numType = BANCS.INPUT.numType
        BANCS.INPARAM.nextNum = BANCS.INPUT.nextNum
        BANCS.INPARAM.checkDigitFlag = BANCS.INPUT.checkDigitFlag
        BANCS.INPARAM.prefixString = BANCS.INPUT.prefixString
        BANCS.INPARAM.suffixString = BANCS.INPUT.suffixString

        # Caution: Always use this user hook to get the next number
        sv_u = urhk_GetNextNumber("")
        if (sv_u != 0) then
            BANCS.OUTPUT.errorString = "could not fetch next number"
            BANCS.OUTPUT.successOrFailure = "F"
            exitscript
        endif

        BANCS.CHKDGT.applyString = BANCS.OUTPARAM.applyString
        BANCS.CHKDGT.nextNum = BANCS.OUTPARAM.nextNum
    ENDIF
    sv_l = STRLEN(BANCS.CHKDGT.applyString)

    GOSUB getModuloRemainder

    sv_x = sv_s / 11
    sv_r = sv_s - (sv_x * 11)
    sv_d = sv_d + 1

    # sub routine is outside the if-endif condition
    getModuloRemainder:
        while (sv_c <= sv_l)
            sv_i = CINT(GETSTRING(CHARAT(BANCS.CHKDGT.applyString,sv_l -
sv_c)))

            sv_s = sv_s + (sv_b * sv_i)
            sv_b = sv_b + 1
            IF (sv_b > 7) THEN

```

```

                                sv_b = 2
                                ENDIF
                                sv_c = sv_c + 1
                                do
                                return
                                do

                                # sv_z is checkDigit
                                # sv_r on exit from while loop.
                                IF (sv_r > 0) THEN
                                    sv_z = 11 - sv_r
                                ELSE
                                    sv_z = 0
                                ENDIF
                                BANCS.OUTPUT.successOrFailure = "S"
                                BANCS.OUTPUT.nextNum = BANCS.CHKDGT.nextNum
                                BANCS.OUTPUT.checkDigit = format$(sv_z, "%d")
                                BANCS.OUTPUT.applyString = BANCS.CHKDGT.applyString

                                exitscript
                                end-->

```

3 ONLINE SCRIPTING FUNCTIONS

Can be used in the context of any scripts invoked by Online programs only

3.1 SET THE DEFAULT ATTRIBUTE OF A FIELD IN THE FORM

This function will provide a facility to change the attributes of a field in the form. It can be used in the context of WorkFlow or in the script executed during the Pre Form Load event (xxxxxxxPreLoad.scr) only (refer to Scripting Events document for more details about Preload scripts)

For setting the attribute of a field in any other online event refer to urhk_TBAF_ChangeFieldAttrib in this document.

SYNTAX

```
sv_a = urhk_TBAF_SetAttrib(Variable)
```

Variable needs to be formatted as

"<formname>.<blockname>.<fieldname>|Attribute"

Attribute	M	Mandatory
	P	Protect (Cursor and data entry not allowed)
	H	Hide
	E	called as Entry Allowed Cursor entry allowed, Data Entry not allowed

The variable can be a scratch pad variable (sv_b) or a string ("baff0009.datablk2.cust_comu_addr1|M")

☞ *Note: Use dispfields utility for finding this information. This utility is documented in Workflow manual.*

FUNCTIONALITY

This function can be used to change the attribute of any field in a form that is currently being loaded or about to be called for loading (in the case of Workflow). By default, all fields in a form that is loaded carry the attribute assigned to it by Infosys. However, a Bank may wish to customise the attributes of certain fields in certain forms which is different from the Infosys setting. For example, a Bank may decide that the 'Communication address line 1' field in the Customer master maintenance screen is to be made mandatory (default setting is non-mandatory). Then this function can be used in the form load event of the form baff0009 to set the attribute of the field 'cust_comu_addr1' as mandatory.

Please note that when a field has a default setting of 'Mandatory', setting 'Protect' or 'Entry allowed' attributes to that field is not allowed.

Also, once an attribute for a field has been set, setting the attribute of that field again (even with a different attribute) before the form is unloaded, will not have any effect.

INPUT

Input String contains two parts separated by a '|'

1. Field name
2. Attribute

EXAMPLE

- "baff0009.datablk2.cust_comu_addr1|M"

Field name should be given as FormName.BlockName.FieldName.

In the above example the attribute is given as M to cust_comu_addr1 field which makes it mandatory to enter a value in the field.

Valid values for Attribute are :

P – Protect – The field is made non-enterable and tab will skip past this field

M – Mandatory – The field is made mandatory and something needs to be entered in this field before proceeding

H – Hide – The field value is not displayed

E – Entry allowed – The cursor can be positioned in the field but no data can be entered. Useful if function keys need to be used

in a protected field (to scroll the field or to explode for example)

OUTPUT

Always returns 0 as return value

The form will display the new attribute after it is loaded if the specified field exists in the form and the attribute does not conflict with the default setting. Otherwise, nothing happens.

EXAMPLE:

The logic discussed in the 'functionality section' can be implemented by coding baff0009PreLoad.scr as follows:

```
<--start
sv_a = urhk_TBAF_SetAttrib("baff0009.datablk2.cust_comu_addr1|M")

exitscript
end-->
```

3.2 SET THE DEFAULT VALUE FOR A FIELD IN THE FORM DURING FORM LOAD.

This function will populate the given value for a field in a form that is currently being loaded. It can be used in the context of WorkFlow or in the script executed during the Form Load event (xxxxxxxPreLoad.scr) only (refer to Scripting Events document for more details about Preload scripts)

For changing the value of a field in any other online event refer to urhk_TBAF_ChangeFieldValue in this document.

SYNTAX

sv_a = urhk_TBAF_SetValue (Variable)

As discussed for TBAF_SetAttrib above.

The variable can be a scratch pad variable (sv_b) or a string ("tdfe3201.datablk1.ledg_num|1")

FUNCTIONALITY

This function can be used to set the default value of any field in a form that is currently being loaded or about to be called for loading(in the case of WorkFlow). By default, many fields in a form that is loaded carry a NULL value assigned to it by Infosys. However, a Bank may wish to assign a default values of certain fields in certain forms. For example, the Customer master maintenance function assigns no value to the 'Customer Non Resident', 'Customer staff' and 'Customer minor' fields. A

Bank may decide that a default value of 'N' needs to be assigned to these 3 fields. Then this function can be used in the form load event of the form baff0009 to set the default value as 'N' for these three fields.

It is also used in conjunction with **urhk_TBFAF_GetValue** to get back values from a form after 'Commit'. In order to do that, this function should be used to specify all the form fields whose values (as they exist after commit processing) is to be accessed, and then after commit processing use **urhk_TBFAF_GetValue** to access their values.

Please note that once a value for a field has been set, setting the value of that field again (even with a different value) before the form is unloaded, will not have any effect.

INPUT

Input String contains three parts separated by a '|'

1. Field name
2. Value - Max. size is 255 characters
3. Repository [Optional]

EXAMPLE –

"tdfe3201.datablk1.ledg_num|1"

Field name should be given as FormName.BlockName.FieldName.

Value is the data to be populated in the field. It can have a NULL value to set the value of a field to NULL. Ex. "tdfe3201.datablk1.ledg_num|"

Repository name is meant for advanced use(in the context of Workflow. See discussion on TEMP repositories under **urhk_WFS_TransferUser** and

urhk_WFS_CreateWFItem sections) and typically should not be specified as shown in the example above. In this case the repository INTBAF is assumed. However, when we intend to specify the list of fieldnames whose value we need to access after 'commit' processing, this repository should be specified as OUTTBAF.

OUTPUT

Always returns 0 as return value.

When the repository is not specified then the form will display the new value after it is loaded if the specified field exists in the form, otherwise, nothing happens.

When the repository OUTTBAF is specified, nothing happens when the form is loaded, but you can then use **urhk_TBFAF_GetValue** in post-commit events to access the value of the field.

EXAMPLE:

The logic discussed in the 'functionality section' can be implemented by coding baff0009PreLoad.scr as follows:

```
<--start
sv_a = urhk_TBAF_SetValue("baff0009.datablk1. cust_nre_flg|N")
sv_a = urhk_TBAF_SetValue("baff0009.datablk1. cust_emp_flg|N")
sv_a = urhk_TBAF_SetValue("baff0009.datablk1. cust_minor_flg|N")
```

```
exitscript
end-->
```

The logic for getting the assigned Customer Id (after commit) can be implemented by coding a Workflow script which calls CUMM menu Option, as follows:

```
<--start
sv_a = urhk_TBAF_SetValue("baff0009.datablk6.cust_id| OUTTBAF")
sv_b = urhk_WFS_CallMenuOption("CUMM|0|3")
if(sv_b == 1) then
  exitscript
endif
sv_d = CLASSEXISTS("OUTTBAF","baff0009")
if (sv_d == 1) then
  sv_a = urhk_TBAF_GetValue("baff0009.datablk6.cust_id ")
  STDWFS.GLBDATA.cust_id = B2KTEMP.TEMPSTD.TBAFRESULT
  DELETECLASS("OUTTBAF","baff0009")
endif
```

```
exitscript
end-->
```

3.3 CHANGE THE FIELD VALUE IN THE FORM.

This function will change the value of a field in the current form to the specified value.

SYNTAX

```
sv_a = urhk_TBAF_ChangeFieldValue(Variable)
```

The variable can be a scratch pad variable (sv_b) or a string ("optionblk.option_code|P")

FUNCTIONALITY

This user hook will change the value of a field in the form. This user hook should not be used in Pre Form load or Post Form Load scripts. In the Pre Form load scripts use the function **urhk_TBAF_SetValue**.

INPUT

Input string has two parts separated by a '|'

1. Field name.
2. Value - Max. size is 255 characters

Example - "optionblk.option_code|P"

Field name should be given as BlockName.FieldName.

Value is the data to be populated in the field. It can have a NULL value to change the value of a field to NULL. Ex. "datablk1.ledg_num|"

OUTPUT

Always returns 0 as return value.

The form will display the new value immediately, if the field is found in the current form, else nothing happens.

EXAMPLE:

Please refer to Example section of **urhk_TBAF_SetReplayKey**.

In **tm1.scr** this function is used to populate the option code as 'P'to request for posting the transaction.

3.4 INQUIRE THE FIELD VALUE IN THE FORM.

This function will get the value of a field in the current form.

SYNTAX

```
sv_a = urhk_TBAF_InquireFieldValue(Variable)
```

The variable can be a scratch pad variable (sv_b) or a string ("optionblk.option_code")

FUNCTIONALITY

This function will get the value of a field in the form in the repository field B2KTEMP.TEMPSTD.TBAFRESULT. This function should not be used during Pre Form load or Post Form Load events.

INPUT

Input string has one part

Field Name

EXAMPLE –

```
"optionblk.option_code"
```

Field name should be given as BlockName.FieldName.

OUTPUT

Always returns 0 as return value

Required value will be available in B2KTEMP.TEMPSTD.TBAFRESULT if the field is found in the current form, else B2KTEMP.TEMPSTD.TBAFRESULT will contain "".

Example:

In this example, a sample script has been written which can be used to invoke all the mandatory forms to be visited while opening a deposit account automatically.

First we will code the PreLoad script of the deposit form to execute an 'accept-key' when the option block is visited.

Then we will code the pre-replay script of the above key to populate the appropriate option code into the option field.

Then we will code the post-replay script of the above key to reset the replay key so that when the option block is visited again the next option is chosen.

Script Name :- **tdfe3201PreLoad.scr**

```
<--start
sv_a = urhk_TBFAF_SetReplayKey("optnblk.key-f2|tdoptpre.scr|0|tdoptpost.scr|0")
exitscript
end-->
```

Script Name :- **tdoptpre.scr**

```
<--start

sv_b = cint(INTBAF.INTBAFC.TbafEventStep)

if (sv_b == 0) then
    GOTO STEP0
endif

if (sv_b == 1) then
    GOTO STEP1
endif

if (sv_b == 2) then
    GOTO STEP2
endif

if (sv_b == 3) then
    GOTO STEP3
endif

exitscript

# Populate option as G to visit general details.

STEP0:
sv_a = urhk_TBFAF_ChangeFieldValue("optnblk.acct_opn_option|G")
exitscript

# Populate option as N to visit Nominee details

STEP1:
sv_a = urhk_TBFAF_ChangeFieldValue("optnblk.acct_opn_option|N")
exitscript

# Populate option as F to visit Flow details.

STEP2:
sv_a = urhk_TBFAF_ChangeFieldValue("optnblk.acct_opn_option|F")
exitscript

# Populate option as V to visit Advance details.

STEP3:
sv_a = urhk_TBFAF_ChangeFieldValue("optnblk.acct_opn_option|V")
exitscript

end-->
```

Script Name :- **tdoptpost.scr**

```
<--start

sv_b = cint(INTBAF.INTBAFC.TbafEventStep)

if (sv_b == 0) then
    GOTO STEP0
endif

if (sv_b == 1) then
    GOTO STEP1
endif

if (sv_b == 2) then
    GOTO STEP2
endif

exitscript

# Populate Replay key to execute Step 1 or Step 2 of pre-script
```

```

STEP0:
sv_a = urhk_TBAF_InquireFieldValue (databl1.nom_available_flg)
if (B2KTEMP.TEMPSTD.TBAFRESULT = "Y")
    sv_a = urhk_TBAF_SetReplayKey("optnblk.key-
f2|tdoptpre.scr|1|tdoptpost.scr|1")
else
    sv_a = urhk_TBAF_SetReplayKey("optnblk.key-
f2|tdoptpre.scr|2|tdoptpost.scr|2")
exitscript

# Populate Replay key to execute Step 2 of pre-script

STEP1:
    sv_a = urhk_TBAF_SetReplayKey("optnblk.key-
f2|tdoptpre.scr|2|tdoptpost.scr|2")
exitscript

# Populate Replay key to execute Step 3 of pre-script

STEP2:
    sv_a = urhk_TBAF_SetReplayKey("optnblk.key-f2|tdoptpre.scr|3")
exitscript

end-->

```

3.5 CHANGE THE ATTRIBUTE OF A FIELD IN THE FORM.

This function will change the attribute of a field in the form to the specified value.

SYNTAX

```
sv_a = urhk_TBAF_ChangeFieldAttrib(Variable)
```

The variable can be a scratch pad variable (sv_b) or a string ("dispblk2.acct_bal|H")

FUNCTIONALITY

This user hook will change the attribute of a field in the form. This user hook should not be used in Pre Form Load and Post Form Load scripts. In the Pre Form Load scripts use the function **urhk_TBAF_SetAttrib**.

INPUT

Input string has two parts separated by a '|'

1. Field Name
2. Attribute

Example – "dispblk2.acct_bal|H"

Field name should be given as BlockName.FieldName.

Valid values for Attribute are:

P – Protect – The field is made non-enterable and tab will skip past this field

M – Mandatory – The field is made mandatory and something needs to be entered in this field before proceeding

H – Hide – The field value is not displayed
 U – UnHide – The field value is displayed
 E – Entry allowed – The cursor can be positioned in the field but no data can be entered. Useful if function keys need to be used in a protected field (to scroll the field or to explode for example)

OUTPUT

Always returns 0 as return value

The form will display the changed attribute immediately if the field is found in the current form, else nothing happens.

EXAMPLE:

Let us assume that a Bank wants to hide the account balance field in the Transaction maintenance screen for Customer accounts. This is achieved by coding the script as under.

```
Script Name - CheckAndDispAcctBal.scr

<--start

sv_a = BANCS.INPUT.AcctId

sv_b = urhk_getAcctDetailsInRepository(sv_a)

# if getAcctDetailsInRepository function returns failure exit out of script.

if( sv_b == 1 ) then
    exitscript
endif

                                sv_c="E"

# -----
                                # If the acctOwnership flag of ACCOUNT class is equal to "E"
                                # then hide the dispblk2.acct_bal field
                                # -----

if (BANCS.OUTPUTPARAM.acctOwnership == sv_c) then
    sv_z = "dispblk2.acct_bal|H"
    sv_z = urhk_TBAF_ChangeFieldAttrib(sv_z)
else
    sv_z = "dispblk2.acct_bal|U"
    sv_z = urhk_TBAF_ChangeFieldAttrib(sv_z)
endif
    exitscript
end-->
```

3.6 GET THE 'POST-COMMIT' VALUE FROM A FIELD IN THE FORM.

This function will get the value of a field in the form, whose 'post-commit' value has been requested by using the **urhk_TBAF_SetValue** as described above. This function should be used only in a post-commit event of a form(Post Form Load event for example) or a Workflow script after a 'CallMenuOption' statement.

If the value of a form field is required to be accessed in any other event, then the function **urhk_TBAF_InquireFieldValue** should be used.

SYNTAX

```
sv_a = urhk_TBAF_GetValue (Variable)
```

The variable can be a scratch pad variable (sv_b) or a string ("baff0009.datablk6.cust_id")

FUNCTIONALITY

This function will typically be used in the context of a WorkFlow script, eventhough it can theoretically be used in any post-commit event. This function (along with **urhk_TBAF_SetValue**) is used to chain multiple menu options into a single business task by passing data from one step to another. For example, we could use this function in a 'Account Opening WorkFlow script' which executes 'CUMM', and 'OAAC' in sequence, to pass the Customer Id assigned in Customer creation step to the Account opening step.

INPUT

Input string has two parts separated by a '|'

1. Field name.
2. Repository [Optional]

Example – "baff0009.datablk6.cust_id"

Field name should be given as FormName.BlockName.FieldName.

Repository name is meant for advanced use (in the context of Workflow. See discussion on TEMP repositories under **urhk_WFS_TransferUser** and **urhk_WFS_CreateWFItem** sections) and typically should not be specified as shown in the example below. If not specified OUTTBAF is the repository that is assumed.

OUTPUT

Always returns 0 as return value.

The value of the required field will be available in the repository field B2KTEMP.TEMPSTD.TBAFRESULT if the field exists in the specified form and was correctly specified in **urhk_TBAF_SetValue** (with OUTTBAF repository) prior to commit. Otherwise, B2KTEMP.TEMPSTD.TBAFRESULT will contain "".

EXAMPLE:

In this example a sample script written to get the value of customer id from CUMM and put in account opening menu option OAAC.

```

<--start
STEP0
sv_a = urhk_TBAF_SetValue("baff0009.datablk6.cust_id| |OUTTBAF")
# we have told the system that we are interested in the post-commit value of
#baff0009.datablk6.cust_id
sv_a = urhk_WFS_CallMenuOption("CUMM|0|1")
if(sv_b == 1) then
    exitscript
endif
STEP1
sv_d = CLASSEXISTS("OUTTBAF","baff0009")
if (sv_d == 1) then
    # We have asked the system to give us the post-commit value of
    #baff0009.datablk6.cust_id
    sv_a = urhk_TBAF_GetValue("baff0009.datablk6.cust_id ")
    DELETECLASS("OUTTBAF","baff0009")
endif

sv_v = "baf3013.funcblk.cust_id|" + B2KTEMP.TEMPSTD.TBAFRESULT
sv_a = urhk_TBAF_SetValue(sv_v)
#We have told the system that it should take the customer id assigned above as the
#customer id in the account opening form (baf3013)
STEP2
sv_a = urhk_WFS_CallMenuOption("OAC|2|")
if(sv_b == 1) then
    exitscript
endif

STDWFS.WFSINPUT.WfsStatus = "D"
exitscript
end-->

```

3.7 EXECUTING A KEY-BASED EVENT FROM THE SCRIPT.

This function will cause the execution of the Infosys defined logic associated with the specified key-based event, along with execution of a specified script before and after the execution of the specified key-based logic (refer to Finacle Forms Customisation manual for more details regarding how to determine what key based events are associated with a form), when the system encounters the specified Block. Only those key-based events that are already defined in the Infosys supplied form can be specified in the context of that form. Though this function can be used in any form-based event, its typical usage is in the context of a WorkFlow script.

SYNTAX

```
sv_a = urhk_TBAF_SetReplayKey (Variable)
```

The variable can be a scratch pad variable (sv_b) or a string ("funcblk.key-f2|TM1.scr|2|TM1.scr|3")

FUNCTIONALITY

This function gives the capability of automating certain operations in a form. Most Finacle forms follow a standard sequence of execution punctuated by data accepted from the user. For example a standard sequence in a maintenance form is 'enter values for field(s) in function block', hit the 'accept-key', 'enter values for field(s) in data block1', hit the 'next-blk' key(or accept-key and enter option in option block)', enter values for field(s) in data block2, hit the 'accept-key' and finally hit the 'commit key'. Now in certain 'WorkFlow' operations, values for a field in a form block maybe

determined without any input from the user (either because default values can be supplied by the script or because values from a previous form can be supplied by the script or there are no values to be supplied). In such cases, this function can be used to automate the operation.

This function works like a 'stack' in the sense that multiple calls to this function will stack-up the 'Replay keys' one after another.

Having used this function, the logic of execution will be as follows:

1. Before accepting any user input in the current block, the system checks whether there is a 'Replay key' at the top of the stack, that has been specified for that block.
2. If so, then it executes the pre-script associated with that 'Replay key'(if any specified), executes the key-based event logic (which has been defined by Infosys), and then executes the post-script associated with that 'Replay key'(if any specified). It then removes the entry from the 'stack' so that the next 'Replay key' comes onto the top.
3. If no 'Replay key' has been specified it goes into the normal user input acceptance mode.

INPUT

Input string has three parts separated by a '|'

Block name.Key

PreScriptName|Step [Optional]

PostScriptName|Step [Optional]

Example - "funcblk.key-f2|TM1.scr|2|TM1.scr|3"

If the block name is not specified then key will be executed in the first block that is encountered after this 'Replay key' comes to the top of the 'stack'.

If Prescript name is specified then before executing the key-event the specified script will be executed

If Postscript is specified then specified script will be executed after the key-based event logic(as defined by Infosys) is executed.

Key can take on any valid values as documented in the Finacle Forms Customisation manual for 'Logical Keys'.

OUTPUT

Always returns 0 as return value.

When the PreScript or PostScript is executed, all the standard repository fields that is given out in any Finacle script event (Refer Scripting Events document)

will be given. In addition the specified step value will be available in the repository field INTBAF.INTBAFC.TbafEventStep.

EXAMPLE:

Let us assume that a Bank frequently opens a specific type of fixed deposit using a cash deposit from the Customer. So it decides that it needs a specific menu option for this, which will accept only the min basic details, create the account and post the cash transaction. The WorkFlow script shown below is an example of how to do this.

Script file name :- **Opndepac.scr**.

```

<--start

sv_d = CLASSEXISTS("STDWFS", "GLBDATA")
if (sv_d == 0) then
    CREATECLASS("STDWFS", "GLBDATA", 5)
endif
sv_b = cint(STDWFS.WFSINPUT.NextStep)

if (sv_b == 0) then
    GOTO STEP0
endif

if (sv_b == 1) then
    GOTO STEP1
endif

if (sv_b == 2) then
    GOTO STEP2
endif
exitscript

# upto now standard WorkFlow statements have been executed

STEP0
# Now accept the min. basic information regarding the deposit

STDWFS.WFSINPARAMVAL.custId="Customer Id"
STDWFS.WFSINPARAMVAL.SchemeCode="Scheme Code"
STDWFS.WFSINPARAMVAL.GlSubHeadCode="GL Sub Head Code"
STDWFS.WFSINPARAMVAL.depositAmount="Deposit Amount"
STDWFS.WFSINPARAMVAL.depPeriodMonths="Dep Prd Months"
STDWFS.WFSINPARAMVAL.depPeriodDays="Dep Prd Days"
sv_a = urhk_WFS_ShowParamAcptFrm("")

STEP1:

# Populate Scheme code, GL Sub Head Code, deposit Amount, Deposit Period in
# OAAC forms.

sv_v = "bafe3013.funcblk.schm_code|" + STDWFS.WFSOUTPARAM.schemeCode
sv_a = urhk_TBAF_SetValue(sv_v)
sv_v = "bafe3013.funcblk.gl_sub_head_code|" + STDWFS.WFSOUTPARAM.glSubHead
sv_v = "tdfe3201.datablk1.deposit_amount|" + STDWFS.WFSOUTPARAM.depositAmount
sv_a = urhk_TBAF_SetValue(sv_v)
sv_a = urhk_TBAF_SetValue(sv_v)
sv_v = "tdfe3201.datablk1.deposit_period_mths|" + STDWFS.WFSOUTPARAM.depPeriodMonths
sv_a = urhk_TBAF_SetValue(sv_v)
sv_v = "tdfe3201.datablk1.deposit_period_days|" + STDWFS.WFSOUTPARAM.depPeriodDays
sv_a = urhk_TBAF_SetValue(sv_v)

# Define field for getting the Account Number in OUTTBAF.

sv_a = urhk_TBAF_SetValue("tdfe3201.funcblk.acct_num| |OUTTBAF")
sv_a = 0
sv_a = urhk_WFS_CallMenuOption("OAAC|1|2")
if (sv_a == 1) then
    exitscript
endif

STEP2

# Get the account number and store in GLBDATA class.
sv_d = CLASSEXISTS("OUTTBAF", "tdfe3201")
if (sv_d == 1) then
    sv_a = urhk_TBAF_GetValue("tdfe3201.funcblk.acct_num")
    STDWFS.GLBDATA.tmpAcctNum = B2KTEMP.TEMPSTD.TBAFRESULT
    DELETECLASS("OUTTBAF", "tdfe3201")
endif

sv_a = urhk_TBAF_SetValue("bafe3012.funcblk.func_code|A")
sv_a = urhk_TBAF_SetValue("bafe3012.funcblk.tran_type|C")
sv_a = urhk_TBAF_SetValue("bafe3012.funcblk.tran_sub_type|NR")

sv_v = "bafe3012.datablk.tran_amt|" + STDWFS.WFSOUTPARAM.depositAmount
sv_a = urhk_TBAF_SetValue(sv_v)

sv_v = "bafe3012.datablk.acct_num|" + STDWFS.GLBDATA.tmpAcctNum
sv_a = urhk_TBAF_SetValue(sv_v)

# Play the key strokes so that Transaction will be created and posted.
# Play F2 to go to data block.

```



```

sv_a = urhk_TBFAF_SetReplayKey("funcblk.key-f2")
# Play F2 to go to Option block.
sv_a = urhk_TBFAF_SetReplayKey("datablk.key-f2")
sv_a = urhk_TBFAF_SetReplayKey("datablk1.key-f2")
sv_a = urhk_TBFAF_SetReplayKey("dummyblk.key-f2")

# Play F2 at Option block. and write tml.scr to populate the 'P' - Post request
value
# and pass as pre script. Pre script will be executed before firing F2.
sv_a = urhk_TBFAF_SetReplayKey("optionblk.key-f2|tml.scr|0")
sv_a = urhk_TBFAF_SetReplayKey("optionblk.key-nxtrec")
sv_a = urhk_TBFAF_SetReplayKey("optionblk.key-f2")

# Play COMMIT at Option block to post transaction.

sv_a = urhk_TBFAF_SetReplayKey("optionblk.key-commit")

# Play KEY - F1 to exit form TM menu option.

sv_a = urhk_TBFAF_SetReplayKey("dispblk3.key-f1")
sv_a = urhk_TBFAF_SetReplayKey("funcblk.key-f1")

#Call the TM menu option for posting the cash transaction
sv_a = urhk_WFS_CallMenuOption("TM|2|")

STDWFS.WFSINPUT.WfsStatus = "D"
exitscript
end-->

```

In **tml.scr** script

```

<--start
sv_b = cint(INTBAF.INTBAFC.TbafEventStep)
if (sv_b == 0) then
    GOTO STEP0
Endif
Exitscript

# Populate option code as P for posting the transaction.

STEP0:
sv_a = urhk_TBFAF_ChangeFieldValue("optionblk.option_code|P")

exitscript
end-->

```

3.8 CALLING GENERAL PARAMETER ACCEPTANCE FORM

This function will invoke a specified form. The form should be script-enabled in the sense that it should operate only of repository variables, both for input and output. bafi2028,aafe0003, aafe0004 is an example of such a form.

SYNTAX

```
sv_a = urhk_B2K_ShowParamAcptFrm("bafi2028").
```

FUNCTIONALITY

This function will bring up the form specified in the argument. As of now only the forms bafi2028,aafe0003,aafe0004 are supported. Each supported form will have its own inputs and outputs.

INPUT

The call to the function requires a form name which is to be brought up for accepting data from the user. As of now 'bafi2028,aafe0003,aafe0004' are supported.

Inputs for bafi2028

The form 'bafi2028' requires certain inputs for bringing up the form, displaying the literals and accepting data. The inputs are to be given by way of a repository field in FRMATTRIB class of BANCS repository, for each field that needs to be displayed/accepted. For each field the following is required to be given

- 1) The name of the field - assigned by referring to this name while populating the rest of the information for the field.
- 2) The literal for the field – first part of the value assigned to the above variable. Maximum length for literal is 30 characters
- 3) Length of the value of the field - second part of the value assigned to the above variable separated by '|'. Maximum length for value is 100 characters
- 4) Field to be protected or not (P/N) – third part of the value assigned to the above variable separated by '|'

Protected field

Ex: BANCS.FRMATTRIB.senderSwiftCode="Sender Swift Code|12|P"

Field name	Literal	Field length

If a default value needs to be specified for any field then that value needs to be assigned to the corresponding output field. For ex. To assign a default value of 'XXX' for the senderSwiftCode, we should do the following before calling this function :

BANCS.FRMVALUE.senderSwiftCode = "XXX"

A well known field BANCS.INPARAM.ErrorMesg can be used to force the form to display an Error message.

A well known field BANCS.FRMVALUE.FormTitle can be used to display the title in the form.

INPUTS for aafe0003

There are 4 enterable fields in this form, one of which is the function code field to accept the character input of the function to be carried out by the calling workflow script. Three other fields are free fields to accept any criteria required for the function to be carried out.

Enterable here means the field in which a value can be given on screen by the end user. The additional functionality :

When funcblk.pg0_exe_name is specified by the user before calling the form - either in the formload script or in the calling workflow script, on pressing <key-f2> the Print Parameter form will be brought up and the executable (com script or any other executable) specified in the above field will be executed. the executable should be available in the appropriate system area.

INPUTS for aafe0004

There are 40 enterable fields in this form, spread in total 4 pages, each page containing 10 fields. These fields can be used to accept data from the end user on the screen and then to call appropriate script to build a workflow.

Additional functionality:

When funcblk.exe_name is specified by the user before calling the form - either in the formload script or in the calling workflow script, on pressing <key-f2> the Print Parameter form will be brought up and the executable (com script or any other executable) specified in the above field will be executed. the executable should be available in the appropriate system area. If funcblk.pg0_mode is populated with the value 'I' then, input will not be allowed in any of the fields (fields will still be enterable). Also, the listing will not be allowed in any of the above fields even if the pg0_field-<x>_type is specified. Here also type can be specified for each field and listing will be available based on the type.

OUTPUT

Always returns 0 as return value.

The values entered in the screen for the specified fields will be available in fields of FRMVALUE class of BANCS repository, after the user hits the <ACCEPT> or <COMMIT> key in the form. The name of the field will be the same as the name assigned in the input described above. i.e. the value that the user entered for the 'senderSwiftCode' field will be available in BANCS.FRMVALUE.senderSwiftCode field.

EXAMPLE:

Example for bafi2028

The following is a sample script which is used to manage a Swift message(mt410) in Finacle. Among other things it requires user input for certain fields like sender swift

code, Sender to receiver info etc. These user input are taken by using the **urhk_B2K_ShowParamAcptFrm("bafi2028")** function.

```

<--start
trace on
# *****
# Script for generation of MT410 swift message
# *****
#
# While forming the message, even if the optional fields are not
# concatenated, a new line separator "|" (pipe character)
# should be concatenated.
#
# In Generate mode, apart from other INPUT repository variables
# (addOrMod) will be populated by the calling program.
#
# In Modify mode, INPUT repository variables (status, addOrMod)
# will be populated by the calling program. Also, all the field[n]
# INPUT variables will be populated.
#
# *****

# *****
# Populate the FRMVALUE fields for display in bafi2028
# in modify mode.
# *****

if (BANCS.INPUT.addOrMod == "M") then
    BANCS.FRMVALUE.senderSwiftCode      =BANCS.INPUT.field1
    BANCS.FRMVALUE.mtNumber               =BANCS.INPUT.field2
    BANCS.FRMVALUE.receiverSwiftCode     =BANCS.INPUT.field3
    BANCS.FRMVALUE.billId                =BANCS.INPUT.field4
    BANCS.FRMVALUE.referenceNum          =BANCS.INPUT.field5
    BANCS.FRMVALUE.amountAcknowledged    =BANCS.INPUT.field6
    BANCS.FRMVALUE.senderToReceiverInfo1=BANCS.INPUT.field7
    BANCS.FRMVALUE.senderToReceiverInfo2=BANCS.INPUT.field8
    BANCS.FRMVALUE.senderToReceiverInfo3=BANCS.INPUT.field9
    BANCS.FRMVALUE.senderToReceiverInfo4=BANCS.INPUT.field10
    BANCS.FRMVALUE.senderToReceiverInfo5=BANCS.INPUT.field11
    BANCS.FRMVALUE.senderToReceiverInfo6=BANCS.INPUT.field12
endif

# *****
# Message the field values present in OUTPARAM
# when in ADD mode
# *****
if (BANCS.INPUT.addOrMod == "A") then

    # -----
    # populate MT number
    # -----
    BANCS.FRMVALUE.mtNumber="410"

    # -----
    # Populate amount Acknowledged
    # -----
    sv_s=BANCS.FRMVALUE.billAmount
    call("swiftAmount.scr")
    BANCS.FRMVALUE.billAmount= sv_t
    sv_w=BANCS.FRMVALUE.dueDate
    call("swiftDate.scr")
    BANCS.FRMVALUE.dueDate = sv_x
    sv_a = sv_x + BANCS.FRMVALUE.billCrncy
    sv_a = sv_a + BANCS.FRMVALUE.billAmount
    BANCS.FRMVALUE.amountAcknowledged = sv_a

    # -----
    # Sender to Receiver information
    # -----
    BANCS.FRMVALUE.senderToReceiverInfo1=""
    BANCS.FRMVALUE.senderToReceiverInfo2=""
    BANCS.FRMVALUE.senderToReceiverInfo3=""
    BANCS.FRMVALUE.senderToReceiverInfo4=""
    BANCS.FRMVALUE.senderToReceiverInfo5=""
    BANCS.FRMVALUE.senderToReceiverInfo6=""

    # -----
    # default value for status of message
    # -----
    BANCS.FRMVALUE.status="N"
endif
# Messaging of data in ADD mode complete

# *****

```

```

# Populate the FRMATTRIB fields - Provide definition for the bafi2028 form
# *****

BANCS.FRMVALUE.FormTitle="MT410 SWIFT Message Generation"

BANCS.FRMATTRIB.senderSwiftCode="Sender Swift Code|12|P"
BANCS.FRMATTRIB.mtNumber="MT|3|P"
BANCS.FRMATTRIB.receiverSwiftCode="Receiver Swift Code|12|P"
BANCS.FRMATTRIB.billId="Sending Bank's TRN|16|P"
BANCS.FRMATTRIB.referenceNum="Related reference|16|P"
BANCS.FRMATTRIB.amountAcknowledged="Amount Acknowledged|24|P"
BANCS.FRMATTRIB.senderToReceiverInfo1="Sender to Receiver Info|35|N"
BANCS.FRMATTRIB.senderToReceiverInfo2=" |35|N"
BANCS.FRMATTRIB.senderToReceiverInfo3=" |35|N"
BANCS.FRMATTRIB.senderToReceiverInfo4=" |35|N"
BANCS.FRMATTRIB.senderToReceiverInfo5=" |35|N"
BANCS.FRMATTRIB.senderToReceiverInfo6=" |35|N"
# -----
# status field
# -----
BANCS.FRMATTRIB.status="Status of Message|1|N"

# *****
# Call general acceptance form
# Conditional validations can be done here.
# *****
BANCS.INPARAM.ErrorMesg = " "
while(BANCS.INPARAM.ErrorMesg != "")
    sv_a = urhk_B2K_ShowParamAcptFrm("bafi2028")
    sv_b = BANCS.FRMVALUE.status
    if ((sv_b == "N") or (sv_b == "R") or (sv_b == "D")) then
        BANCS.INPARAM.ErrorMesg = " "
    else
        BANCS.INPARAM.ErrorMesg = "Status can have only the following values. [N-Not
ready, R
-Ready, D-Delete]"
    endif
endwhile
do

# *****
# Form the swift message
# For optional tags check if the value is null or not
# *****
# -----
# Form MT410 message
# -----
sv_a = BANCS.FRMVALUE.senderSwiftCode
sv_a = sv_a + "|" + BANCS.FRMVALUE.mtNumber
sv_a = sv_a + "|" + BANCS.FRMVALUE.receiverSwiftCode
sv_a = sv_a + "|" + ":20:" + BANCS.FRMVALUE.billId
sv_a = sv_a + "|" + ":21:" + BANCS.FRMVALUE.referenceNum
sv_a = sv_a + "|" + ":32A:" + BANCS.FRMVALUE.amountAcknowledged

if ((BANCS.FRMVALUE.senderToReceiverInfo1 != "") or
(BANCS.FRMVALUE.senderToReceiverInfo2 !=
"") or (BANCS.FRMVALUE.senderToReceiverInfo3 != "") or
(BANCS.FRMVALUE.senderToReceiverInfo4
!= "") or (BANCS.FRMVALUE.senderToReceiverInfo5 != "") or
(BANCS.FRMVALUE.senderToReceiverInfo
6 != "")) then
    sv_a = sv_a + "|" + ":72:" + BANCS.FRMVALUE.senderToReceiverInfo1
    sv_a = sv_a + "|" + BANCS.FRMVALUE.senderToReceiverInfo2
    sv_a = sv_a + "|" + BANCS.FRMVALUE.senderToReceiverInfo3
    sv_a = sv_a + "|" + BANCS.FRMVALUE.senderToReceiverInfo4
    sv_a = sv_a + "|" + BANCS.FRMVALUE.senderToReceiverInfo5
    sv_a = sv_a + "|" + BANCS.FRMVALUE.senderToReceiverInfo6
else
    sv_a = sv_a + "|||||"
endif

BANCS.OUTPUT.swiftMessage = sv_a

exitscript
end-->

```

3.9 GET THE NAME OF THE CURRENT FIELD IN THE CURRENT FORM.

This function will give the name of the current field in the current form.

SYNTAX

```
sv_a = urhk_TBAF_GetCurFldName("")
```

No input variable required

FUNCTIONALITY

Whenever invoked, this user hook will give the name of the current field on which the cursor is lying in the current form in "formname.blockname.fieldname" format. This value will be available in B2KTEMP.TEMPSTD.TBAFRESULT field.

INPUT

Input string is null : ""

OUTPUT

Always returns 0 as return value

The current field name (field on which the cursor is lying) will be returned as output in the B2KTEMP.TEMPSTD.TBAFRESULT field in <formname>.<blockname>.<fieldname> format.

EXAMPLE

```
<--start
sv_a = urhk_TBAF_GetCurFldName("")
if (B2KTEMP.TEMPSTD.TBAFRESULT = "baff0009.datablk6.cust_id") then
  # some actions
endif
exitscript
end-->
```

3.10 DISPLAY A MESSAGE AT THE BOTTOM LINE IN A FORM.

This function will display a given message at the bottom line (the place where messages are generally shown in a form) in a form.

SYNTAX

```
sv_a = urhk_TBAF_DisMesg (Variable)
```

The variable can be a scratch pad variable (sv_b) or a string ("Value cannot be greater than 1000")

FUNCTIONALITY

This user hook will display a given message at the bottom line of a form when invoked. This can be used to specify any error messages that the user would want to display during some processing in a script.

INPUT

Input string has only one part

The message to be displayed.

Ex : – "Value cannot be greater than 1000"

OUTPUT

Always returns 0 as return value

The message will be displayed at the bottom line of the form once the control is returned back.

EXAMPLE

Let us assume that a Bank wants to display a message "Value cannot be greater than 1000" in case it is so during a script processing.

```
<--start
sv_a = BANCS.INPUT.tran_amount
if (sv_a > 1000) then
  sv_b = urhk_TBFAF_DispatchMsg(sv_e)
  exitscript
else
  # go ahead further
endif
exitscript
end-->
```

3.11 SPECIFY ADDITIONAL ACTIONS FOR A FUNCTION KEY AT ANY FIELD

This function will enable the user to specify a script name that he would like to be invoked at the pressing of a specified function key at a specified field on a form, in addition to the normal action for that key.

SYNTAX

sv_a = urhk_TBFAF_SetKeyScript(Variable)

The variable can be a scratch pad variable (sv_b) or a string ("baff0009.decision_blk.cust_option_code.key-nxtfld| CUMMadet.scr|1")

FUNCTIONALITY

This user hook will set the script name specified, as the script to be invoked when a particular function key is pressed on a particular field. Two scripts can be specified, one to be executed before processing the function key pressed and one to be executed after processing the function key. The step number for each of these scripts can also be specified.

Once set in this manner, whenever the specified function key is hit on the specified field of a form, the specified pre script gets executed (at specified step) followed by the normal processing for that function key followed by the specified post script (at specified step).

At least one of the pre or post scripts is mandatory, though one can specify both.

INPUT

Input string has five (5) parts separated by a '|'

<formname>.<blockname>.[<fieldname>.]<function-key-name>

Name of the script to be executed before key processing

Step number of above script to be executed

Name of the script to be executed after the key processing

Step number of the above script to be executed

The function-key-name must be a valid logical key name like key-f2 or key-nxtfld. Also the fieldname is not mandatory. If not specified, then it means for any field.

Following are examples of some valid input strings :

"baff0009.funcblk.key-f2||CUMMadet.scr|3"

This means that the script CUMMadet.scr at step 3 must be executed whenever key-f2 is pressed on form baff0009 at block funcblk. The script is a post script (4th part of input) and hence must be executed after the normal processing for the key-f2 happens.

"baff0009.decision_blk.cust_option_code.key-nxtfld|CUMMadet.scr|1"

This means that the script CUMMadet.scr at step 1 must be executed whenever key-nxtfld is pressed on the field cust_option_code of decision_blk in form baff0009. Note that in this example, the script is executed before the key-nxtfld logic is processed since the script was specified as a prescript (2nd part of input).

Formname and blockname is mandatory. Field name is optional (if not specified, it would mean for any field). Function Key must be specified. The step numbers are not mandatory parts. One of pre or post scripts must be specified.

Valid values for function keys are all valid keys specified in the crd file that begin with a 'key-'

OUTPUT

Always returns 0 as return value

Once the key script is set, the execution of the form will ensure that the appropriate scripts are run at appropriate times.

EXAMPLE

In the preload script of CUMM menu option one could incorporate this feature as shown in the example below :

baff0009PreLoad.scr :

```
<--start
# baff0009PreLoad.scr
# This sets up key events for passing control to scripts
# when the baff0009 form is loaded either thru the CUMM option or
# 'Customer explode'. This allows the "X" option in the option code to be
# processed by the CUMMadet.scr . It also sets certain fields as mandatory
sv_a = urhk_TBAF_SetKeyScript("baff0009.decision_blk.cust_option_code.key-
nxtfld|CUMMadet.scr|1")
sv_a = urhk_TBAF_SetKeyScript("baff0009.decision_blk.cust_option_code.key-
commit|||CUMMadet.scr|2")
sv_a = urhk_TBAF_SetKeyScript("baff0009.funcblk.key-f2|||CUMMadet.scr|3")
sv_a = urhk_TBAF_SetAttrib("baff0009.datablk1.cust_sex|M")
sv_a = urhk_TBAF_SetAttrib("baff0009.datablk1.cust_const|M")
sv_a = urhk_TBAF_SetAttrib("baff0009.datablk1.cust_introd_stat|M")
sv_a = urhk_TBAF_SetAttrib("baff0009.datablk2.cust_comu_addr1|M")
sv_a = urhk_TBAF_SetAttrib("baff0009.datablk2.cust_comu_city_code|M")
sv_a = urhk_TBAF_SetAttrib("baff0009.minor_datablk.minor_guard_name|M")
sv_a = urhk_TBAF_SetAttrib("baff0009.minor_datablk.minor_guard_addr1|M")
sv_a = urhk_TBAF_SetAttrib("baff0009.minor_datablk.minor_guard_city|M")
sv_a = urhk_TBAF_SetAttrib("baff0009.datablk.nre_nationality|M")
exitscript
end-->
```

4 WORKFLOW SCRIPTING FUNCTIONS

(Can be used only in the context of any WorkFlow script) (A WorkFlow script is any script that is tied to a menu option using the 'Customisation' feature of the Finacle menu. Refer to WorkFlow manual for more details)

4.1 EXECUTING A MENU OPTION

This function provides a facility to execute a menu option available in BANCS200 from a WorkFlow script.

SYNTAX

```
sv_a = urhk_WFS_CallMenuOption (Variable)

if(sv_a == 1) then
    exitscript
endif
```

The variable can be a scratch pad variable (sv_b) or a string ("CUMM|0|1")

FUNCTIONALITY

This function will execute the menu option specified in the input string. This will be useful while writing a script for workflow to chain different menu options to execute one after the other to complete the workflow operation.

INPUT

Input String contains three parts separated by a `|`

Menu option – As displayed in the Finacle menu screen

Current Step.

Next Step [Optional].

Example - "CUMM|0|1"

Menu option can be any FINACLE menu option other than a Workflow type of menu option.

Current Step is the step in which this statement is being executed. This is required so that even if the called menu option is in a different executable, the script's execution continues correctly.

'Next step' is the step that a workflow script should be restarted at, if the system aborts during the workflow after the currently called menu option commits its changes. It is required to be specified only for 'update' menu options, and typically should specify the next step in the script. In effect it is equal to calling the function **urhk_WFS_CheckPoint(Next Step)** immediately after calling the 'update' menu option.

OUTPUT

The return value will be ZERO (0) in case the menu option is in the same executable, after executing the menu option. The return will be 1 if the menu option is in another executable, before executing the menu option. In this case the script should exit, so that the system can switch exe and continue the execution of the script in the context of the new executable.

Example:

Please see example of **urhk_WFS_ShowParamAcptFrm** function below.

4.2 QUEING UP A NEW WORKFLOW ITEM FROM WITHIN A WORKFLOW SCRIPT

This function provides a facility to queue up a new workflow item, during the course of a Workflow script, which can then be separately processed by the designated user or workgroup.

SYNTAX

```
sv_a = urhk_WFS_CreateWFItem(Variable)
```

The variable can be a scratch pad variable (sv_b) or a string ("3|RAJKUMAR|10")

FUNCTIONALITY

Sometimes during the execution of a workflow item by a user, there can arise conditions which may require that certain other supporting business tasks be done asynchronously and by some different personnel. For example, consider the business task of opening an account. It requires one person to create the customer master and open the account, while a different person is required to verify it. Hence, in the account opening workflow script, it is possible to queue up a workflow item to a 'Customer service' group for verification, after creating the customer record, as well as to queue up a workflow item to the user's supervisor to verify the account opening, after opening the account.

INPUT

Input String contains three parts.

1. Step number The step in the script from where this workflow item will start to execute.
2. User Id [Optional].
3. User Work Group [Optional].

Both User Id and User Work Group cannot be NULL at least one should be given.

Example - "3|RAJKUMAR|10"

If the user id is not valid then system will give error.

The workflow description in TEMPWFS.WFSOUTPUT.WorkFlowDesc will be taken as the description of the Workflow (for display purposes) and TEMPWFS.WFSINPUT.WFItemPriority will be taken as the priority of the queued up item.

OUTPUT

Always returns 0 as return value.

EXAMPLE:

Please see example of **urhk_WFS_ShowParamAcptFrm function** below for the Account opening script, which finally queues up a workflow item for the back office personnel to complete the data entry. The script below is a sample logic for the back office processing which makes use of `urhk_WFS_CreateWFItem` function to queue up 'verification' workflow items.

Please note that when this function is used, the state of the repositories `TEMPWFS`, `INTMP` and `OUTEMP` is preserved as is and is made available (as `STDWFS`, `INTBAF` and `OUTTBAF` repositories) to the user who takes up this item for processing. So any data that needs to be passed from the calling script to the queued up script should be stored in one of these repositories.

BOACOPN.scr

```

                                <--start
                                # For all scripts that are called from another script using Start
(scriptname)
                                # command, we should reset the STDWFS.WFSINPUT.CallScriptName to
                                # the current script, since in case the process needs to
restarted(because of
                                # suspension or fatal error), this script should be called rather
than the
                                # original script, since the step information will pertain to this
script.

                                STDWFS.WFSINPUT.CallScriptName = "BOACOPN.scr"

# We will create GLBDATA in TEMPWFS repository rather than STDWFS #repository since
we intend to fork off new Work Flow items for verification. This is because TEMPWFS
is the repository saved
#along with a newly queued up item (using CreateWFItem function)

sv_d = CLASSEXISTS("TEMPWFS", "GLBDATA")
if (sv_d == 0) then
    CREATECLASS("TEMPWFS", "GLBDATA", 5)
else
    DELETECLASS("TEMPWFS", "GLBDATA")
    CREATECLASS("TEMPWFS", "GLBDATA", 5)
endif

sv_a = cint(STDWFS.WFSINPUT.NextStep)

if (sv_a == 0 ) then
    GOTO STEP0
endif
if (sv_a == 1 ) then
    GOTO STEP1
endif
if (sv_a == 2 ) then
    GOTO STEP2
endif
if (sv_a == 3 ) then
    GOTO STEP3
endif

exitscript

# We have the customer Id and account number values in #STDWFS.GLBDATA since the
front-office script/back-office script that #queued up execution of this script had
stored it in TEMPWFS.GLBDATA #before queuing this.

STEP0
# setup fields for Customer modification
sv_a = urhk_TBAF_SetValue("baff0009.funcblk.func_code|M")
sv_b = "baff0009.funcblk.cust_id|" + STDWFS.GLBDATA.cust_id
sv_a = urhk_TBAF_SetValue(sv_b)

# Take the user directly to the decision block
sv_a = urhk_TBAF_SetReplayKey("funcblk.key-f2")
# Make certain fields mandatory
sv_a = urhk_TBAF_SetAttrib("baff0009.datablk2.cust_comu_addr1|M")

# Call the Customer Maintenance form in M'odify mode
sv_b = urhk_WFS_CallMenuOption("CUMM|0|1")
if(sv_b == 1) then
    exitscript
endif
# If user has not committed the customer modification step suspend the
# workflow
if (STDWFS.WFSINPUT.StepCommitFlg == 'N') then
    STDWFS.WFSINPUT.WfsStatus = "S"
    exitscript
endif
# Now queue up the Customer modification request to Customer processing #work group
(workgroup 10 for example) after populating the customer id #in TEMPWFS repository
TEMPWFS.GLBDATA.cust_id = STDWFS.GLBDATA.cust_id
sv_a = urhk_WFS_CreateWFItem(2||10)

STEP1

#Set up fields for the account maintenance form
sv_a = urhk_TBAF_SetValue("baff0077.funcblk.func_code|M")
sv_b = "baff0077.funcblk.acct_num|" + STDWFS.GLBDATA.acct_num

```

```

sv_a = urhk_TBAF_SetValue(sv_b)
# Take the user directly to the data block
sv_a = urhk_TBAF_SetReplayKey("funcblk.key-f2")

#Call the account maintenance form in M'odify mode
sv_b = urhk_WFS_CallMenuOption("ACM|1|")
if(sv_b == 1) then
    exitscript
endif
# If user has not committed the account modification step suspend the
# workflow
if (STDWFS.WFSINPUT.StepCommitFlg == 'N') then
    STDWFS.WFSINPUT.WfsStatus = "S"
    exitscript
endif

# Now queue up the Account modification request to the user's supervisor #for
verification after populating the account nbr in TEMPWFS repository
TEMPWFS.GLBDATA.acct_num = STDWFS.GLBDATA.acct_num
TEMPWFS.WFSOUTPUT.WorkFlowDesc = "Account Modification Verification"
TEMPWFS.WFSINPUT.WFItemPriority = 5

sv_a = urhk_WFS_CreateWFItem(3||10)

STDWFS.WFSINPUT.WfsStatus = "D"
exitscript

STEP2

# setup fields for Customer verification
sv_a = urhk_TBAF_SetValue("baff0009.funcblk.func_code|V")
sv_b = "baff0009.funcblk.cust_id|" + STDWFS.GLBDATA.cust_id
sv_a = urhk_TBAF_SetValue(sv_b)

# Take the user directly to the decision block
sv_a = urhk_TBAF_SetReplayKey("funcblk.key-f2")

# Call the Customer Maintenance form in V'erify mode
sv_b = urhk_WFS_CallMenuOption("CUMM|2|")
if(sv_b == 1) then
    exitscript
endif
# If user has not committed the customer modification step suspend the
# workflow
if (STDWFS.WFSINPUT.StepCommitFlg == 'N') then
    STDWFS.WFSINPUT.WfsStatus = "S"
    exitscript
endif

STDWFS.WFSINPUT.WfsStatus = "D"
exitscript

STEP3

#Set up fields for the account verification
sv_a = urhk_TBAF_SetValue("baff0077.funcblk.func_code|V")
sv_b = "baff0077.funcblk.acct_num|" + STDWFS.GLBDATA.acct_num
sv_a = urhk_TBAF_SetValue(sv_b)
# Take the user directly to the data block
sv_a = urhk_TBAF_SetReplayKey("funcblk.key-f2")

#Call the account maintenance form in V'erify mode
sv_b = urhk_WFS_CallMenuOption("ACM|3|")
if(sv_b == 1) then
    exitscript
endif
# If user has not committed the account modification step suspend the
# workflow
if (STDWFS.WFSINPUT.StepCommitFlg == 'N') then
    STDWFS.WFSINPUT.WfsStatus = "S"
    exitscript
endif
STDWFS.WFSINPUT.WfsStatus = "D"
exitscript

end-->

```

4.3 FORWARD THE WORKFLOW (TRANSFER TO SOME OTHER USER).

This function provides the facility to queue the current workflow item (with its status preserved) to some other user / user work group.

SYNTAX

```
sv_a = urhk_WFS_TransferUser(Variable)
```

The variable can be a scratch pad variable (sv_b) or a string ("6|RAJKUMAR|10")

FUNCTIONALITY

Sometimes during the execution of a workflow item there can arise conditions which may require that a different user needs to perform part or rest of the workflow steps. For example, consider the business task of opening an account. For reasons of quick customer service, the bank may decide that the customer-facing personnel only enters the bare minimum details while opening the account and the rest of the details will be entered into the system from the account opening form in a back-office process. In this example a Workflow menu option for 'Rapid opening of an account' will be created, which chains the Customer opening, account opening and transaction posting options together. At the end of this workflow script, the workflow item which refers to the Customer Id and Account nbr can be queued, so that a back-office personnel can process that item when the necessary paperwork is received by him. This will also ensure that this process gets done even if the paperwork does not reach him, since the queued entry will be pending in the system.

Please note that when this function is used, the state of the repositories STDWFS, INTBAF and OUTTBAF is preserved as is and is made available to the user who takes up this item and continues the processing. So any data that needs to be passed from the calling script to the queued up script should be stored in one of these repositories.

INPUT

Input String contains three parts.

1. Next Step.
2. User Id [Optional].
3. User Work Group [Optional].

Example - "6|RAJKUMAR|10"

Next step is the step from which execution should start when next user starts continuing on this workflow.

User Id who can continue on this workflow.

User work group name. All the users belong to this work group can continue on this workflow.

If user / user work group is not specified then workflow will be transferred to the current users authorised user id in Finacle

OUTPUT

Always returns 0 as return value.

EXAMPLE:

Please see example of **urhk_WFS_ShowParamAcptFrm function** below for the Account opening script which finally transfers the workflow item to a back office personnel to complete the data entry.

4.4 CALLING INPUT ACCEPTANCE FORM

This function will invoke a standard user acceptance form, which can be called in workflow script to accept additional data during the workflow. See also **urhk_B2K_ShowParamAcptFrm("bafi2028")**.

SYNTAX

```
sv_a = urhk_WFS_ShowParamAcptFrm("").
```

FUNCTIONALITY

This function will bring a standard form with all fields defined in WFSINPARAMVAL class to accept additional data that may be required during workflow.

A maximum of 34 fields can be accepted in one call to this function.

INPUT

NULL

This function requires certain inputs for bringing up the form, displaying the literals and accepting data. The inputs are to be given by way of a repository fields in WFSINPARAMVAL and WFSINPARAMLEN class of STDWFS repository, for each field that needs to be displayed/accepted. For each field the following is required to be given

1) The name of the field - assigned by referring to this name while populating the literal and length of the field.

2) The literal for the field – Maximum length for literal is 20 characters

The length of the field - Maximum length for value is 70 characters

```
Ex: STDWFS.WFSINPARAMVAL.ledgerNo ="Ledger No."  
    STDWFS.WFSINPARAMLEN.ledgerNo ="5"
```

If a default value needs to be specified for any field then that value needs to be assigned to the corresponding output field. For ex. To assign a default value of 'XXX' for the ledgerNo, we should do the following before calling this function :
STDWFS.WFSOUTPARAM.ledgerNo = "XXX"
A well known field STDWFS.WFSINPARAMVAL.ErrorMesg can be used to force the form to display an Error message.
A well known field STDWFS.WFSINPARAMVAL.FormTitle can be used to display the title in the form.

OUTPUT

Always returns 0 as return value.

The values entered on the screen will be available in WFSOUTPARAM class under the corresponding name in WFSOUTPARAM class, after the user presses <ACCEPT> or <COMMIT> key in the form.

EXAMPLE:

In this example a sample script has been written for account opening workflow which will invoke standard from to accept initial data and then call CUMM and OAAC one after another. At the end of the script the workflow item is transferred to a back-office user for completing the dataentry.

```

<--start

sv_d = CLASSEXISTS("STDWFS", "GLBDATA")
if (sv_d == 0) then
    CREATECLASS("STDWFS", "GLBDATA", 5)
else
    DELETECLASS("STDWFS", "GLBDATA")
    CREATECLASS("STDWFS", "GLBDATA", 5)
endif

sv_a = cint(STDWFS.WFSINPUT.NextStep)

if (sv_a == 0 ) then
    GOTO STEP0
Endif
if (sv_a == 1 ) then
    GOTO STEP1
endif

if (sv_a == 3 ) then
    GOTO STEP3
endif

if (sv_a == 4 ) then
    GOTO STEP4
Endif
if (sv_a == 5 ) then
    GOTO STEP5
endif
exitscript

STEP0
# Define the fields to be accepted
STDWFS.WFSINPARAMVAL.existCustId="Exist Cust Id"
STDWFS.WFSINPARAMVAL.custName="Customer Name"
STDWFS.WFSINPARAMVAL.custShortName="Cust Short Name"
STDWFS.WFSINPARAMVAL.glSubHead="GL Sub Head"
STDWFS.WFSINPARAMVAL.schemeCode="Scheme Code"
STDWFS.WFSINPARAMVAL.currency="Account currency"

# Define the length of the fields to be accepted
STDWFS.WFSINPARAMLEN.existCustId="9"
STDWFS.WFSINPARAMLEN.custName="30"
STDWFS.WFSINPARAMLEN.custShortName="10"
STDWFS.WFSINPARAMLEN.glSubHead="5"
STDWFS.WFSINPARAMLEN.currency="5"
STDWFS.WFSINPARAMLEN.schemeCode="5"

# Define the default value of the fields to be accepted
STDWFS.WFSOUTPARAM.schemeCode="XXXX"

sv_a = urhk_WFS_ShowParamAcptFrm("")

if(STDWFS.WFSOUTPARAM.existCustId!="") then
    # set the Customer Id for the account opening form
    sv_b = "baf3013.funcblk.cust_id|" + STDWFS.WFSOUTPARAM.existCustId
    sv_a = urhk_TBAF_SetValue(sv_b)
    GOTO STEP1
endif

# setup fields for Customer creation form
sv_c = urhk_TBAF_SetValue(baff0009.datablk6.cust_id|OUTTBAF)
sv_b = "baff0009.datablk1.cust_name|" + STDWFS.WFSOUTPARAM.custName
sv_c = urhk_TBAF_SetValue(sv_b)
sv_b = "baff0009.datablk1.cust_short_name |" + STDWFS.WFSOUTPARAM.custShortName
sv_c = urhk_TBAF_SetValue(sv_b)
sv_a = urhk_TBAF_SetValue("baff0009.funcblk.func_code|A")

# Store remaining data from acceptance form in global class for later use
STDWFS.GLBDATA.glSubHead = STDWFS.WFSOUTPARAM.glSubHead
STDWFS.GLBDATA.schemeCode = STDWFS.WFSOUTPARAM.schemeCode
STDWFS.GLBDATA.currency = STDWFS.WFSOUTPARAM.currency

# Take the user directly to the first mandatory field in the data block
sv_a = urhk_TBAF_SetReplayKey("funcblk.key-f2")
sv_a = urhk_TBAF_SetReplayKey("datablk1.key-f2")

```

```

# Call the Customer Maintenance form in A'dd mode
sv_b = urhk_WFS_CallMenuOption("CUMM|0|3")
if(sv_b == 1) then
    exitscript
endif
# If user has not committed the customer creation step end the workflow
if (STDWFS.WFSINPUT.StepCommitFlg == 'N') then
    STDWFS.WFSINPUT.WfsStatus = "D"
    exitscript
endif
sv_b = urhk_TBAF_GetValue(baff0009.datablk6.cust_id|OUTTBAF)
STDWFS.GLBDATA.cust_id = B2KTEMP.TEMPSTD.TBAFRESULT

# restart step if system fails after Customer creation commit
STEP3
#We will get it auto verified, since the back-office person will pull it up and
#check the data anyway.
STDWFS.WFSINPUT.IgnoreSameUserVerifyFlg = "Y"
sv_a = urhk_TBAF_SetValue("baff0009.funcblk.func_code|V")
sv_b = "baff0009.funcblk.cust_id|" + STDWFS.GLBDATA.cust_id
sv_a = urhk_TBAF_SetValue(sv_b)

# Take the user directly to the decision block and commit
sv_a = urhk_TBAF_SetReplayKey("funcblk.key-f2")
sv_a = urhk_TBAF_SetReplayKey("decision_blk.key-commit")

# Call the Customer Maintenance form in V'erify mode
sv_b = urhk_WFS_CallMenuOption("CUMM|3|4")
if(sv_b == 1) then
    exitscript
endif

# restart step if system fails after Customer verification commit
STEP4
# If customer creation verification step is not committed display error and end the
workflow
if (STDWFS.WFSINPUT.StepCommitFlg == 'N') then
    # display error here
    STDWFS.WFSINPUT.WfsStatus = "D"
    exitscript
endif

# set the Customer Id for the account opening form
sv_b = "bafe3013.funcblk.cust_id|" + STDWFS.GLBDATA.cust_id
sv_a = urhk_TBAF_SetValue(sv_b)

STEP1
# setup fields for Account opening form

sv_a = urhk_TBAF_SetValue("bafe3013.funcblk.func_code|0")
sv_b = "bafe3013.funcblk.gl_sub_head_code|" + STDWFS.GLBDATA.glSubHead
sv_a = urhk_TBAF_SetValue(sv_b)
sv_b = "bafe3013.funcblk.schm_code|" + STDWFS.GLBDATA.schemeCode
sv_a = urhk_TBAF_SetValue(sv_b)
sv_b = "bafe3013.funcblk.crncy_code|" + STDWFS.GLBDATA.currency
sv_a = urhk_TBAF_SetValue(sv_b)

# Take the user directly to the the data block
sv_a = urhk_TBAF_SetReplayKey("funcblk.key-f2")

# Get the account id of the account for use in the backoffice processing
# This will really depend on the scheme that the account is being opened, but
# here we will assume SB scheme - hence sbfe3201
sv_c = urhk_TBAF_SetValue(sbfe3201.lastblk.acct_num|OUTTBAF)

# Call the account opening function in O'pen mode
sv_b = urhk_WFS_CallMenuOption("OAC|1|")
if(sv_b == 1) then
    exitscript
endif
# Store account number in global class for later use
sv_b = urhk_TBAF_GetValue(sbfe3201.lastblk.acct_num|OUTTBAF)
STDWFS.GLBDATA.acct_num = B2KTEMP.TEMPSTD.TBAFRESULT

# Queue up the BO workflow item
sv_a = urhk_WFS_TransferUser(5|BOUser)

STDWFS.WFSINPUT.WfsStatus = "D"
exitscript

```

```
# Now we code the workflow for the backoffice user who will fill in the details for
the customer and account

STEP5
# Though we can code the back-office workflow in this script itself, we will #code
the Back-office script in another script file. The script file should exist #in
[TBA_PROD_ROOT]/cust/scripts directory. We should
# reset the NextStep to 0, since that is the first step to be executed in that
# script and not 5
STDWFS.WFSINPUT.NextStep = "0"
Start ([TBA_PROD_ROOT]/cust/scripts, BOACOPN.scr)
exitscript
end-->
```

See example under `urhk_WFS_CreateWFItem` section for a description of the `BOACOPN.scr`

4.5 SET CHECK POINT TO RESTART THE SCRIPT IN CASE OF FATAL / RUN TIME ERRORS.

This function will provide the facility save the repositories so that in case of any fatal error / run time error occurred in the subsequent menu option calls system will start executing the script from the step specified in the function call while continuing the workflow.

SYNTAX

`sv_a = urhk_WFS_CheckPoint (Variable)`

The variable can be a scratch pad variable (`sv_b`) or a string ("2")

FUNCTIONALITY

This function allows the script to preserve the status at a given point in the execution, so that even if the system fails, the script can be restarted at that point and data entered earlier is not lost. Please note that automatic checkpointing happens whenever a Finacle transaction (like account opening, or Customer modification) is committed successfully. This function will be required only when data has been entered and status has been saved in repository variables and we wish to provide restartability from that point. Specifically if we use **`urhk_WFS_ShowParamAcptFrm`** to accept data, then immediately after that call we should call this function.

INPUT

Input String contains the Next Step.

Example - "2"

Next step is the step from which execution should start when next user starts continuing on this workflow.

OUTPUT

Always returns 0 as return value.

EXAMPLE:

In this example we will show a workflow for opening an account by invoking CUMM and OAAC. This workflow contains three steps involved.

Call urhk_WFS_ShowParamAcptFrm to accept account opening data.

Call CUMM menu option.

Call OAAC menu option.

```
<--start

sv_a = cint(STDWFS.WFSINPUT.NextStep)

if (sv_a == 0 ) then
    GOTO STEP0
endif
if (sv_a == 1 ) then
    GOTO STEP1
endif
if (sv_a == 2 ) then
    GOTO STEP2
endif

exitscript

STEP0
STDWFS.WFSINPARAMVAL.existCustId="Exist Cust Id"
STDWFS.WFSINPARAMVAL.existAcctNum="Exist A/c Num."
STDWFS.WFSINPARAMVAL.glSubHead="GL Sub Head"
STDWFS.WFSINPARAMVAL.schemeCode="Scheme Code"
STDWFS.WFSINPARAMVAL.depositAmount="Deposit Amount"
STDWFS.WFSINPARAMVAL.ledgerNo="Ledger Number"
STDWFS.WFSINPARAMVAL.repaymentCond="Repaymnt Condition"
STDWFS.WFSINPARAMVAL.depPeriodMonths="Deposit Prd In Mon"

STDWFS.WFSINPARAMLEN.existCustId="9"
STDWFS.WFSINPARAMLEN.existAcctNum="16"
STDWFS.WFSINPARAMLEN.glSubHead="5"
STDWFS.WFSINPARAMLEN.schemeCode="5"
STDWFS.WFSINPARAMLEN.repaymentCond="3"

STDWFS.WFSOUTPARAM.ledgerNo="1"
STDWFS.WFSOUTPARAM.repaymentCond="ALL"

sv_a = urhk_WFS_ShowParamAcptFrm("")

# If fatal error occurred in CUMM system will start form STEP1 while
# continuing the workflow.

sv_a = urhk_WFS_CheckPoint("1")

STEP1
sv_b = urhk_WFS_CallMenuOption("CUMM|0|2")
if(sv_b == 1) then
    exitscript
endif
# sv_a = urhk_WFS_CheckPoint("2")
#The above statement is not required since we have achieved the same by #specifying
the next step parameter while calling CUMM

STEP2
Sv_b = urhk_WFS_CallMenuOption("OAAC|1|")
if(sv_b == 1) then
    exitscript
endif

EXITSCRIPT
end-->
```

4.6 SET WORK GROUP OF A USER FOR USING WORK FLOW

The user hook urhk_WFS_SetWorkGroup(variable) works as follows :

This user hook sets the workflow workgroup for the user as specified in the variable .

SYNTAX

The Syntax for calling the function is

sv_r = urhk_WFS_SetWorkGroup (variable)

The variable can be a scratch pad variable (sv_a) or a string("CAB|1098").

FUNCTIONALITY

This user hook is provided to set the workflow workgroup for the user at time of log in.

INPUT

Work group

Sol id

OUTPUT

NIL.

EXAMPLE

```
<--start
sv_a=Work Group
sv_b=Sol Id
sv_c=sv_a + "|" + sv_b
sv_r = urhk_WFS_SetWorkGroup (sv_c)
end-->
```

4.7 WFSRE_EXECUTE

SYNTAX

sv_a = urhk_WFSRE_Execute(Variable)

FUNCTIONALITY

This user hook sends a script for execution to the WFSRE server running in the DC logged on and obtains the result.

INPUT

Input consists of 3 parts separated by '|'

DC Id – Under which a logon has already been done

Script file path – The path where the script to be executed is located

QueueJobOnError – 'Y' or 'N' flag. 'Y' indicates that if the script is not executed successfully then the job is to be queued up at the datacenter for remedial action. 'N' indicates that it should not be queued up.

OUTPUT

Return value is 0 if execution is successful, otherwise it returns 1. Output from the server, including server error codes, are populated in the repositories.

EXAMPLE

Check the example usrexcsample2.pc in sample/scripts directory.

4.8 WFSRE_LOGON

The repository WFSRE, is used as the interface between the remote application and the remote workflow executor. The remote application should create the WFSRE repository and as many classes and fields as the application requires under this repository. All classes/fields created and populated by the remote application at the time of reference to the *WFSRE_Execute (Script)* function are made available to the script when it is being executed at the data center. Similarly, all classes and fields that exist under the WFSRE repository at the end of execution, at the datacenter, are made available to the remote application after the *WFSRE_Execute (Script)* function has been completed.

The following fields have a special significance and must exist in the WFSRE repository before using the functions. All other fields are user definable and will be transported from the remote application to the datacenter before execution and transferred back from the datacenter to the remote application after execution at the datacenter.

Rep.Class.Field Name	Remarks
WFSRE.STDIN. Solid	If it is specified, then this is the Sol under which the script will be executed
WFSRE.STDIN.AccountNum	If Sol id is not specified, then the owner solid of this account will be the sol under which the script will be executed
WFSRE.STDOUT.ErrorCode	Reference only. Contains an error code if an error occurs during execution of the script at the datacenter. If no error occurs this field will contain `.`.
WFSRE.STDOUT.ErrorDesc	Reference only. Contains the description of the error that occurred during execution of the script at the datacenter.

SYNTAX

```
sv_a = urhk_WFSRE_Logon(Variable)
```

FUNCTIONALITY

This user hook logs in to WFSRE server of the specified DC and keeps the connection open till the logout function is called. This user hook can be called many times in a single session. Already available connections to DCs are reused.

INPUT

Input consists of 3 parts separated by '|'

DC Id – Should correspond to the entry set up in wfsreclntsetup

User Id – Should be a valid user in the Data Center pointed to by DC id

Password – Password of the above user

OUTPUT

Return value is 0, if login was successful or a previous login already exists, else it is 1.

EXAMPLE

Check the eample usrexecsample2.pc in sample/scripts directory.

4.9 WFSRE_LOGOUT**SYNTAX**

```
sv_a = urhk_WFSRE_Logout(Variable)
```

FUNCTIONALITY

This user hook logs out the user from a DC, if the user is already logged on.

INPUT

Variable should be the DC id from where to log out.

OUTPUT

Return value is 0 if an existing login is found and the user was logged out successfully, otherwise it is 1.

EXAMPLE

Check the eample usrexecsample2.pc in sample/scripts directory.

5 CLEARING SCRIPTING FUNCTIONS

(Can be used in the context of any scripting event in the Clearing module)

5.1 GET DETAILS FROM CTC TABLE FOR A TRANSACTION CODE

SYNTAX

The Syntax for calling the function is

```
sv_r = urhk_GetCTCDetailsFromTranCode(Variable)
```

The variable can be a scratch pad variable (sv_b) or a string ("10")

FUNCTIONALITY

This function gets all the attributes of a 'Clearing transaction code' (as printed on the instrument) that is maintained in the CTC table of Finacle. Clearing transaction codes are used in India to distinguish between various types of clearing

INPUT

The input to the function is Clearing transaction code

Example - "10"

OUTPUT

Returns 1 as return value if the transaction code does not exists in the CTC table, else returns 0.

All the attributes of the transaction code stored in the CTC table is returned in the following fields of BANCS repository and OUTPARAM class

Field Name in CTC table	Field name in BANCS repository
1. sys_arrived_acct_flg	BANCS.OUTPARAM.SysArrivedAcctFlg

"Y" or "N" Automatically arrive at the account number using cheque issue records.

2. dr_bacid	BANCS.OUTPARAM.DrBacid
-------------	------------------------

If system arrived flag is "N", then account place holder that is to be used.

3. instrmnt_type	BANCS.OUTPARAM.InstrmntType
------------------	-----------------------------

Default instrument type for the transaction code.

media_load_flg	BANCS.OUTPARAM.MediaLoadFlg
----------------	-----------------------------

Can this record be loaded electronically.

applicability_ind BANCS.OUTPARAM.ApplicabilityInd

Inward clearing, Outward Clearing or Both

consolidation_flg BANCS.OUTPARAM.ConsolidationFlg

default_upld_acct_prefix BANCS.OUTPARAM.

DefaultUpldAcctPrefix

Not being used now.

search_schm_type BANCS.OUTPARAM.SearchSchmType

System will look at these scheme types for faster access based on cheque book issue *accounts*.

5.2 GET INWARD CLEARING ZONE HEADER DETAILS

SYNTAX

The Syntax for calling the function is

```
sv_r = urhk_GetZoneHeaderDetails(Variable)
```

The variable can be a scratch pad variable (sv_b) or a string ("AM|12-12-1999 00:00:00|123456")

FUNCTIONALITY

This function returns the attributes of the Inward clearing zone from the IZH table.

INPUT

Input String contains three parts separated by a '|'

1. Zone Code
2. Zone Date (DD-MM-YYYY 00:00:00)
3. SolId

Example - "AM|12-12-1999 00:00:00|123456"

OUTPUT

Return value will be 1 if no zone is found, else it will be 0.

The attributes of the zone stored in the IZH table is returned in the following fields of BANCS repository and OUTPARAM class

Field Name in IZH table	Field name in BANCS repository
-------------------------	--------------------------------

value_date	BANCS.OUTPARAM.ValueDate (DD-MM-YYYY 00:00:00)
------------	--

Value date applicable for the zone in question

tran_date	BANCS.OUTPARAM.TranDate (DD-MM-YYYY 00:00:00)
-----------	---

Transaction date applicable for the zone in question. Will be different only for back dated zones.

micr_clg_flg BANCS.OUTPUTPARAM.MICRClgFlg

Is MICR clearing allowed for the zone? Y or N

media_inp_alwd_flg BANCS.OUTPUTPARAM.MediaInpAlwdFlg

Is input allowed via media upload.

dflt_carving_flg BANCS.OUTPUTPARAM.DefaultCarveFlg

Carving permitted by default? Y or N

tran_code_mandatory BANCS.OUTPUTPARAM.

TranCodeMandatory

Is Transaction code mandatory? Y or N

inst_type_mandatory BANCS.OUTPUTPARAM.

InstTypeMandatory

Is instrument type mandatory? Y or N

5.3 GET OUTWARD CLEARING ZONE HEADER DETAILS

SYNTAX

The Syntax for calling the function is

sv_r = urhk_getOutZoneHeaderDetails (Variable)

The variable can be a scratch pad variable (sv_b) or a string ("SOL100|12-12-1999 00:00:00|ZONE10")

FUNCTIONALITY

This function returns the attributes of the Outward clearing zone from the OZH table.

INPUT

Input String contains three parts separated by a '|'

- Sol Id to which the zone belongs.
- Zone Date (DD-MM-YYYY 00:00:00)
- Zone Code

Example - "SOL100|12-12-1999 00:00:00|ZONE10"

This zone should be a valid zone. If the zone does not exist, the user will get a fatal error.

OUTPUT

Return value will be 0 when zone is found. If zone is not found, then user will get a fatal error.

The attributes of the zone stored in the OZH table is returned in the following fields of BANCS repository and OUTPARAM class

Field Name in OZH table	Field name in BANCS repository
sol_id	BANCS.OUTPARAM. ClgSolId
clg_zone_date	BANCS.OUTPARAM. ClgZoneDate
clg_zone_code	BANCS.OUTPARAM. ClgZoneCode
cr_acid	BANCS.OUTPARAM. ParkingCrAcid
dr_acid	BANCS.OUTPARAM. ClgDrAcid
home_clg_flg	BANCS.OUTPARAM. HomeClgFlg
shadow_bal_flg	BANCS.OUTPARAM. ShadowBalFlg
micr_clg_flg	BANCS.OUTPARAM. MicrClgFlg
encode_flg	BANCS.OUTPARAM. EncodeFlg

Field Name in OZH table	Field name in BANCS repository
zone_micr_chrg_code	BANCS.OUTPARAM. ZoneMicrChrgCode
clg_sec_dp_code	BANCS.OUTPARAM. ClgSecDpCode
tran_date1	BANCS.OUTPARAM. OZHTranDate1
tran_date2	BANCS.OUTPARAM. OZHTranDate2
zone_stat	BANCS.OUTPARAM. ZoneStat
value_date1	BANCS.OUTPARAM. OZHValueDate1
value_date2	BANCS.OUTPARAM. OZHValueDate2
bar_inst_amt	BANCS.OUTPARAM. BarInstAmt
bar_inst_num	BANCS.OUTPARAM. BarInstNum
bar_num	BANCS.OUTPARAM. BarNum
bar_date	BANCS.OUTPARAM. BarDate
frgn_crncy_clg_flg	BANCS.OUTPARAM. FrgnCrncyClgFlg
sttlmnt_in_inst_crncy_flg	BANCS.OUTPARAM. SttlmntInInstCrncyFlg
sttlmnt_crncy_code	BANCS.OUTPARAM. SttlmntCrncyCode
zone_crncy_code	BANCS.OUTPARAM. ZoneCrncyCode
zone_to_sttlmnt_crncy_rate	BANCS.OUTPARAM. ZoneToSttlmntCrncyRate
rate_code	BANCS.OUTPARAM. ZoneToSttlmntCrncyRateCode
event_id	BANCS.OUTPARAM. EventId
event_type	BANCS.OUTPARAM. EventType
def_rate_code1	BANCS.OUTPARAM. DefRateCode1
def_rate_code2	BANCS.OUTPARAM. DefRateCode2
clg_sec_bank_code	BANCS.OUTPARAM. ClgSecBankCode
inst_or_acct_ptran_cre_flg	BANCS.OUTPARAM. InstOrAcctPtranCreFlg
set_rel_reg_flg	BANCS.OUTPARAM. set_rel_reg_flg
set_rel_dr_acid	BANCS.OUTPARAM. SetRelDrAcid
zone_latency	BANCS.OUTPARAM. ZoneLatency
out_rej_event_id	BANCS.OUTPARAM. OutRejEventId
cr_acid_for_out_rej	BANCS.OUTPARAM. CrAcidForOutRej
control_sol_id	BANCS.OUTPARAM. ControlSolId

All dates will be in DD-MM-YYYY 00:00:00 format.

5.4 GET OUTWARD CLEARING INSTRUMENT DETAILS

SYNTAX

The Syntax for calling the function is

```
sv_r = urhk_getOutClgInstDetails (Variable)
```

The variable can be a scratch pad variable (sv_b) or a string ("SOL100|12-12-1999 00:00:00|ZONE10| 1| 10")

FUNCTIONALITY

This function returns the details of the Outward clearing instrument from the OCI table.

INPUT

Input String contains five parts separated by a '|'

- 1 Sol Id to which the zone belongs.
- 2 Zone Date (DD-MM-YYYY 00:00:00)
- 3 Zone Code
- 4 Set Number
- 5 Instrument Srl Num.

Example - "SOL100|12-12-1999 00:00:00|ZONE10| 1| 10"

This instrument should exist in the OCI table. If the instrument does not exist, the user will get a fatal error.

OUTPUT

Return value will be 0 when Instrument is found. If instrument is not found, then user will get a fatal error.

The details of the instrument stored in the OCI table is returned in the following fields of BANCS repository and OUTPARAM class

Field Name in OCI table	Field name in BANCS repository
sol_id	BANCS.OUTPARAM. ClgSolId
clg_zone_date	BANCS.OUTPARAM. ClgZoneDate
clg_zone_code	BANCS.OUTPARAM. ClgZoneCode
set_num	BANCS.OUTPARAM. SetNum
instrmnt_srl_num	BANCS.OUTPARAM. InstrmntSrlNum
shd_bal_code	BANCS.OUTPARAM. ShdBalCode
tran_code	BANCS.OUTPARAM. TranCode

Field Name in OCI table	Field name in BANCS repository
instrmnt_id	BANCS.OUTPUTPARAM. InstrmntId
instrmnt_amt	BANCS.OUTPUTPARAM. InstrmntAmt
ncode_flg	BANCS.OUTPUTPARAM. NcodeFlg
rep_code	BANCS.OUTPUTPARAM. RepCode
bank_code	BANCS.OUTPUTPARAM. BankCode
br_code	BANCS.OUTPUTPARAM. BrCode
sort_code	BANCS.OUTPUTPARAM. SortCode
encode_seq_num	BANCS.OUTPUTPARAM. EncodeSeqNum
status_flg	BANCS.OUTPUTPARAM. StatusFlg
inw_rej_zone_cod	BANCS.OUTPUTPARAM. InwRejZoneCode
inw_rej_zone_date	BANCS.OUTPUTPARAM. InwRejZoneDate
inw_rej_inst_num	BANCS.OUTPUTPARAM. InwRejInstNum
inw_rej_ctrl_key_num	BANCS.OUTPUTPARAM. InwRejCtrlKeyNum
acct_sol_id	BANCS.OUTPUTPARAM.AcctSolId
value_date	BANCS.OUTPUTPARAM.InstValueDate
rej_type	BANCS.OUTPUTPARAM.RejType
rej_code_1	BANCS.OUTPUTPARAM.RejCode_1
rej_code_2	BANCS.OUTPUTPARAM.RejCode_2
rej_code_3	BANCS.OUTPUTPARAM.RejCode_3
rej_code_4	BANCS.OUTPUTPARAM.RejCode_4
rej_code_5	BANCS.OUTPUTPARAM.RejCode_5
rej_date	BANCS.OUTPUTPARAM.RejDate
inw_rej_zone_srl_num	BANCS.OUTPUTPARAM.InwRejZoneSrlNum
inw_rej_sol_id	BANCS.OUTPUTPARAM.InwRejSolId

All dates will be in DD-MM-YYYY 00:00:00 format.

5.5 GET OUTWARD CLEARING PART TRAN DETAILS

SYNTAX

The Syntax for calling the function is

```
sv_r = urhk_getOutClgPtranDetails (Variable)
```

The variable can be a scratch pad variable (sv_b) or a string ("SOL100|12-12-1999 00:00:00|ZONE10| 1| 10")

FUNCTIONALITY

This function returns the details of the Outward clearing part tran from the OCP table.

INPUT

Input String contains five parts separated by a '|'

- 1 Sol Id to which the zone belongs.
- 2 Zone Date (DD-MM-YYYY 00:00:00)
- 3 Zone Code
- 4 Set Number
- 5 Part Tran Srl Num.

Example - "SOL100|12-12-1999 00:00:00|ZONE10| 1| 10"

This part tran should exist in the OCP table. If the part tran does not exist, the user will get a fatal error.

OUTPUT

Return value will be 0 when Part Tran is found. If Part Tran is not found, then user will get a fatal error.

The details of the Part tran stored in the OCP table is returned in the following fields of BANCS repository and OUTPARAM class

Field Name in OCP table	Field name in BANCS repository
ClgSolId	BANCS.OUTPARAM.sol_id
ClgZoneDate	BANCS.OUTPARAM.clg_zone_date
ClgZoneCode	BANCS.OUTPARAM.clg_zone_code
SetNum	BANCS.OUTPARAM.set_num

PtranSrlNum	BANCS.OUTPARAM.part_tran_srl_num
PtranAcid	BANCS.OUTPARAM.Acid
TranAmt	BANCS.OUTPARAM.tran_amt
PartTranType	BANCS.OUTPARAM.part_tran_type
PrntAdvFlg	BANCS.OUTPARAM.prnt_adv_flg
PendAmt	BANCS.OUTPARAM.pend_amt
Partcls	BANCS.OUTPARAM.partcls
TranRptCode	BANCS.OUTPARAM.tran_rpt_code
TranRmks	BANCS.OUTPARAM.tran_rmks
ShdBalRemAmt	BANCS.OUTPARAM.shd_bal_rem_amt
PtranStatusFlg	BANCS.OUTPARAM.status_flg
NavigationFlg	BANCS.OUTPARAM.navigation_flg
AcctCrncyCode	BANCS.OUTPARAM.acct_crncy_code
PartTranAmtInInstCrncy	BANCS.OUTPARAM.part_tran_amt_in_inst_crncy
SttlInstToHmCrncyRate	BANCS.OUTPARAM.sttl_inst_to_hm_crncy_rate
HmToAcctCrncyRate	BANCS.OUTPARAM.hm_to_acct_crncy_rate
SttlInstToAccCrncyRate	BANCS.OUTPARAM.sttl_inst_to_acc_crncy_rate
SttlToHmRateCode	BANCS.OUTPARAM.sttl_to_hm_rate_code
HmToAcctRateCode	BANCS.OUTPARAM.hm_to_acct_rate_code
SttlToAcctRateCode	BANCS.OUTPARAM.sttl_to_acct_rate_code
AcctSolId	BANCS.OUTPARAM.acct_sol_id
PendAmtInInstCrncy	BANCS.OUTPARAM.pend_amt_in_inst_crncy
TdRefNum	BANCS.OUTPARAM.td_ref_num
SetRelRegStat	BANCS.OUTPARAM.set_rel_reg_stat

All dates will be in DD-MM-YYYY 00:00:00 format.

6 MTT'S SCRIPTING FUNCTIONS

These hooks can be used only in the context of any MTT script. An MTT script is any script that is executed in the context of an MTT event to create and post financial transactions.

6.1 GET THE INPUT DETAILS OF THE MTT EVENT

SYNTAX

```
sv_a = urhk_MTTSPopulateInputDetails ("" )
```

FUNCTIONALITY

This function gets all the details put out by the MTT event in repository variables. The details depend on the current event that is in process. This is equivalent to defining the transaction header details in some respects.

INPUT

There is no input to this function.

OUTPUT

The return value will be ZERO (0) in case the processing is successful. If not, it will return 1.

The actual fields that are given out by this user hook depends on the event and as such is documented in Scripting Events Document for each MTT event.

EXAMPLE:

Check any MTT enabled script like BillTran.sscr in sample/scripts directory.

6.2 DEFINING A FINANCIAL TRANSACTION

SYNTAX

```
sv_a = urhk_MTTSDefineTransaction (Variable)
```

The variable can be a scratch pad variable (sv_b) or a string ("XYZ")

FUNCTIONALITY

This function allows the script to initiate a financial transaction by defining some of its attributes. A name is used while the transaction is being defined so that multiple transactions can be defined in the same script.

INPUT

Input String contains the identification of the transaction. This identifier should be referred to while defining the part-transaction to link the part-transaction to the transaction.

Example - "XYZ"

All other inputs to this function is passed via repository variables in MTT.TranDetails class. The fields are:

TranDate	The date in terms of number of days past BOD date
TranType	The transaction type – valid values are 'Cash, Clearing and Transfer
TranSubType	A valid sub-type. Same validation as in TM will be done for tran type, sub-type combination
Remarks	String of max size 50 characters
IgnoreExcp	Flag to indicate whether the system can override exceptions during posting or not
Consolidate	Indicates that some part-transactions under this transaction will be consolidated into one part-transaction ("Y"/"N")
ReversalFlag	Flag to indicate whether the system should automatically put a future dated transaction (on reversal date) to reverse the current transaction.
ReversalDate	The date in terms of number of days past BOD date.

OUTPUT

The return value will be ZERO (0) in case the transaction definition is accepted. If not, it will return 1.

EXAMPLE:

Same as above

6.3 DEFINING A FINANCIAL PART-TRANSACTION

SYNTAX

```
sv_a = urhk_ MTTS_DefinePartTran (Variable)
```

The variable can be a scratch pad variable (sv_b) or a string ("XYZ")

FUNCTIONALITY

This function allows the script to initiate a financial transaction by defining some of its attributes. The attributes are first made available in the MTT.PartTranDetails class. A call is made to define the part transaction after that. The class variables are identified in the table below.

INPUT

Input String contains the identification of the part-transaction. This is used to consolidate multiple part-transactions into one part-transaction for all part-transactions whose consolidate flag is set.

Example - "XYZ"

All other inputs to this function is passed via repository variables in MTT. PartTranDetails class. The fields are:

Account	The account nbr (foracid) of the account to be debited/credited
RefAmount	The amount to be debited/credited. Positive amount indicates credit, while a negative amount indicates a debit.
ValueDate	The value date of the part-transaction in terms of number of days from BOD date (+ or -)
Refnum	Corresponds to ref_num in DTD table. Max 20 characters
RefCurrency	The currency that the amount is in
RateCode	Rate code to be used in case the account currency is not them same as the amount currency
Rate	The rate of conversion can be specified instead of a rate code. If both are specified, then the rate is used.
ReversalDate	The date in terms of number of days past BOD date.
Remarks	Remarks for the part-transaction. Max of 30 characters
Particulars	Particulars for the part-transaction. Max of 50 characters
CustomerId	Customer Id of the part-transaction if other than customer who owns the account
ReportCode	Valid report code for the part-transaction
PrintAdviceInd	Flag indicating whether advise to customer needs to be printed or not.
PtranBusinessType	This is specific to the MTT event. Each event will describe the additional processing that happens for special values of this field
Consolidate	Flag indicating if this part-transaction has to be consolidated with other part-transactions which have the same part-transaction identifier
ProxyPstgInd	Flag indicating whether this part-transaction is allowed to be proxy-posted ,if required.
TODRefType	Reference type of the TOD(which is set up at scheme level) that should be used in case of insufficient balance in the account. If this field is set, automatic TOD will be generated in case of shortfall.
CarveAmtFlg	If set to "Y", the system will automatically carve the amount during transaction creation which will be automatically removed when the part-transaction is posted. This is used to lock sufficient funds in the account, even if posting fails.
TranIdentifier	The identifier of the transaction to which this part-transaction belongs

OUTPUT

The return value will be ZERO (0) in case the transaction definition is accepted. If not, it will return 1.

EXAMPLE:

```
#The following defines a commission part transaction based on Bill Amount, Bill
Currency and Commission Code
if (MTT.InputDetails.CommissionCode != "") then
BANCS.INPARAM.InputAmount=MTT.InputDetails.BillAmount
BANCS.INPARAM.CurrencyCode=MTT.InputDetails.HomeCurrency
BANCS.INPARAM.AmountTableCode=MTT.InputDetails.CommissionCode
sv_r = urhk_B2k_GetASTMAmount("")
sv_z = CDOUBLE(BANCS.OUTPARAM.OutputAmount)
MTT.PartTranDetails.RefAmount=CDOUBLE(BANCS.OUTPARAM.OutputAmount)
MTT.PartTranDetails.Account=MTT.InputDetails.CommAcct
sv_z = MTT.InputDetails.CommAcct
MTT.PartTranDetails.RefCurrency=MTT.InputDetails.HomeCurrency
MTT.PartTranDetails.ValueDate="0"
MTT.PartTranDetails.Consolidate="N"
MTT.PartTranDetails.TranIdentifier="BillCharges"
MTT.PartTranDetails.PTranBusinessType="5"
sv_r = urhk_MTTTS_DefinePartTran("COMMISSION")
# "COMMISSION" is used to refer to this part transaction later.
endif
```

6.4 END THE DEFINITION OF A TRANSACTION/PART-TRANSACTION

SYNTAX

```
sv_a = urhk_MTTTS_EndDefinition ("" )
```

FUNCTIONALITY

This function signals the end of definition for a part-transaction or transaction. It is needed only if the script has a loop for processing multiple entities. This hook ensures that a part transaction is created in internal storage variables within the application based on the MTT.PartTranDetails class variables populated.

INPUT

There is no input to this function.

OUTPUT

The return value will be ZERO (0) in case the processing is successful. If not, it will return 1.

6.5 REPORTING AN ERROR CONDITION

SYNTAX

```
sv_a = urhk_MTTs_ReportErrorCondition ("" )
```

FUNCTIONALITY

This function allows the script to report an error condition, based on which the system will abort the processing of the current transaction(s). The handling of this condition is event-specific. For example, the error description is printed in the error report in the Interest Calculation event.

INPUT

There is no input string to this function.

All other inputs to this function is passed via repository variables in MTT. Error class.
The fields are:

Description	The description of the error condition
-------------	--

OUTPUT

The return value will be ZERO (0) in case the transaction definition is accepted. If not, it will return 1.

EXAMPLE:

```
....  
#sv_k contains interest to be recovered  
# If there is no balance and there is no operative account specified,  
# raise an error condition ...  
if (sv_k>CDOUBLE(MTT.InputDetails.firstCustAcctBal)) THEN  
    if (MTT.InputDetails.drOperAcctIfNoBalFlg=="N") THEN  
        MTT.Error.Description="Interest Skipped insuff.Bal"  
        sv_r = urhk_MTTs_ReportErrorCondition ("" )  
    ENDIF  
ENDIF  
ENDIF
```

6.6 CHECK IF CUSTOMER INSTRUCTION EXISTS

SYNTAX

```
sv_a = urhk_MTTs_CheckIfCustInstExists ("<LABEL>")
```

Label is not used within the user hook.

FUNCTIONALITY

This hooks checks for Customer instructions for a given account and instruction type. The hook returns 0 if customer instruction exists.

INPUT

There is no input string to this function.

All other inputs to this function is passed via repository variables in MTT. PayCollDetails class. The fields are:

AccountNo	The account number for which Customer instructions are to be checked(foracid)
PrB2kType	The event type for which the customer instructions have been set-up. Valid values are: 'IO' – Interest outflow of term deposits 'IP' – Interest payment of SB/CA/OD/CC accounts

OUTPUT

The return value will be ZERO (0) in case the customer instruction exists. If not, it will return 1.

EXAMPLE:

See section below

6.7 DISTRIBUTE AMOUNT AS PER CUSTOMER INSTRUCTION

SYNTAX

```
sv_a = urhk_ MTTS_DefineAsPerCustomerInstruction ("")
```

FUNCTIONALITY

This function allows the distribution of a given amount as per Customer instructions. There is a facility to check if customer instructions ave been entered for a given acccount and if so, distribute the amount as per the instructions. Remainder amount will be assigned to a default account.

INPUT

There is no input string to this function.

All other inputs to this function is passed via repository variables in MTT. PayCollDetails class. The fields are:

PayCollAmount	Amount to be distributed as per Customer instructions
PayCollCrncy	Currency code of the above amount
AccountNo	The account number for which Customer instructions are to be checked(foracid)
TranIdentifier	The identifier of the transaction to which this part-transaction belongs
PrB2kType	The event type for which the customer instructions have been set-up. Valid values are : 'IO' – Interest outflow of term deposits 'IP' - Interest payment of SB/CA/OD/CC accounts
RemainAmtAccountNo	Default account to which the remainder amount is to be put
RemTranPartclrs	Tran particulars of the default part-transaction. Max 50 characters
RemPTranRemarks	Tran remarks of the default part-transaction. Max 30 characters
RemPTranRefNum	Corresponds to ref_num in DTD table. Max 20 characters

OUTPUT

The return value will be ZERO (0) in case the transaction definition is accepted. If not, it will return 1.

EXAMPLE:

```

...
#Other portion of scripts earlier
# Populate PayColl Class before calling Check for Customer Inst.
MTT.PayCollDetails.PrB2kType = MTT.InputDetails.tdFlowId
MTT.PayCollDetails.RemainAmtAccountNo = MTT.InputDetails.ActualEntity

sv_s = urhk_MTTS_CheckIfCustInstExists("CHECK")

if (sv_s !=0)THEN
    sv_r = urhk_MTTS_DefinePartTran("CUST1")
ELSE
    MTT.PayCollDetails.PayCollCrncy = MTT.InputDetails.refCrncyCode
    MTT.PayCollDetails.AccountNo = MTT.InputDetails.ActualEntity
    MTT.PayCollDetails.PayCollAmount = MTT.PartTranDetails.RefAmount
    MTT.PayCollDetails.TranIdentifier = "MainInterest"
    MTT.PayCollDetails.PrB2kType = MTT.InputDetails.tdFlowId
    MTT.PayCollDetails.RemTranPartclrs = ""
    MTT.PayCollDetails.RemPTranRemarks = ""
    MTT.PayCollDetails.RemPTranRefNum = ""
    MTT.PayCollDetails.RemainAmtAccountNo = MTT.InputDetails.firstCustomerAcct
    sv_r = urhk_MTTS_DefineAsPerCustomerInstruction("PAY1")
Endif

```

6.8 MTTS_DEFINECHARGEPARTTRAN

You can use this script hook to initiate a transaction. This will put the relevant charge records also.

SYNTAX:

```
sv_a = urhk_MTTs_DefineChargePartTran(Variable)
```

The variable can be a scratch pad variable or a string. And this variable will be used as the Part Transaction identifier by MTT.

FUNCTIONALITY:

This function allows the script to initiate a financial transaction by defining some of its attributes along with the Charge Record. The attributes are first made available in the MTT.PartTranDetails class and MTT.ChargeDetails class. If fields from the MTT.ChargeDetails class related to the Charge Record available then this user hook will add the data pertaining to the charge record into the database.

INPUT:

The input required for this user hook will be the input required for urhk_MTTs_DefinePartTran. In addition to these details the following details will be required in MTT.ChargeDetails:

Field Name	Description	Data Type	Length
EventType	The event type for the account for the charge that is being calculated.	String	5
EventId	The event id for the account for the charge that is being calculated.	String	25
CollectionCrncy	The currency in which the charge is to be collected.	String	3
CalculatedAmount	The charge amount as calculated by the system.	String	35
ActualAmount	The actual charge amount that will be charged to the customer.	String	35
ServiceSolId	The solid id for the transaction.	String	8
ChrgPrtcls	The charge particulars for the charge details.	String	50
ChrgRmks	The charge remarks for the charge details.	String	30
PtranBusType	This is specific to the MTT event. Each event will describe the additional processing that happens for special values of this field.	String	5
RateCode1	The rate code that will be used in case the calculation and collection currency of the charge amounts are different.	String	5
ModifyFlg	This flag indicates whether the charges calculated by the system can be modified by the	Char	1

	user or not. The valid values are Y and N.		
ConsolidatedDr	This flag indicates whether the charges calculated by the system that has to be debited to customer account is to be consolidated or not. The valid values are Y and N.	Char	1
ChargeCustId	The customer Id for whom the charge is being calculated.	String	9
Rate1	The rate to be used in case the calculation and collection currency of the charge amounts are different.		35
ParentTranIdentifier	The Tran Identifier for the transaction	String	20
ParentPartTranIdentifier	The Part Tran identifier for the transaction.	String	20

OUTPUT:

The return value will be 0 in case the transaction definition is accepted. If not, it will return a non-zero value.

For a sample script refer to the sample scripts directory in \$TBA_PROD_ROOT/sample/scripts/ bcacctmaintchrg.sscr.

6.9 MTT'S_INITPTT

In Finacle charges are setup through the menu option PTTM. For a given Event ID & Event Type PTTM stores a set of records. This particular user hook will initialise the repository fields related to charges class , for given inputs access the data as set up in PTTM and create a linklist of PTT records. This user hook should be called before calling the user hook MTT'S_GetPttRec.

SYNTAX:

```
sv_a = urhk_MTT'S_InitPtt(Variable)
```

The variable can be a scratch pad variable or a string.

FUNCTIONALITY:

In Finacle charges are setup through the menu option PTTM. For a given Event ID & Event Type, PTTM stores a set of records. This particular user hook will initialise the repository fields related to charges class, for given inputs access the data as set up in PTTM and create a linklist of PTT records. This user hook should be called before calling the user hook MTT'S_GetPttRec (Description of this is given in section 1.1.2).

INPUT:

The input required for this user hook will be required in the class MTT.ChargeDetails. They are as follows :

Field Name	Description	Data Type	Length
EventType	The event type for the account for the charge that is being calculated.	String	5
EventId	The event id for the account for the charge that is being calculated.	String	25
ChrgInpCrncy	The input currency will be currency of the account for which the charge is being calculated.	String	3
ChrgInpAmount	The input amount based on which the charge will be calculated. Will be useful in cases where the PTM set up calculates charge using the percentage method.	String	35
PartyAccount	The account for which the charge is being calculated.	String	16

OUTPUT:

The return value will be 0 in case the initialisation is done. If not, it will be a non zero value. Also when the return value is 0, the following field will be populated in the class MTT.ChargeDetails

Field Name	Description	Data Type	Length
MultipleCrncyChrg	If this field has the value of 'Y', it means that the charges collected are in more than one currency. If the field has the value 'N' it means that the charges are in a single currency.	Character	1

For a sample script refer to the sample scripts directory in \$TBA_PROD_ROOT/sample/scripts/ bcacctmaintchrg.sscr.

6.10 MTTS_GETPTTREC

This user hook will be used to access the PTM records that would have been initialised by the user hook MTTS_InitPtt

SYNTAX:

```
sv_a = urhk_MTTS_GetPttRect(Variable)
```

The variable can be a scratch pad variable or a string.

FUNCTIONALITY:

Each call to this user hook will get the next record from the PTT link list that was initialised by calling the user hook MTTs_InitPtt.

INPUT:

There are no inputs for this user hook. This should be called only after calling the user hook MTTs_InitPtt.

OUTPUT:

The return value will be 0 if the call to the user hook is successful. If the call is not successful the return value will be 1.

If the return value is 0, the fields returned by the user hook are as given below in the class MTT.ChargeDetails:

Field Name	Description	Data Type	Length
EventType	The event type for the account for the charge that is being calculated.	String	5
EventId	The event id for the account for the charge that is being calculated.	String	25
CalculatedAmount	The charge amount as calculated by the system.	String	35
ActualAmount	The actual charge amount that will be charged to the customer.	String	35
CollectionCrncy	The currency in which the charge will be collected.	String	3
ServiceSolId	The solid id for the transaction.	String	8
ChrgPrtcls	The charge particulars for the charge details.	String	50
ChrgRmks	The charge remarks for the charge details.	String	30
PtranBusType	This is specific to the MTT event. Each event will describe the additional processing that happens for special values of this field.	String	5
RateCode1	The rate code that will be used in case the calculation and collection currency of the charge amounts are different.	String	5
ModifyFlg	This flag indicates whether the charges calculated by the system can be modified by the user or not. The valid values are Y and N	Char	1
ConsolidatedDr	This flag indicates whether the charges calculated by the system that has to be debited to customer	Char	1

Field Name	Description	Data Type	Length
	account is to be consolidated or not. The valid values are Y and N.		
ChargeCustId	The customer Id for whom the charge is being calculated.	String	9
The following fields are available in the MTT. PartTranDetails			
Account	The Account to which the charge amount is credited/debited as per the set up in PTM	String	16
Amount	The Charge amount as calculated by PTM	String	35
RefAmount	The Ref Amount for the transaction	String	35
ValueDate	The Difference between the Value date from the transaction date	Integer	
Refnum	The Ref Num for the part tran	String	20
RefCurrency	The reference currency for the transaction.	String	3
Rate	The Rate that is used for the part tran.	String	35
ReportCode	The report code for the part tran.	String	5
Remarks	The remarks for the part tran.	String	30
Particulars	The particulars for the part tran	String	50
PrintAdviceInd	Whether advice needs to be printed or not.	Character	1

OUTPUT:

The return value will be 0 in case the initialization is done. If not, it will be 1.

For a sample script refer to the sample scripts directory in \$TBA_PROD_ROOT/sample/scripts/ bcacctmaintchrg.sscr.

7 USER HOOKS TO RAISE EXCEPTIONS FOR FINANCIAL TRANSACTIONS

7.1 RAISE EXCEPTIONS FOR TRANSACTION CREATED THROUGH TM

This function will raise a transaction exception, given the exception code. In case the exception is set as an error, the error description is returned as an outparam field. It can be used in the context of any Online transaction Event or Workflow.

SYNTAX

sv_a = urhk_FTS_RaiseException (Variable)

The variable can be a scratch pad variable (sv_b) or a string("481").

FUNCTIONALITY

This function will raise an exception corresponding to the exception code passed, just in the manner the built-in exceptions are raised. In case the exception turns out to be an error, the error description is returned as an outparam field and the function returns a value of 1.

INPUT

The inputs to this function (through the INPARAM fields) are ExceptionIgnoreFlg, DrCrInd, ValMode.

The input through function argument is exception code.

OUTPUT

Return value is 1 if exception is an error, else 0.

This function will return the corresponding error description into the OUTPARAM class of BANCS repository if the exception code passed has been set as an error.

Field name in BANCS repository

BANCS.OUTPARAM.ErrorDesc

EXAMPLE:

In the TM_ENTRY event of the FinTran.scr script, if the following two checks need to be done for an account of SBFIX scheme :

If the transaction amount is greater than 1,00,000.00 then throw an error.

If the transaction amount is not a multiple of 1000, then raise an exception code, say 432.

The relevant part of the script will look like this :


```

<--start

BANCS.OUTPUT.successOrFailure = "S"
BANCS.OUTPUT.ErrorDesc = ""
BANCS.OUTPUT.AddtlDetails = ""
BANCS.OUTPUT.DeleteUAD = "Y"

if (BANCS.INPUT.Event == "TM_ENTRY") then
    # If the scheme is SBFIX
    if (BANCS.INPUT.SchemeCode == "SBFIX") then
        # If the amount is > 1 Lakh, throw error.
        sv_a = cdouble(BANCS.INPUT.TranAmt)
        if (sv_a > 100000) then
            BANCS.OUTPUT.successOrFailure = "F"
            BANCS.OUTPUT.ErrorDesc = "Amt exceeds 1 Lakh in SBFIX Scheme!"
            exitscript
        endif
    endif

    # If the amount is not multiple of 1000, raise an exception 432.
    sv_a = BANCS.INPUT.TranAmt
    sv_b = strlen(sv_a)
    sv_c = mid$(sv_a, sv_b - 6, sv_b)
    if (sv_c != "000.00") then
        BANCS.INPARAM.ValMode = BANCS.INPUT.ValMode
        BANCS.INPARAM.DrCrInd = BANCS.INPUT.DrCrInd
        BANCS.INPARAM.ExceptionIgnoreFlg = BANCS.INPUT.ExceptionIgnoreFlg
        sv_d = urhk_FTS_RaiseException("432")
        # If the return value is 1 then it is an error. Otherwise the
        # exception has been raised successfully.
        if (sv_d == 1) then
            # The exception is an error.
            BANCS.OUTPUT.successOrFailure = "F"
            BANCS.OUTPUT.ErrorDesc = BANCS.OUTPARAM.ErrorDesc
            exitscript
        endif
    endif
endif
endif
endif
endif

```

7.2 RAISE EXCEPTIONS FOR CLEARING TRANSACTIONS

This function raises an exception corresponding to the given exception code, during online lodging of outward clearing transactions. It will return the error description in case the exception code given was set as an error. It can be used in the context of Online clearing transaction Event (Event "OWCLG_LODGE" of FinTran.scr script).

SYNTAX

```
sv_a = urhk_OWCLG_RaiseException (Variable)
```

The variable can be a scratch pad variable (sv_b) or a string("482").

FUNCTIONALITY

This function will check if the given exception code is an error and if so, populates the error description in the OUTPARAM field with the appropriate error and returns 1. Otherwise, it will raise the exception just the same way the built-in exceptions are raised during lodging.

INPUT

The input to this function through the INPARAM field is "AcctNum". The input through the function argument is an exception code.

OUTPUT

Return value is 1, if exception was found to be an error. Else 0.

This function will return the corresponding error description into the OUTPARAM class of BANCS repository if the given exception was an error.

Field name in BANCS repository

BANCS.OUTPARAM. ErrorDesc

EXAMPLE:

Say during lodging, an exception (code 482) needs to be raised if the part transaction amount is not multiple of 1000 in SBFIX scheme code. This will be under the "OWCLG_LODGE" event of the FinTran.scr script.

```
<-- start

BANCS.OUTPUT.successOrFailure = "S"
BANCS.OUTPUT.ErrorDesc = ""
BANCS.OUTPUT.AddtlDetails = ""
BANCS.OUTPUT.DeleteUAD = "Y"

if (BANCS.INPUT.Event == "OWCLG_LODGE") then
    # If the amount is not multiple of 1000's in SBFIX scheme code, then raise
    # an exception
    if (BANCS.INPUT.SchemeCode == "SBFIX") then
        sv_a = BANCS.INPUT.TranAmt
        sv_b = strlen(sv_a)
        sv_c = mid$(sv_a, sv_b - 6, sv_b)

        if (sv_c != "000.00") then
            # fill the INPARAM
            BANCS.INPARAM.AcctNum = BANCS.INPUT.AcctNum
            sv_d = urhk_OWCLG_RaiseException("482")
            if (sv_d == 1) then
                # It is an error.
                BANCS.OUTPUT.successOrFailure = "F"
                BANCS.OUTPUT.ErrorDesc = BANCS.OUTPARAM.ErrorDesc
                exitscript
            endif
        endif
    endif
endif
exitscript
<-- end
```

8 USER HOOKS FOR QUEUING UP WORKFLOW ITEMS FROM WITHIN A NON-WORKFLOW SCRIPT

8.1 INITIALIZING THE WORK FLOW REPOSITORIES IN A SPECIFIC EVENT HOOK

In order for non-workflow scripts to have the capability of queuing up workflow Scripts, the userhooks **urhk_ EVTSCR_InitWFRepos** and **urhk_EVTSCR_CreateWFItem** have to be used as documented.

This function is used to initialize the workflow repositories before calling the user hook **urhk_EVTSCR_CreateWFItem** to create a work flow item during execution of a event hook script. This function needs to be called once in the script before calling the **urhk_EVTSCR_CreateWFItem** any number of times.

SYNTAX

```
sv_a = urhk_EVTSCR_InitWFRepos ("")
```

FUNCTIONALITY

This function will create and initialise the workflow repositories required for initiating a workflow item within an event specific script. This user hook is mandatory to be called before initiating a workflow from an event specific script.

INPUT

Input through INPARAM repository :

NONE

Input String as function argument :

NONE

OUTPUT

Output through OUTPARAM repository :

NONE

Function return value :

Always returns 0 as return value.

EXAMPLE:

Say for example, a work flow has to be initiated in the part tran posting event script, then the following piece of code illustrates the usage of the above user hook:

```

<--start

if (BANCS.INPUT.Event == "PTRAN_POSTING") then
# -----
# Call user hook to init the Work Flow repositories
# -----

sv_a = urhk_EVTSCR_InitWFRepos("")

# -----
# Create class to populate the data required
# to pass to the queued work flow menu option
# -----

sv_a = CLASSEXISTS("TEMPWFS", "GLBDATA")
if (sv_a == 0) then
    CREATECLASS("TEMPWFS", "GLBDATA", 5)
endif

# -----
# Populate the data
# -----

if (BANCS.INPUT.AddtlDetails != "") then
    sv_a = BANCS.INPUT.AddtlDetails
    sv_l = getposition(sv_a, "|")
    TEMPWFS.GLBDATA.ItemType = mid$(sv_a, 0, sv_l-1)
    sv_b = strlen(sv_a)
    sv_a = mid$(sv_a, sv_l, sv_b - sv_l)

    sv_l = getposition(sv_a, "|")
    TEMPWFS.GLBDATA.ItemId = mid$(sv_a, 0, sv_l-1)
    sv_b = strlen(sv_a)
    sv_a = mid$(sv_a, sv_l, sv_b - sv_l)

    sv_l = getposition(sv_a, "|")
    TEMPWFS.GLBDATA.NotifyDate = mid$(sv_a, 0, sv_l-1)
    sv_b = strlen(sv_a)
    sv_a = mid$(sv_a, sv_l, sv_b - sv_l)

    TEMPWFS.GLBDATA.ItemAmount = sv_a

    # -----
    # Create New workflow item
    # -----
    TEMPWFS.WFSOUTPUT.WorkFlowDesc="Part tran posting work Flow"
    TEMPWFS.WFSINPUT.WFItemPriority = "5"
    sv_b = "1|" + STDWFS.WFSINPUT.CurrentUserId + "|11|" + "FinTran.scr"
    sv_a = urhk_EVTSCR_CreateWFItem(sv_b)

    # The file FinTran.scr in the above statement is the script file name in
    # which the next step to execute as part of the workflow, is present

endif

endif
exitscript
<-- end

```

8.2 CREATING WORK FLOW ITEM FROM SPECIFIC EVENTS

In order for non-workflow scripts to have the capability of queuing up workflow Scripts, the userhook **urhk_EVTSCR_InitWFRepos** and **urhk_EVTSCR_CreateWFItem** have to be used as documented.

This function is used to create (initiate) a workflow item during execution of an event hook.

SYNTAX

```
sv_a = urhk_EVTSCR_CreateWFItem (Variable)
```

The variable can be a scratch pad variable (sv_b) or a string ("3|SBGEN|10")

FUNCTIONALITY

This function will queue a workflow item which can be executed at a future time. It could typically be used in some specific events like transaction posting event (event "PTRAN_POSTING" of FinTran.scr script) or the clearing regularization event (event "OWCLG_REGULARIZE" of FinTran.scr script) to initiate or queue-up a workflow item.

INPUT

Input through INPARAM repository:

NONE

Input through TEMPWFS

TEMPWFS.WFSINPUT. WfItemPriority – priority of the workflow item to be queued

TEMPWFS.WFSOUTPUT.WorkFlowDesc – Description of the workflow item to be queued

Input String as function argument :

contains three parts separated by a '|' character.

1 Step number – The step in the script from where this workflow item will start to execute.

2 User Id [Optional] – The userid who should handle the workflow item

3 User Work Group [Optional]. - The Work group who should handle the workflow item

Both User Id and User Work Group cannot be NULL. At least one of them should be given.

Example - "3|SBGEN|10"

If the user id is specified and not valid then an error will be given.

OUTPUT

Output through OUTPARAM repository :

NONE

Function return value :

Always returns 0 as return value.

EXAMPLE:

Refer to the example given in the **urhk_ EVTSCR_InitWFRepos** section.

9 USER HOOKS FOR BANCSCONNECT

9.1 DETERMINING IF THE BANCSCONNECT ONLINE FEES IS TO BE TREATED AS CHARGE INCOME OR NOT

This function will allow the user to specify which of the fee types expected in online BancsConnect messages have to be treated as charge income. These charges will appear in the charge income report generated through the menu option CHGIR. By default, the fees will not be treated as charges.

SYNTAX

```
sv_a = urhk_ B2k_CdciPopulateFeeInfoLL ("")
```

FUNCTIONALITY

This function will add a record in the linklist maintained for determining whether an online fee is to be treated as a charge. This decision is based on fee type.

It should be called from the script "CDGetFeeInfo".

INPUT

Input through INPARAM repository:

BANCS.INPARAM.FeeType : As expected in the online BancsConnect message.

BANCS.INPARAM.CXLInsertReqdFlag: Whether the fees is a charge or not. Valid values are 'Y' (Yes) and 'N' (No).

OUTPUT

Output through OUTPARAM repository:

NONE

Function return value:

0 – Linklist record addition successful.

1 – Linklist record addition unsuccessful.

EXAMPLE:

```
BANCS.OUTPUT.successOrFailure = "S"

BANCS.INPARAM.FeeType = "01"
BANCS.INPARAM.CXLInsertReqdFlag = "Y"
sv_r = urhk_B2k_CdciPopulateFeeInfoLL("")
if(sv_r == 1) then
    BANCS.OUTPUT.successOrFailure = "F"
    exitscript
endif

# Repeat above logic for all fee types
```

9.2 GENERATE HASHED NUMBER FOR BANCS CONNECT MESSAGES

SYNTAX

```
sv_a = urhk_B2k_generateHashedNoForMesg("mesg")
```

FUNCTIONALITY

This user hook will generate the hash number for the Bances Connect messages.

INPUT

CDCI message

OUTPUT

The user hook will return 0 on the success of the user hook . Output is stored in BANCS.OUTPARAM.HashValue.

EXAMPLE

```
<--start
#trace on
sv_a = "This is a test mesg"
sv_e = urhk_B2k_generateHashedNoForMesg(sv_a)
print(BANCS.OUTPARAM.HashValue)
#trace off
end-->
```

9.3 CREATE DCTI RECORD

SYNTAX

```
sv_a = urhk_CreateDCTIRecord("")
```

FUNCTIONALITY

This USER HOOK Create/Insert record into DCTI table for Delivery channel transaction Inquiry. Before inserting the record into the table it validates the Acct_id

passed. If there is any error in inserting the record into the table the error mesg is stored in outparam repository.

INPUT

BANCS.INPARAM.ReferenceNo
BANCS.INPARAM.DelChannelId
BANCS.INPARAM.DCDeviceId
BANCS.INPARAM.DCLocation
BANCS.INPARAM.DCMesgId
BANCS.INPARAM.TranDate
BANCS.INPARAM.DCUserId
BANCS.INPARAM.CustId
BANCS.INPARAM.SolId
BANCS.INPARAM.AcctNum
BANCS.INPARAM.TranType
BANCS.INPARAM.TranAmt
BANCS.INPARAM.Currency
BANCS.INPARAM.TranStatus
BANCS.INPARAM.TranRmks
BANCS.INPARAM.FreeCode1
BANCS.INPARAM.FreeCode2
BANCS.INPARAM.FreeText1
BANCS.INPARAM.FreeText2
BANCS.INPARAM.FreeText3

OUTPUT

This userhook will return 0 on successful completion and on failure it will return 1 and error mesg will be stored in BANCS.OUTPARAM.ErrMsg

EXAMPLE

```
<--start
trace on
BANCS.INPARAM.ReferenceNo="ATM12345545403200166556"
BANCS.INPARAM.DelChannelId="BTM"
BANCS.INPARAM.DCDeviceId="DD556"
BANCS.INPARAM.DCLocation="BANGALORE"
BANCS.INPARAM.DCMesgId="MSG6"
BANCS.INPARAM.TranDate="10-12-1998"
BANCS.INPARAM.DCUserId="USER1236"
BANCS.INPARAM.CustId="ACME"
BANCS.INPARAM.Solid="SCGL"
BANCS.INPARAM.AcctNum="SB4"
BANCS.INPARAM.TranType="A6"
BANCS.INPARAM.TranAmt="10000"
BANCS.INPARAM.Currency="INR"
BANCS.INPARAM.TranStatus="FINE"
BANCS.INPARAM.TranRmks="EXCELLENT"
BANCS.INPARAM.FreeCode1="FREE1"
BANCS.INPARAM.FreeCode2="FREE2"
BANCS.INPARAM.FreeText1="FREETEXT1"
BANCS.INPARAM.FreeText2="FREETEXT2"
BANCS.INPARAM.FreeText3="FREETEXT3"
sv_e = urhk_CreateDCTIRecord("")
print(sv_e)
if(sv_e != 0 ) then
    print(BANCS.OUTPUTPARAM.ErrMsg)
trace off
end-->
```

10 USER HOOKS TO GENERATE PAST DUE DETAILS INQUIRY/REPORT

10.1 GET PAST DUE RECORD

This user hook will return selected records based on selection criteria specified in PDADI menu option. This user hook is specific to menu option PDADI.

SYNTAX

```
sv_a = urhk_getPastDueRecord ("" )
```

sv_a: scratch pad variable to store the return value.

FUNCTIONALITY

Records selected based on FINACLE specific selection criteria specified in PDADI menu option will be populated into FINACLE repository fields. On first call to this user hook, the first selected record will be passed. On subsequent calls, the next selected record will be passed through repositories. A call after fetching all records will return failure (return value 1).

INPUT

OUTPUT

The various field values are populated into the following repositories. These field values will be overwritten with the new values on sub subsequent calls.

Description	Repository Field
Sol Set Id	BANCS.OUTPUTPARAM.setId
Account number	BANCS.OUTPUTPARAM.acctId
Account Currency code	BANCS.OUTPUTPARAM.acctCrncyCode
Customer Id	BANCS.OUTPUTPARAM.custId
Account scheme Code	BANCS.OUTPUTPARAM.schmCode
GI Sub Head Code	BANCS.OUTPUTPARAM.gISubHeadCode
Account sol Id	BANCS.OUTPUTPARAM.solId
Account manager user id	BANCS.OUTPUTPARAM.acctMgrId

EXAMPLE

```

<--start
.....
.....
sv_b = urhk_getPastDueRecord("")
while (sv_b == 0)
.....
.....
sv_a = BANCS.OUTPUTPARAM.acctId
sv_b = BANCS.OUTPUTPARAM.custId
sv_c = BANCS.OUTPUTPARAM.acctName
...
...
sv_j = "|"
sv_l = sv_a + sv_j + sv_b + sv_c
sv_r = urhk_putPastDueDispPrintString (sv_l)
sv_b = urhk_getPastDueRecord("")
do
sv_r = urhk_generatePastDueReport (sv_a)

# If urhk_generatePastDueReport function returns failure, populate error message and
exit out of script.
if ( sv_r == 1 ) then
BANCS.OUTPUT. errorMessage="Could not print Report"
exitscript
endif

end-->

```

☞ For more details refer *\$TBA_SAMPLE/scripts/pdainq.scr*.

10.2 PUT PAST DUE DISPLAY/PRINT STRING

This user hook will accept a string, which will either be displayed through PDADI menu option or dumped for report generation. This user hook is specific to PDADI menu.

SYNTAX

sv_a = urhk_putPastDueDispPrintString (Variable)

sv_a: scratch pad variable to store the return value

Variable: Scratch pad variable containing formatted string.

FUNCTIONALITY

A formatted string is accepted and the same will be displayed on multiple record screen or dump for report generation based on 'Display or Report' flag specified in PDADI menu option. This user hook may be called any number of times. The maximum length of the string can be 2500 characters.

The string passed for display will be cut to 150 characters and display the string in two lines of 75 characters each. The string passed for display should be a formatted one.

For generating report, string passed must contain individual fields separated by a '|' character. The number of fields in string must match with the record layout of the Maha Report Template being used for report generation.

INPUT

Formatted string.

OUTPUT

Return value will always be success ('0').

EXAMPLE

Example to build and pass a display string

```
#Get past due record and populate all required values into
#scratch pad variables.
```

```
sv_b = urhk_getPastDueRecord("")
while (sv_b == 0)
# Pad string with spaces
sv_r = sv_r + 1
sv_a = BANCS.OUTPARAM.setId
sv_a = rpad(sv_a, 8)
sv_c = BANCS.OUTPARAM.acctId
sv_d = BANCS.OUTPARAM.solId
sv_d = rpad(sv_d, 8)
sv_e = BANCS.OUTPARAM.schmCode
sv_e = rpad(sv_e, 5)
sv_f = BANCS.OUTPARAM.glSubHeadCode
sv_f = rpad(sv_f, 5)
sv_h = BANCS.OUTPARAM.acctCrncyCode
sv_h = rpad(sv_h, 3)
sv_i = BANCS.OUTPARAM.acctMgrId
sv_i = rpad(sv_i, 15)
sv_j = BANCS.OUTPARAM.custId
sv_j = rpad(sv_j, 9)
sv_k = rpad(sv_c, 16)

sv_l = urhk_getAcctDetailsInRepository(sv_c)
sv_m = BANCS.OUTPARAM.acctName
if (strlen(sv_m) >= 10) then
    sv_m = left$(sv_m, 10)
else
    sv_m = rpad(sv_m, 10)
endif

sv_p = BANCS.OUTPARAM.ClearBalance
sv_p = CDOUBLE(sv_p)
sv_x = sv_x + sv_p
sv_p = format$(sv_p, "%17.02f")
sv_l = urhk_getAdvanceDetailsForAccount(sv_c)
if (sv_l == 0) then
    sv_n = BANCS.OUTPARAM.PastDueXferDate
else
    sv_n = ""
endif
if (strlen(sv_n) >= 10) then
    sv_n = left$(sv_n, 10)
else
    sv_n = rpad(sv_n, 10)
endif

# Build the final string.
sv_t = " "
sv_g = sv_k + sv_t + sv_d + sv_t + sv_m + sv_t + sv_n

sv_z = urhk_putPastDueDispPrintString(sv_g)
sv_b = urhk_getPastDueRecord("")

do
```

Example to build and pass string for report generation

```
sv_b = urhk_getPastDueRecord("")
while (sv_b == 0)
# Pad string with spaces
sv_r = sv_r + 1
sv_a = BANCS.OUTPARAM.setId
sv_c = BANCS.OUTPARAM.acctId
sv_d = BANCS.OUTPARAM.solId
sv_e = BANCS.OUTPARAM.schmCode
sv_f = BANCS.OUTPARAM.glSubHeadCode
sv_h = BANCS.OUTPARAM.acctCrncyCode
sv_i = BANCS.OUTPARAM.acctMgrId
sv_j = BANCS.OUTPARAM.custId

sv_l = urhk_getAcctDetailsInRepository(sv_c)
sv_m = BANCS.OUTPARAM.acctName
sv_p = BANCS.OUTPARAM.ClearBalance

sv_l = urhk_getAdvanceDetailsForAccount(sv_c)
if (sv_l == 0) then
    sv_n = BANCS.OUTPARAM.PastDueXferDate
else
    sv_n = ""
```

```
endif
# Build the final string.
sv_t = "|"
sv_g = sv_a + sv_t + sv_c + sv_t + sv_d + sv_t + sv_e sv_t + sv_m + sv_t + sv_n

sv_z = urhk_putPastDueDispPrintString(sv_g)
sv_b = urhk_getPastDueRecord("")

do
    end →
```

10.3 GENERATE PAST DUE REPORT

This user hook will past due generate report using the report template specified. This user hook is specific to PDADI menu in print mode.

SYNTAX

```
sv_a = urhk_generatePastDueReport(Variable)
```

sv_a: scratch pad variable to store the return value

Variable: Scratch pad variable containing formatted string or maha report template name.

FUNCTIONALITY

This user hook may be used to generate additional reports in formats specified. This user hook is applicable only in print mode of PDADI menu option.

Before calling this function a few strings need to be passed to system through user hook **urhk_putPastDueDispPrintString**. These strings will be dumped into a spool file and report generation routine will be called. Report generated by this user hook will be in addition to default report generated in any case. The report generated will be put into print queue.

INPUT

Maha report template

OUTPUT

Return value will be success ('0') if reported is generated successfully.

EXAMPLE

Example to build and pass string for report generation

```
sv_b = urhk_getPastDueRecord("")
while (sv_b == 0)
# Pad string with spaces
sv_r = sv_r + 1
sv_a = BANCS.OUTPUTPARAM.setId
sv_c = BANCS.OUTPUTPARAM.acctId
sv_d = BANCS.OUTPUTPARAM.solId
sv_e = BANCS.OUTPUTPARAM.schmCode
sv_f = BANCS.OUTPUTPARAM.glSubHeadCode
sv_h = BANCS.OUTPUTPARAM.acctCrncyCode
sv_i = BANCS.OUTPUTPARAM.acctMgrId
sv_j = BANCS.OUTPUTPARAM.custId

sv_l = urhk_getAcctDetailsInRepository(sv_c)
sv_m = BANCS.OUTPUTPARAM.acctName
sv_p = BANCS.OUTPUTPARAM.ClearBalance

sv_l = urhk_getAdvanceDetailsForAccount(sv_c)
if (sv_l == 0) then
    sv_n = BANCS.OUTPUTPARAM.PastDueXferDate
else
    sv_n = ""
endif
# Build the final string.
sv_t = "|"
sv_g = sv_a + sv_t + sv_c + sv_t + sv_d + sv_t + sv_e sv_t + sv_m + sv_t + sv_n

sv_z = urhk_putPastDueDispPrintString(sv_g)
sv_b = urhk_getPastDueRecord("")
do

sv_r = urhk_generatePastDueReport("pastDue.mrt")

# If urhk_generatePastDueReport function returns failure,
# populate error message and exit out of script.
if ( sv_r == 1 ) then
BANCS.OUTPUT. errorMessage="Could not print Report"
exitscript
endif

end→
```

11 TRADE FINANCE RELATED HOOKS

11.1 GET USER ADDITIONAL DETAILS

SYNTAX

```
sv_r=urhk_B2k_GetUserAdditionalDetails("")
```

FUNCTIONALITY:

This function gets user additional detail for an entity say bill or forward contract.

INPUT:

The inputs are ModuleId and ModuleKey through BANCS.INPARAM class.

Valid values of Module Id are BIL, FEX, BG, FWC, and DC.

Module Key is a field to identify unique entity e.g. for a bill module id can be sol id and bill id concatenated with a slash '/' character.

OUTPUT

This function will return the user additional detail into AdditionalDetails field of OUTPARAM class of BANCS repository. Field can comprise of many values concatenated by pipe characters.

EXAMPLE:

Let us say that the bank has captured two remarks for each bill. Bank can get the value of these two fields through following script in BM menu option.

```
BANCS.INPARAM.ModuleId="BIL"
BANCS.INPARAM.ModuleKey="SOL1" + "/" + "BILL125"
sv_r=urhk_B2k_GetUserAdditionalDetails("")
if (sv_r == 0) then
    sv_a=BANCS.OUTPARAM.AdditionalDetails
    sv_l = getposition(sv_a, "|")
    BANCS.FRMVALUE.Remarks1 = mid$(sv_a, 0, sv_l-1)
    sv_b = strlen(sv_a)
    sv_a = mid$(sv_a, sv_l, sv_b - sv_l)
    BANCS.FRMVALUE.Remarks2=sv_a
else
    BANCS.FRMVALUE.Remarks1 = ""
    BANCS.FRMVALUE.Remarks2 = ""
endif
```

11.2 SAVE USER ADDITIONAL DETAILS

SYNTAX

```
sv_r=urhk_B2k_PutUserAdditionalDetails("")
```

FUNCTIONALITY:

This function adds or modifies user additional detail for an entity say bill or forward contract. Currently this function is enabled for following menu options BM, FBM, IRM, ORM, IDCM, ODCM, MNTFWC, CNCLFWC, EXTFWC.

INPUT:

The inputs are ModuleId, ModuleKey and AdditionalDetails through BANCS.INPARAM class.

Valid values of Module Id are BIL, FEX, BG, FWC, and DC.

Module Key is a field to identify unique entity e.g. for a bill module id can be sol id and bill id concatenated with a slash '/' character.

OUTPUT

None

EXAMPLE:

Let us say that the bank wish to put two remarks for each bill. Bank can put the value of these two fields through following script in BM menu option.

```
BANCS.INPARAM.ModuleId="BIL"
```

```
BANCS.INPARAM.ModuleKey="SOL1" + "/" + "BILL125"
```

```
BANCS.INPARAM. AdditionalDetails = "REAMRKS1" + "REMARKS2"
```

```
sv_a=urhk_B2k_PutUserAdditionalDetails("")
```

11.3 RAISING EXCEPTIONS

SYNTAX


```
sv_r=urhk_B2k_PutUserException (ExceptionCode)
```

FUNCTIONALITY:

Using this function exceptions can be raised at verify time in following menu options
BM, FBM, IRM, ORM, IDCM, ODCM, MNTFWC, CNCLFWC, EXTFWC.

INPUT:

The input is exception code. Exception code must be a valid exception code setup by using EXCDM.

OUTPUT

None

EXAMPLE:

In case bank wish to raise an exception whenever forward contract is booked for USD
(as from currency)

```
<--start
sv_a = urhk_TBAF_InquireFieldValue("databl1.from_crncy_code")
sv_l = B2KTEMP.TEMPSTD.TBAFRESULT
if (sv_l == "USD") then
    sv_a = urhk_B2k_PutUserException("FWU")
endif
exitscript
end-->
```

11.4 GETTING ADDITIONAL INPUT DETAILS IN MTT

SYNTAX

```
sv_r = urhk_MTTS_PopulateSubInputDetails("")
```

FUNCTIONALITY:

Using this function multiple additional inputs can be obtained in MTT script. Currently this is enabled in FBM/IRM/ORM to get conversion screen details. Using this function total credit amount to customer can be obtained.

INPUT:

None

OUTPUT

Following fields gets populated in PartTranDetails Class of MTT repository in case of FBM/IRM/ORM menu option.

RefAmount, RefCurrency, Amount, TranCurrency, PtranBusinessType, RefPtranSrlNum, ProtectValue, Rate, Ratecode, Particulars, Remarks, ValueDate (diff from bod date) and Refnum

EXAMPLE:

```
sv_r = urhk_MTTs_PopulateSubInputDetails("")
while(sv_r < 1)
    print (MTT.PartTranDetails.RefAmount)
    print (MTT.PartTranDetails.PTranBusinessType)
    MTT.PartTranDetails.Consolidate="N"
    MTT.PartTranDetails.TranIdentifier="FBCharges"
    sv_r = urhk_MTTs_PopulateSubInputDetails("")
do
```

11.5 FXBILL_GETTRAN

SYNTAX

```
sv_a = urhk_FxBill_GetTran("")
```

FUNCTIONALITY

This assumes that there is a linklist available globally containing the part tran details as glb_fxb_main_ptr->tran_ll , if it does not exist then it will return fatal error. We can call this userhook in a loop so that it'll fetch one record at a time and will put the details into OUTPARAM class into the following fields

MTT_PTRAN_ACCOUNT
MTT_PTRAN_REFAMOUNT
MTT_PTRAN_AMOUNT
MTT_PTRAN_REFCURRENCY
MTT_PTRAN_CURRENCY
MTT_PTRAN_BUSINESSTYPE
MTT_PTRAN_REFPTRANSRLNUM
MTT_PTRAN_PROTECT_VALUE
MTT_PTRAN_RATECODE
MTT_PTRAN_CONVRATE
MTT_PTRAN_PARTICULARS
MTT_PTRAN_REMARKS

MTT_PTRAN_INST_TYPE

MTT_PTRAN_INST_DATE

MTT_PTRAN_INST_NUM

MTT_PTRAN_VALDATE

MTT_PTRAN_REPORTCODE

MTT_PTRAN_REFNUM

These part tran details can be used to post the part tran.

INPUT

none

OUTPUT

None.

EXAMPLE

11.6 FXBILL_INITTRAN

SYNTAX

```
sv_a = urhk_FxBill_InitTran("")
```

FUNCTIONALITY

This userhook Sets fxUsrHkPtr to NULL which is used by FxBill_GetTran.

INPUT

none

OUTPUT

None.

EXAMPLE

12 DSA RELATED USER HOOKS

12.1 URHK_GETDSATURNOVERDETAILS

This user hook gives the DSA turnover details for the given dsa id, product code, crncy code and as on date. The inputs are to be separated by a delimiter '|'.
`|`

12.2 URHK_GETDSAMASTERDETAILS

This user hook gives the DSA master details for the given DSA Id.

12.3 URHK_GETDSAPRODUCTDETAILS

This user hook gives the DSA Product details for the given DSA Id, Product code and crncy

12.4 URHK_GETDSAPARAMDETAILS

This user hook gives the DSA Parameter details

12.5 URHK_GETPASTDUECYCLES

This user hook gets the 10 past due cycles given an A/c.

12.6 UPDATE DSA TURNOVER DETAILS

SYNTAX

```
sv_a = urhk_acopDsatoUpdate("")
```

FUNCTIONALITY

An account may be sourced/disbursed through DSA (direct selling agent). Every DSA has a turnover history maintained, i.e. how many accounts opened, how many accounts closed during a particular duration.

This DSA turnover is updated as a batch job depending on the frequency. so, if, say in January, 23 accounts were opened and 20 accounts were closed, the same is stored in the turnover table on the day the job runs, say 1st Feb.

However, if a value dated account opening is done on 10th feb with acct open date as 11th Jan, it will not figure in the turnover table.

For this purpose at the time of account opening this user hook is provided. The purpose of this hook is to update the turnover at the time of acct opening itself when the value dated acct opening is done and the turnover has been already inserted by the batch program.

The same is the case for closure of accounts

If the opening is value dated but the turnover job for the day is not yet run, the hook will do nothing, as this opening will be picked up when the job runs next.

INPUT

```

usrhk_getval(TBAFATAL, "inpSourceDealCode", usrhk_SourceDealCode,
sizeof(dsaIdType));
usrhk_getval(TBAFATAL, "inpSchmCode", usrhk_SchmCode,
sizeof(schemeCodeType));
usrhk_getval(TBAFATAL, "inpAcctOpnDate", usrhk_AcctOpnDate,
sizeof(tbaDateType));
usrhk_getval(TBAFATAL, "inpAcctCrncyCode", usrhk_AcctCrncyCode,
sizeof(currencyCodeType));
usrhk_getval(TBAFATAL, "inpSanctLim", usrhk_SanctLimStr,
sizeof(tbaAmtStrType));
usrhk_getval(TBAFATAL, "inpDrwngPower", usrhk_DrwngPowerStr,
sizeof(tbaAmtStrType));
usrhk_getval(TBAFATAL, "inpIncOrDecFlag", SingleCharStr, 2);

```

OUTPUT

output parameters are

errMesg - can be non null if successOrFailure is "F"

and successOrFailure - can be "S" or "F"

EXAMPLE

Pls check the example B2K_AcopDsatoUpdate.sscr in sample/scripts directory

12.7 GET TOTAL SANCTION LIMIT OF ALL ACCOUNTS LINKED TO A DSA

SYNTAX

```
sv_a = urhk_ getCPCAcctSanclLimitForDSA ()
```

FUNCTIONALITY

This user hook gives the total sanction limit of all accounts linked to the DSA which are either closed/ charged off/ paid off for the given dsa id, product code, crncy code, start date and end date. The inputs are to be separated by a delimiter `|`.

INPUT

DSA Id, Product code, Currency code, Start date and End Date.

OUTPUT

The output is available in BANCS.OUTPARAM. totSanclimit

EXAMPLE

```
# Get the inputs from repository

sv_a = BANCS.INPUT.dsaId
sv_b = BANCS.INPUT.productCode
sv_c = BANCS.INPUT.crncyCode
sv_d = BANCS.INPUT.startDate
sv_e = BANCS.INPUT.endDate

sv_f = sv_a + "|" + sv_b + "|" + sv_c + "|" + sv_d + "|" + sv_e

#call the user hook for getting total Sanction Limit
sv_n = urhk_getCPCAcctSanclimitForDSA (sv_f)

Output will be in this field: BANCS.OUTPARAM.totSanclimit
```

12.8 GET DSA TURNOVER DETAILS FOR A SCHEME TYPE

SYNTAX

```
sv_a = urhk_getDSATODetailsForSchemeType ()
```

FUNCTIONALITY

This user hook gives the DSA turnover details for the given dsa id, scheme type, crncy code and as on date. The inputs are to be separated by a delimiter '|'.

INPUT

DSA Id, Scheme type, Currency code and As on Date

OUTPUT

DSA Turnover details: Self and Others.

The Self-Turnover details will be available only if "DSATOInd_S" is set to YES. Otherwise other fields should not be accessed.

BANCS.OUTPARAM. DSATOInd_S

BANCS.OUTPARAM. dsaId_S

BANCS.OUTPARAM. schemeType_S

BANCS.OUTPARAM. crncyCode_S

BANCS.OUTPARAM. turnoverInd_S

BANCS.OUTPARAM. endDate_S

BANCS.OUTPARAM. numOfDrTrans_S

BANCS.OUTPARAM. numOfCrTrans_S

BANCS.OUTPARAM. totDrBalAmt_S
BANCS.OUTPARAM. totCrBalAmt_S
BANCS.OUTPARAM. highDrBalAmt_S
BANCS.OUTPARAM. highCrBalAmt_S
BANCS.OUTPARAM. lowDrBalAmt_S
BANCS.OUTPARAM. lowCrBalAmt_S
BANCS.OUTPARAM. cumDrTotAmt_S
BANCS.OUTPARAM. cumCrTotAmt_S
BANCS.OUTPARAM. lastRunCumBal_S
BANCS.OUTPARAM. lastRunAvgBal_S
BANCS.OUTPARAM. cumCurBal_S
BANCS.OUTPARAM. cumOpenBal_S
BANCS.OUTPARAM. cumCloseBal_S
BANCS.OUTPARAM. totSanctionLim_S
BANCS.OUTPARAM. cumTotSanctionLim_S
BANCS.OUTPARAM. totDrawingPower_S
BANCS.OUTPARAM. cumTotDrawingPower_S
BANCS.OUTPARAM. numOfAcctsOpened_S
BANCS.OUTPARAM. totNumOfAcctsOpened_S
BANCS.OUTPARAM. numOfAcctsClosed_S
BANCS.OUTPARAM. totNumOfAcctsClosed_S
BANCS.OUTPARAM. clsAcctsElgForPenal_S
BANCS.OUTPARAM. totClsAcctsElgForPenal_S

The Others-Turnover details will be available only if “DSATOInd_O” is set to YES.
Otherwise other fields should not be accessed.

BANCS.OUTPARAM. DSATOInd_O
BANCS.OUTPARAM. dsaId_O
BANCS.OUTPARAM. schemeType_O
BANCS.OUTPARAM. crncyCode_O
BANCS.OUTPARAM. turnoverInd_O
BANCS.OUTPARAM. endDate_O
BANCS.OUTPARAM. numOfDrTrans_O
BANCS.OUTPARAM. numOfCrTrans_O

BANCS.OUTPUTPARAM. totDrBalAmt_O
BANCS.OUTPUTPARAM. totCrBalAmt_O
BANCS.OUTPUTPARAM. highDrBalAmt_O
BANCS.OUTPUTPARAM. highCrBalAmt_O
BANCS.OUTPUTPARAM. lowDrBalAmt_O
BANCS.OUTPUTPARAM. lowCrBalAmt_O
BANCS.OUTPUTPARAM. cumDrTotAmt_O
BANCS.OUTPUTPARAM. cumCrTotAmt_O
BANCS.OUTPUTPARAM. lastRunCumBal_O
BANCS.OUTPUTPARAM. lastRunAvgBal_O
BANCS.OUTPUTPARAM. cumCurBal_O
BANCS.OUTPUTPARAM. cumOpenBal_O
BANCS.OUTPUTPARAM. cumCloseBal_O
BANCS.OUTPUTPARAM. totSanctionLim_O
BANCS.OUTPUTPARAM. cumTotSanctionLim_O
BANCS.OUTPUTPARAM. totDrawingPower_O
BANCS.OUTPUTPARAM. cumTotDrawingPower_O
BANCS.OUTPUTPARAM. numOfAcctsOpened_O
BANCS.OUTPUTPARAM. totNumOfAcctsOpened_O
BANCS.OUTPUTPARAM. numOfAcctsClosed_O
BANCS.OUTPUTPARAM. totNumOfAcctsClosed_O
BANCS.OUTPUTPARAM. clsAcctsElgForPenal_O
BANCS.OUTPUTPARAM. totClsAcctsElgForPenal_O

EXAMPLE

```
# Get the inputs from repository

sv_a = BANCS.INPUT.dsaId
sv_b = BANCS.INPUT.schemeType
sv_c = BANCS.INPUT.crncyCode
sv_d = BANCS.INPUT.asOnDate

sv_e = sv_a + "|" + sv_b + "|" + sv_c + "|" + sv_d

#call the user hook for getting DSA turnover details
sv_n = urhk_getDSATODetailsForSchemeType (sv_e)

IF (sv_n != 0) THEN
    BANCS.OUTPUT.successOrFailure="F"
BANCS.OUTPUT.error= "DSA Turnover details not found"
ENDIF
```


13 DATE FUNCTIONS

13.1 ADDING DAYS AND MONTHS IN A GIVEN DATE

SYNTAX

```
Sv_a = ("old_date" + "|" + "no_of_months" + "|" + "no_of_days")
```

```
Sv_b = urhk_B2k_Add_MonthsDays_To_date(sv_a)
```

FUNCTIONALITY

This user hook will give the new date after adding the month and date in old date.

INPUT

Input is a char string.

OUTPUT

The user hook will return 0 on the success.
BANCS.OUTPARAM.MODIFIED_DATE will contain the modified date.

EXAMPLE

```
<--start
#trace on
sv_a = "10"
sv_b = "12"
sv_d = "10-10-1998"
sv_f = sv_d + "|" + sv_a + "|" + sv_b
sv_e = urhk_B2k_Add_MonthsDays_To_date(sv_f)
if ( sv_e == 1 ) then
    print("Date is invalid")
else
    print(BANCS.OUTPARAM.MODIFIED_DATE)
#trace off
end-->
```

13.2 DIFFERENCE BETWEEN TWO DATES

SYNTAX

```
sv_a = urhk_B2k_dateDiff("")
```

FUNCTIONALITY

This user hook will give the difference between two dates in terms of no of days.

INPUT

1. BANCS.INPARAM.date1
2. BANCS.INPARAM.date2

OUTPUT

The user hook will return 0 on the success of the user hook . Difference of date is stored in BANCS.OUTPUTPARAM.dateDiff.

EXAMPLE

```
<--start
#trace on
BANCS.INPARAM.date1 = "10-12-1999"
BANCS.INPARAM.date2 = "10-10-1998"
sv_e = urhk_B2k_dateDiff("")
print(BANCS.OUTPUTPARAM.dateDiff)
#trace off
end-->
```

13.3 GETNEXTWORKINGDATE**SYNTAX**

```
Sv_e = "Sol_id" + "|" + "Date"
sv_a = urhk_TBAF_Confirm(sv_e)
```

FUNCTIONALITY

This user hook will return 0 on success and put the next and previous working dates into the repository. To calculate the next or previous working date it will consider the HOL (holiday) table to exclude the holidays. This will return the dates corresponding to Sol_id and datacentre.

INPUT

Input to userhook will be a string which will contain the sol_id and date.

OUTPUT

This userhook will return 0 on success.

The next and previous dates will be stored in repository as

BANCS.OUTPUTPARAM.NextWorkingDate

BANCS.OUTPUTPARAM.PrevWorkingDate

BANCS.OUTPUTPARAM.NextDataCentreWorkingDate

BANCS.OUTPUTPARAM.PrevDataCentreWorkingDate

EXAMPLE

```
<--start
#trace on
sv_a = "SCGL" + "|" + "19-12-1998"
sv_e = urhk_GetNextWorkingDate(sv_a)
print(sv_e)
print(BANCS.OUTPUTPARAM.NextWorkingDate)
print(BANCS.OUTPUTPARAM.PrevWorkingDate)
print(BANCS.OUTPUTPARAM.NextDataCentreWorkingDate)
print(BANCS.OUTPUTPARAM.PrevDataCentreWorkingDate)
#trace off
end-->
```

13.4 GETTING THE HOLIDAY STATUS FOR A GIVEN DATE**SYNTAX**

```
sv_r = urhk_TBAF_GetHolidayStatus(sv_a)
```

Where sv_a is a input of the format "SOL_ID | date in DD-MM-YYYY format"

FUNCTIONALITY

Using this user hook, we can get the holiday status of the date passed. The system checks the local calender (with the SOLID and DATE passed) and gives back the holiday status.

The output is available in BANCS.OUTPUTPARAM.holidayStatus variable. The valid values for the field will be "N", "Y" and "X".

Value for holidayStatus	Meaning
"N"	The date passed is not a holiday
"Y"	The date passed is a holiday
"X"	Record not found for the date passed in the calender.

INPUT

Input in the format "SOLID| date in DD-MM-YYYY"

OUTPUT

The output is available in BANCS.OUTPUTPARAM.holidayStatus variable. The valid values are "N", "Y" and "X"

EXAMPLE

This is for checking whether the due date given in scheme details screen of PCA account opening lies on holiday or not.If it lies on holiday, just give a warning message.

```
sv_a = urhk_TBAF_InquireFieldValue("ccodblk2.due_date")

if (B2KTEMP.TEMPSTD.TBAFRESULT == "") then
    exitscript
endif

sv_b=B2KTEMP.TEMPSTD.TBAFRESULT
sv_c = urhk_TBAF_InquireFieldValue("funcblk.sol_id_2")
sv_d=B2KTEMP.TEMPSTD.TBAFRESULT
sv_e = sv_d + "|" + sv_b
sv_f = urhk_TBAF_GetHolidayStatus(sv_e)
if (BANCS.OUTPARAM.holidayStatus == "Y") then
    sv_d = urhk_TBAF_DispMesg(" Warning :Due date lies on Holiday ")
endif
```

****{}****

14 INDEX OF USER HOOKS

urhk_B2k_GetAccountsForInstrument	
Get List of Amounts based on an Instrument	16
urhk_B2k_CdciPopulateFeeInfoLL	132
urhk_B2k_valBankCode	
validate a Bank Code.....	17
urhk_generatePastDueReport.....	139
urhk_MTTTS_DefineAsPerCustomerInstruction.....	119
urhk_MTTTS_DefinePartTran.....	115
urhk_B2k_BacidToAcctId	
bacid to Account Id and Name	11
urhk_B2k_checkForPipeChar	41
urhk_B2k_ConvertAmount	
Convert an Amount from One Currency to another.....	12
urhk_B2k_date_add	
date Add.....	23
urhk_B2k_generateHashedNoForMesg(“mesg”).....	133
urhk_B2k_GetASTMAmount	
Get amount calcul	15
urhk_B2k_getDateDiffFromBODDate	44
urhk_B2k_GetUserAdditionalDetails	141
urhk_B2k_getYear	
get 4 Digit year.....	23
urhk_B2k_isStringNumeric	
Validate Numeric	20
urhk_B2k_PutUserAdditionalDetails.....	142
urhk_B2k_PutUserException.....	143
urhk_B2k_RoundOff	
Roundoff An Amount.....	13
urhk_B2K_ShowParamAcptFrm	79
urhk_B2k_valBankBranchCode	
Validate Bank and Branch Code	18
urhk_B2k_valCashAccount	41
urhk_B2k_valCrncy	44
urhk_B2k_valFCNRAccount	42
urhk_B2k_valHOAccount.....	43
urhk_B2k_valHoTranCode	
Validate HO Transaction Code.....	19
urhk_B2k_validateAmtWithCrncy.....	43
urhk_B2k_validateInstrmntDate	47
urhk_B2k_valSystemAccount.....	42
urhk_B2k_valTranRptCode	45
urhk_createUserDefndTOD	60
urhk_dbSelect	
Selection of Fields from database.....	24
urhk_dbSQL	
Run a SQL Statement.....	25
urhk_EVTSCR_CreateWFItem	130
urhk_EVTSCR_InitWFRepos.....	129
urhk_FTS_RaiseException	126
urhk_getAcctBalanceAsOnDate	
Get Account Balance as on a date	35
urhk_getAcctDetailsInRepository	
get Account details into BANCS Repository	4
urhk_getAdvanceDetailsForAccount	

get Advance Details for an account	26
urhk_getAssetClassfnDetailsForAccount	
get Asset Classification for an Account	31
urhk_GetCTCDetailsFromTranCode	105
urhk_getCustomerDetails	
Get customer details into BANCS repository	28
urhk_getDDSchemeCrncyDetails	45
urhk_getFileLocation	
Getting the location of a File in the path	9
urhk_getInterestDetailsForAccount	
Get Interest Details for an account	33
urhk_getInterestRateForAccount	
get Interest rate for an Account	36
urhk_GetNextNumber	61
urhk_getOutClgInstDetails	109
urhk_getOutClgPtranDetails	112
urhk_getOutZoneHeaderDetails	107
urhk_getPastDueRecord	135
urhk_getSchemeDetailsInRepository	
get Scheme Details into BANCS Repository	8
urhk_getSecurityDetailsForAccount	39
urhk_getSumCrDrTranAmtsForPeriod	
Sum credit & Debit Transactions over a period	37
urhk_getSumTranAmtForAcctLabel	38
urhk_GetZoneHeaderDetails	106
urhk_MTTTS_CheckIfCustInstExists	118
urhk_MTTTS_DefineChargePartTran	121
urhk_MTTTS_DefineTransaction	114
urhk_MTTTS_EndDefinition	117
urhk_MTTTS_GetPttRect	123
urhk_MTTTS_InitPtt	122
urhk_MTTTS_PopulateInputDetails	114
urhk_MTTTS_PopulateSubInputDetails	143
urhk_MTTTS_ReportErrorCondition	118
urhk_OWCLG_RaiseException	127
urhk_putPastDueDispPrintString	136
urhk_PutUDTXN_into_IBR	48
urhk_SE_Dispositories	
Display Repository Fields	26
urhk_TBAF_ChangeFieldAttrib	72
urhk_TBAF_ChangeFieldValue	68
urhk_TBAF_DisMesg	85
urhk_TBAF_GetCurFldName	85
urhk_TBAF_GetValue	74
urhk_TBAF_InquireFieldValue	69
urhk_TBAF_SetAttrib	64
urhk_TBAF_SetKeyScript	86
urhk_TBAF_SetReplayKey	75
urhk_TBAF_SetValue	66
urhk_valAcctNumber	
validating the Account Number	3
urhk_WFS_CallMenuOption	88
urhk_WFS_CheckPoint	99
urhk_WFS_CreateWFItem	90
urhk_WFS_SetWorkGroup	101
urhk_WFS_ShowParamAcptFrm	95
urhk_WFS_TransferUser	94